



Chevron Attention - Dual Opposition in Neural Network Transformers

A series of [substack posts](#) applying [Chevron Networks](#) with dual A and N channels and 2 times 2 mixing (via 4 weights) to the Q,K/V Transformer ([Attention Is All You Need](#)). Continuous learning without catastrophic forgetting and wider AI safety are the motivations.

We build on earlier work on constraint Chevron Networks, applying the concept to predictive architectures in two explorative posts, before we dive into IDL and Chevron Attention. You may wish to skip straight to IDL on first reading and start with the 4th post.

Andre Kramer,
July 2026



Chevron Predictive Architecture

Why World Models Need Retained Difference

[Andre Kramer](#)

May 20, 2026



Yann LeCun's JEPA architecture makes a powerful wager: perhaps world models can be learned by predicting missing or future representations in

latent space, rather than by reconstructing raw pixels or memorizing linguistic tokens.

That is an important shift.

A system does not need to predict every blade of grass, every photon, every texture. It needs to predict what matters: the hidden structure, the persistent object, the likely continuation, the affordance, the constraint. JEPA's elegance lies in this abstraction. It says: do not predict the world as sensation; predict the world as representation.

But there is a deeper question.

Is prediction enough?

A world model is not merely a machine that guesses what comes next. It is a retained structure of expectations, constraints, distinctions, and affordances that survives repeated contact with the world. It is not just a predictor. It is a memory of what continues to matter.

This post proposes a possible extension of the JEPA idea: **Chevron Predictive Architecture**.

The central claim is simple:

Prediction gives the gradient. Inertia gives the world.

The JEPA Wager

[JEPA](#) — Joint Embedding Predictive Architecture — does not try to reconstruct sensory input directly. Instead of predicting missing pixels, it predicts missing or future representations.

In simplified form:

`context representation → predicted target representation`

This is a powerful idea. It avoids wasting effort on irrelevant details. If part of an image is hidden, the system does not need to predict the exact

texture of every surface. It needs to infer the meaningful structure: object continuation, spatial relation, category, motion, or possibility.

In this sense, JEPA is closer to how animals seem to learn. A cat does not need to generate a photorealistic image of the other side of the sofa. It needs to know whether the mouse might be there.

So JEPA shifts the problem from raw generation to latent compatibility.

But this raises the central question:

Does predicting latent targets automatically produce a world model?

Perhaps not.

Prediction tells us that something follows from something else. But a world model requires more than following. It requires retention. It requires knowing which differences are stable, which are accidental, which matter for action, and which should be ignored.

Prediction can generate error.

Error can generate difference.

But only retention produces a world.

Prediction Is Not Retention

A predictive system can learn correlations. It can learn that one representation tends to follow another. It can learn that one hidden patch is compatible with a visible context.

But a world model must do more.

It must retain object persistence.

It must distinguish signal from noise.

It must preserve causal regularities.

It must learn affordances.

It must remember failed expectations.

It must survive distribution shift.

It must know what remains true when the surface changes.

This is not just prediction. It is the stabilization of useful difference.

That phrase matters: **useful difference**.

Not every difference matters. Most differences are noise. Not every prediction error deserves structural change. Many errors are local, accidental, or misleading. A world model forms only when certain differences are repeatedly selected, stabilized, and retained.

This is where I think JEPA may need a richer training story.

The problem is not that JEPA is wrong. The problem is that latent prediction may be only one part of world-model formation. It gives the system a pressure toward compatibility, but it does not by itself explain how the system decides which differences become durable structure.

A world model is not simply learned.

It is sedimented.

Biology Does Not Learn From Scratch

Biological systems do not begin as blank predictive encoders.

Animals inherit bodies, sensors, reflexes, drives, developmental sequences, ecological niches, and species-level priors. A newborn

organism is not an empty JEPa waiting for data. It is already the result of a long evolutionary search.

Evolution has already discovered a great deal before individual learning begins:

- which sensory distinctions are likely to matter,
- which movements are possible,
- which bodily needs are urgent,
- which environmental regularities are worth tracking,
- which forms of plasticity are safe,
- which structures should change quickly,
- which should change slowly.

An organism does not merely learn from the environment. It is built by an evolutionary history that has already shaped how the environment will be sampled, parsed, and acted upon.

This is an important correction to any overly simple claim that world models are “easily learned.”

They may be learned efficiently by animals because animals are not starting from zero.

They are born inside retained structure.

Evolution does not knowingly design intelligence. But it does retain what has worked. It builds on inertia. It does not search the space of possible organisms afresh every generation. It modifies what already exists.

That is the clue.

A world model may need not only prediction, but **inertial retention**.

Inertial VSR

The evolutionary pattern is usually described as:

variation → selection → retention

Variation produces difference.

Selection tests difference.

Retention preserves difference.

But real evolution is not a clean reset at each cycle. It is inertial. Existing structures constrain what variations are possible. What has been retained becomes the ground for what can vary next.

This gives us a general learning principle:

Learning should not merely reduce error. It should decide which differences are worth retaining.

That principle is what I call **Inertial VSR**.

In a learning system, variation might appear as exploration, perturbation, gradient movement, masking, alternative predictions, or competing internal proposals.

Selection might appear as prediction error, action success, robustness, recurrence, or compatibility with existing structure.

Retention might appear as memory, consolidation, slower weights, replay, habits, norms, or world-model structure.

This reframes learning.

The question is not only:

What prediction minimizes error now?

The deeper question is:

Which predictive differences remain worth keeping?

Enter the Chevron

[Chevron Networks](#) begin with a simple architectural move.

Instead of treating a node or representation as a single scalar or vector, split it into two interacting channels:

A = adaptive / active / proposal channel
N = normative / retained / constraint channel

The A channel is fast, plastic, present-facing.

The N channel is slower, stabilizing, past-facing.

A proposes.

N constrains.

Their mismatch becomes meaningful.

The point is not that A is “good” and N is “bad,” or that one is conscious and the other unconscious. The point is that learning needs an internal tension between what changes quickly and what changes slowly.

A system that changes too quickly forgets.

A system that changes too slowly cannot adapt.

A system in which both channels collapse into sameness has no productive internal difference.

The Chevron idea is that intelligence may require a structured opposition between adaptation and retention.

In a predictive architecture, this becomes especially interesting.

A can learn what works now.

N can retain what keeps working.

Prediction can test their compatibility.

This suggests a new form:

Chevron Predictive Architecture — a JEPA-like architecture in which latent prediction is mediated by fast/slow A/N dynamics.

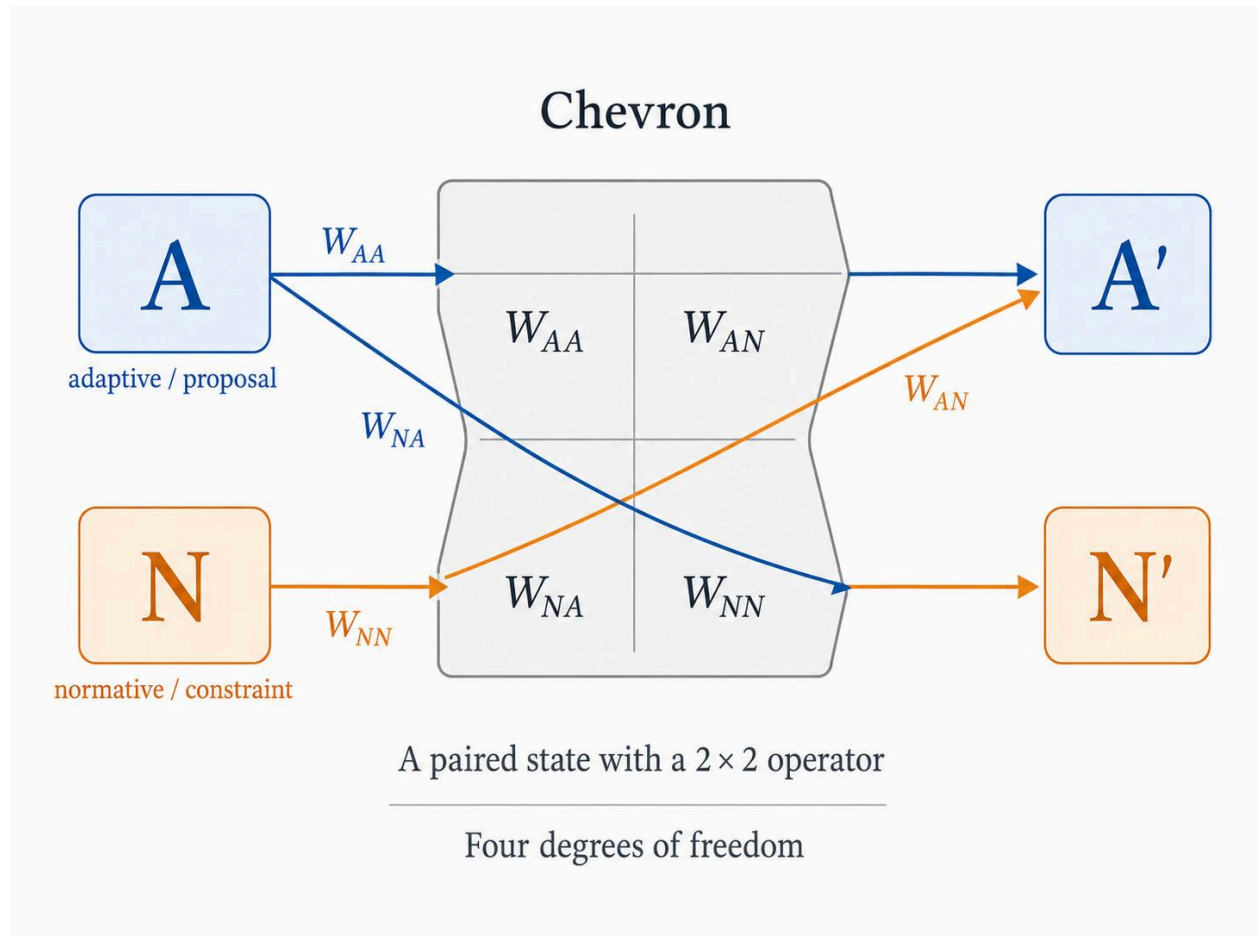
Breakout: The Minimal Chevron Block

A Chevron block begins by replacing a single hidden state with a paired state:

$$\mathbf{x} = [\mathbf{A}, \mathbf{N}]$$

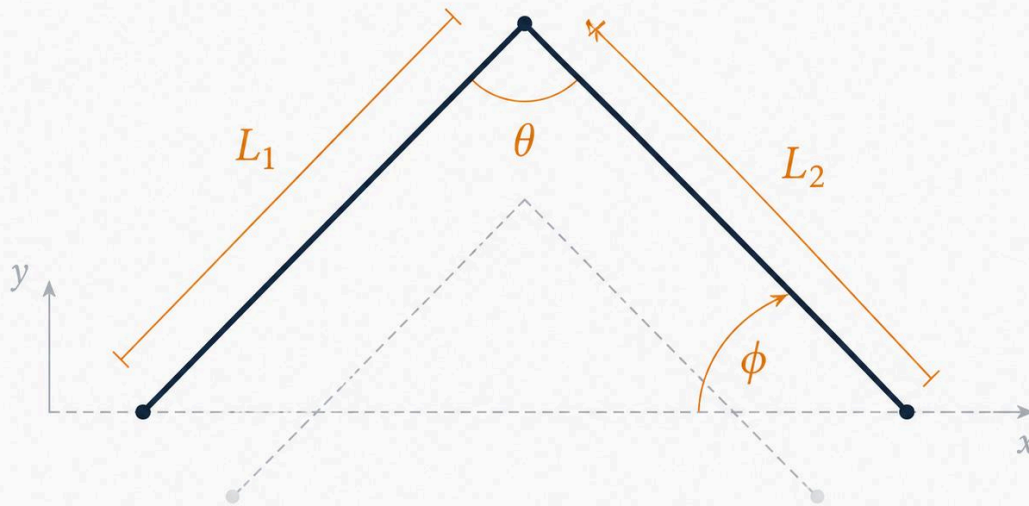
Instead of passing one vector through one weight matrix, the Chevron block passes the paired state through a local 2×2 operator:

$$\begin{bmatrix} \mathbf{A}' \\ \mathbf{N}' \end{bmatrix} = \begin{bmatrix} \mathbf{W}_{AA} & \mathbf{W}_{AN} \\ \mathbf{W}_{NA} & \mathbf{W}_{NN} \end{bmatrix} \begin{bmatrix} \mathbf{A} \\ \mathbf{N} \end{bmatrix}$$



Think of a Chevron with it's 4 degrees of freedom - length of arms, angle and orientation.

Chevron Geometry



① Left arm length L_1

③ Internal angle at apex θ

② Right arm length L_2

④ Global orientation ϕ

Four degrees of freedom: two arm lengths, internal angle, and orientation

Each term has a possible interpretation:

Term / Role

W_AA A preserves and transforms its own proposal

W_AN N constrains or steers A

W_NA A perturbs or informs N

W_NN N preserves and transforms retained structure

In Python-like pseudo-code:

```
class ChevronBlock:
    def __init__(self, dim):
        self.W_AA = Linear(dim, dim)
        self.W_AN = Linear(dim, dim)
        self.W_NA = Linear(dim, dim)
        self.W_NN = Linear(dim, dim)

        self.norm_A = LayerNorm(dim)
        self.norm_N = LayerNorm(dim)

    def forward(self, A, N, input=None):
        # Fast proposal update
        A_update = self.W_AA(A) + self.W_AN(N)

        # Slow constraint update
        N_update = self.W_NA(A) + self.W_NN(N)

        if input is not None:
            A_update = A_update + input

        A_next = self.norm_A(A + activation(A_update))
        N_next = self.norm_N(N + activation(N_update))

        return A_next, N_next
```

This is the minimal mathematical idea.

The architecture is not merely “two networks.” The point is the

cross-coupling:

A influences N through W_{NA}
N influences A through W_{AN}

So the block can express several regimes:

Regime / Chevron interpretation

W_{AN} strong retained constraint strongly shapes current proposal

W_{NA} strong current experience strongly updates retained structure

W_{AA} dominant fast adaptive continuity

W_{NN} dominant slow normative continuity

off-diagonal balance active A/N negotiation

off-diagonal suppression channel separation / anti-collapse

For a predictive architecture, the block becomes:

```
A_context, N_context = chevron_encoder(context)
```

```
prediction = predictor(A_context, N_context)

loss = distance(prediction, target_representation)
```

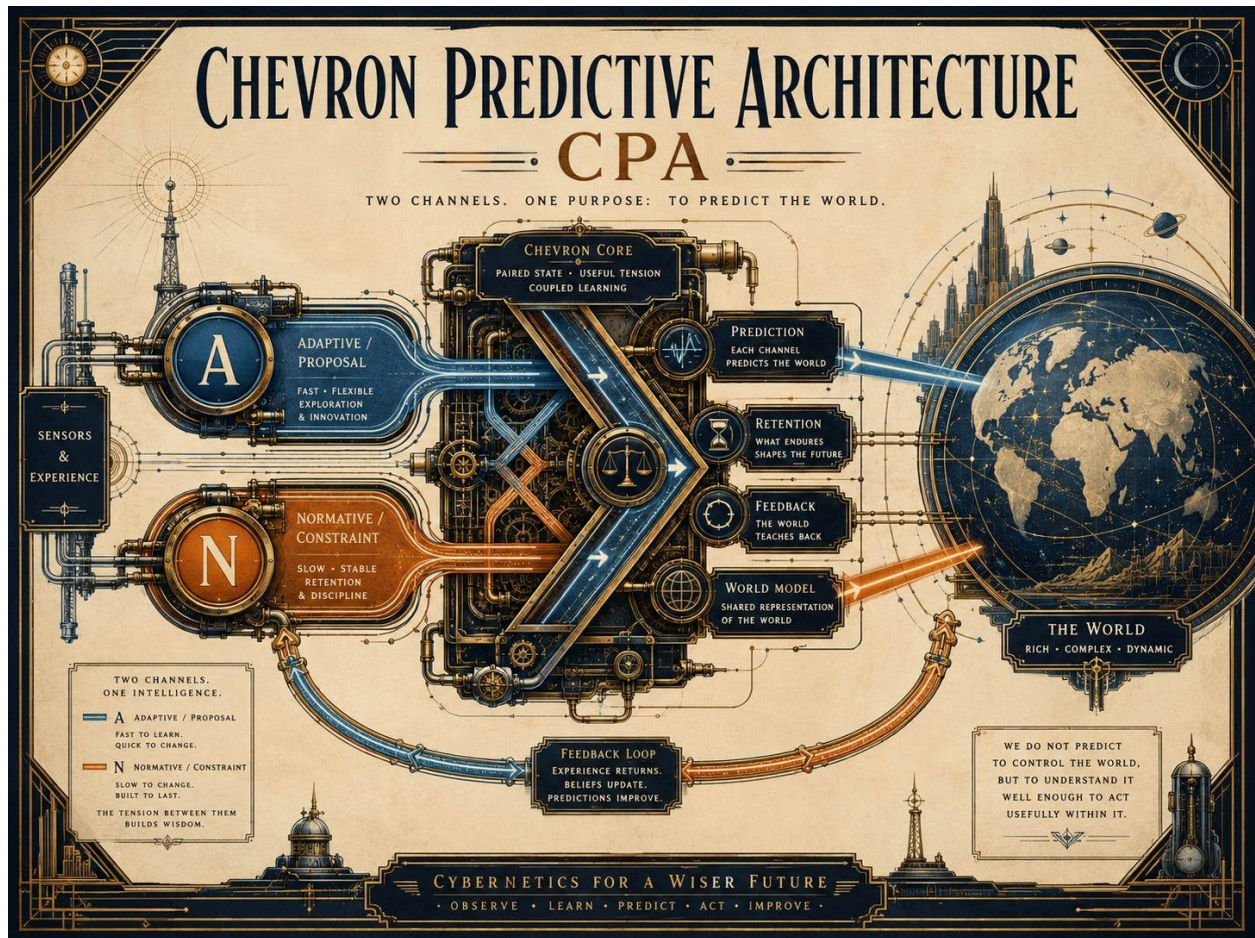
The training rule then gives the channels different time-scales:

```
optimizer_A = AdamW(A_parameters, lr=fast_lr)
optimizer_N = AdamW(N_parameters, lr=slow_lr)
```

This is where the Chevron becomes more than a block matrix. It becomes an allostatic learning unit:

```
A learns quickly from present error.
N learns slowly from retained usefulness.
The world model forms in their negotiated difference.
```

The Chevron block is mathematically simple: a paired state and a 2×2 operator. The speculative claim is not in the matrix itself, but in the training discipline imposed on it: A is allowed to move quickly, N is made to retain slowly, and prediction tests whether their difference is useful.



CPA as a JEPA-like Architecture

A simplified JEPA says:

context representation → predictor → target representation

A Chevron Predictive Architecture says:

$A(\text{context}) + N(\text{retained constraints})$
→ predictor

→ hidden / future / target representation

This changes the meaning of prediction.

The system is no longer merely asking:

What target representation follows from this context?

It is asking:

What target representation follows when the current proposal is
constrained by retained structure?

This makes the prediction less like a one-step mapping and more like a
negotiation.

A is the local proposal.

N is the retained constraint.

The predictor works through their tension.

In ordinary JEPA, asymmetry is often introduced through mechanisms such as target encoders, stop-gradient, or slowly updated representations. These are important anti-collapse devices.

CPA internalizes that asymmetry.

Instead of only having an online encoder and a target encoder, CPA asks whether every representation should contain an internal fast/slow polarity.

A does not merely predict.

N does not merely store.

Their relation becomes the site of learning.

N Attends to the Past

This also suggests a different way to think about attention.

Modern architectures often use attention to decide which tokens or patches matter. More recent residual-attention ideas extend this into depth: later layers can selectively retrieve earlier layer or block representations rather than simply accumulating them through fixed residual streams.

CPA gives that a Chevron interpretation:

A attends to the present. N attends to the retained past.

N should not merely be a slower copy of A. It should selectively retrieve prior constraints: earlier block states, previous interpretations, replay traces, failed predictions, stable object expectations, or action consequences.

In a block-based CPA, each block could leave behind an N-summary — a retained trace of its A/N negotiation. Later blocks could attend over these summaries.

Conceptually:

```
A_l = fast transform of current input
```

```
N_l = slow transform of:  
    previous N  
    attended prior N blocks  
    current A/N mismatch
```

```
prediction = P(A_l, N_l)
```

This gives a different kind of attention.

Not just:

Which token matters?

But:

Which retained constraint matters now?

That is closer to the problem a world model faces. It must not simply process what is present. It must bring the right past structure to bear on the present.

A world is not given all at once. It is reassembled through memory, expectation, and constraint.

Collapse as the Central Training Problem

All of this depends on training.

An architecture can suggest a space of possibilities, but training determines what actually forms.

The obvious danger is collapse.

A JEPA-like system may learn trivial embeddings.

A predictive system may learn shortcuts.

A two-channel system may allow A and N to become identical.

A slow channel may dominate and suppress adaptation.

A fast channel may drift and lose coherence.

So the training problem is not simply:

Minimize prediction error.

It is:

Learn useful differences without collapsing into sameness or diverging into noise.

This is why the A/N distinction matters.

If A and N become identical, the system loses its internal opposition.

If A and N become unrelated, the system loses coherence.

The useful regime lies between collapse and chaos.

The aim is not maximum difference.

The aim is productive tension.

Breakout: Between Collapse and Chaos

The useful regime for Chevron Predictive Architecture lies between two failures. If A and N collapse into sameness, the system becomes over-ordered: stable, but unable to generate meaningful difference. If A drifts too far from N, the system becomes chaotic: flexible, but unable to retain structure.

This echoes the entropic brain hypothesis, where rich conscious dynamics are associated with a critical zone between excessive order and excessive disorder. In CPA terms, this zone is not merely high entropy. It is **productive tension**: a state in which proposal and constraint remain distinct enough to generate novelty, but coupled enough to preserve coherence.

This is also the familiar **explore/exploit** dilemma in another form. A, the fast proposal channel, corresponds to exploration: trying new variations, loosening existing expectations, testing what might be possible. N, the slower constraint channel, corresponds to exploitation: using what has already proven reliable, preserving structure, acting from retained knowledge. But intelligence cannot live at either pole alone. Pure

exploitation becomes rigidity; pure exploration becomes noise. The useful regime is a controlled alternation: A opens the space of possible differences, while N selects and stabilizes the differences worth keeping. In CPA, explore/exploit is not merely a policy choice; it is built into the architecture as the tension between adaptive variation and inertial retention.

Gradient Descent as Efficient (A) Search

This is where next-token prediction remains instructive.

Large language models work astonishingly well partly because next-token prediction provides a dense training signal. Every token gives a local correction. Gradient descent turns those corrections into an efficient search process.

CPA should not reject that.

Gradient descent is not the enemy of Inertial VSR. It can be understood as a compressed local version of variation and selection.

In VSR terms:

VSR / Gradient descent analogue

Variation possible local parameter changes

Selection gradient direction

Retention weight update

Gradient descent searches efficiently. But it does not by itself decide what should become durable world structure. It updates according to local error.

In CPA, the proposal is:

Use gradient descent for efficient local search, but use A/N asymmetry and inertial retention to decide what becomes stable.

A can be trained quickly by ordinary predictive gradients.

N can be trained more slowly, through delayed updates, replay, lower learning rates, or consolidation-like mechanisms.

So we get:

Gradient descent searches. Inertia retains.

Playing A and N Against Each Other

Both A and N could be trained by SGD, but under different rules.

A receives fast updates.

N receives slow updates.

A explores the current predictive landscape.

N preserves what has remained useful.

This gives a training rhythm:

1. Hold N fixed or slow.
2. Let A adapt quickly to prediction error.
3. Test whether A's changes remain compatible with N.
4. Replay or perturb the situation.
5. Consolidate into N only what remains useful.

This is not full neuroevolution. It does not require a population of whole models.

The evolutionary logic can be local.

A produces variations.

Prediction tests them.

N retains what survives.

The selected unit is not merely a weight update. It is an A/N difference.

This is the important shift:

Learning is not just changing parameters. Learning is selecting which differences deserve inertia.

Evolutionary Training Without Full Evolution

Evolutionary language can sound computationally expensive. But CPA does not require replacing gradient descent with full evolutionary search.

The VSR process can be approximated inside gradient training.

Variation can come from:

- masking,
- dropout,
- perturbation,
- augmentation,
- alternative predictions,
- different temporal contexts,
- action-conditioned futures.

Selection can come from:

- prediction loss,
- robustness,
- recurrence,
- compatibility with N,
- avoidance of collapse,
- usefulness under replay.

Retention can come from:

- slow N updates,
- replay,
- EMA-like consolidation,
- memory blocks,
- recurrent stabilization.

This gives an evolutionary style without requiring literal biological evolution.

The slogan is:

SGD supplies variation. A/N tension supplies selection. N supplies retention.

Or more compactly:

A learns by gradient. N learns by inertia.

Not Just Prediction Loss

A CPA objective cannot be only prediction loss.

If the system only minimizes prediction error, it may find cheap shortcuts. It may collapse representations. It may make A and N redundant. It may preserve local correlations rather than world structure.

The objective must encourage prediction and productive difference.

Conceptually:

```
Loss =  
  latent prediction loss  
+ anti-collapse pressure  
+ A/N distinction pressure  
+ temporal consistency  
+ perturbation robustness
```

+ slow retention penalty

The exact mathematics is a later question.

The conceptual point is that the system must be rewarded not merely for getting the next representation right, but for doing so in a way that preserves useful internal structure.

A and N should not be forced apart arbitrarily.

They should not be fused prematurely.

They should remain different in ways that help the system predict, adapt, and retain.

A world model is not a bag of accurate predictions. It is a stable structure of useful distinctions.

World Model as Retained Difference

This brings us back to the central thesis.

JEPA says: predict latent structure.

CPA says: retain the differences that keep mattering.

A world model is not simply a predictor. It is the retained structure of differences that have survived prediction, action, replay, perturbation, and environmental recurrence.

This also returns us to Bateson's phrase: a difference that makes a difference.

The world model is not all difference.

It is selected difference.

It is difference under inertia.

That is why biological intelligence is such an important clue. Organisms are not just trained by experience. They are shaped by an evolutionary history of retained differences. Individual learning then continues this process at a faster time scale.

Evolution supplies deep N.

Development reshapes N.

Experience perturbs A.

Prediction tests A against N.

Replay consolidates what survives.

A world model forms when useful differences become retained constraints.

So perhaps the missing piece in purely predictive accounts is not prediction itself, but retention.

JEPA learns latent compatibility.

CPA learns retained difference.

Breakout: The R-rule as a Possible Update Law

There is a possible mathematical refinement here, which I will only gesture toward.

If A is the adaptive proposal and N is the retained constraint, then learning can be understood not only as scalar loss minimization, but as movement in the plane formed by A and N.

This is where the [R-rule](#) may enter.

The R-rule treats update as a controlled movement through tension:

```
A = proposal / action / adaptive pressure  
N = norm / constraint / retained structure  
tension = uncertainty or mismatch  
update = movement toward better compatibility
```

In this interpretation, learning is not merely “downhill” motion on a loss surface. It is a rotation or reorientation in the A/N plane.

That may eventually give CPA a more precise formal core. But the conceptual point can stand without it:

Predictive learning needs a rule for how proposal and constraint change each other without collapsing into sameness.

The R-rule is one possible candidate for that rule.

From Prediction to World

JEPA is valuable because it shifts AI away from raw reconstruction and toward latent prediction. That is a real advance.

But world models may require something more.

They may require an architecture of retained difference.

A world is not learned merely by guessing what comes next. It is built by retaining what continues to matter.

That is the intuition behind Chevron Predictive Architecture.

A proposes.

N remembers.

Prediction tests.

Inertia decides what becomes structure.

The first claim is conceptual: world models are not predictions alone.

They are retained differences under constraint.

The next question is practical:

Can we train them that way?

That is where Chevron Predictive Architecture becomes an experimental program. Start with JEPA. Split the state. Slow one channel. Measure collapse. Add memory. Then add selection.

There is a useful connection here to an already successful trick in modern self-supervised learning: slow weights and exponential moving averages (EMA). BYOL uses a target network whose weights are an exponential moving average of the online network, and MoCo uses a slowly progressing momentum encoder to maintain more consistent representations. I-JEPA similarly uses a target encoder updated by exponential moving average of the context encoder. These systems suggest that a delayed, slower representation can stabilize learning and help prevent representational collapse.

Chevron Predictive Architecture can be read as taking this macro-level design pattern and turning it into a micro-level representational principle. Instead of keeping a separate slow target network outside the main representation, CPA builds a fast/slow relation into each unit or edge. The A-channel plays the role of the fast, adaptive proposal; the N-channel plays the role of the slower retained constraint. The point is not merely to stabilize a loss function, but to make proposal and constraint native to the representation itself.

In that sense, CPA does not reject the lesson of BYOL, MoCo, or JEPA. It generalizes it. The delayed target encoder becomes a local architectural motif: every Chevron can carry both the current adaptive movement and the retained structure against which that movement is tested. What appears in self-supervised learning as a useful training trick becomes, in CPA, a candidate substrate for continual learning, semantic control, and resistance to collapse.

EMA says: keep a slow copy of the learner. CPA says: make fast/slow learning the structure of the representation.

Chevron Predictive Architecture is an attempt to trade off two demands: the practical efficiency of current AI techniques — gradient descent, latent prediction, residual attention, and self-supervised training — with the conceptual framework developed in the [Mandala of Meaning](#): oppositional structure, retained difference, recursive tension, and inertial VSR.

Previous Posts on [Chevron Networks](#) and the [R rule](#) are available as Amazon Kindle eBooks (or [here](#) and [here](#) on my substacks).

Andre with ChatGPT 5.5, May 2026



Before Computation: Difference, Use, and the A/N Dynamics of Mind

An update on consciousness, meaning, and the Hypercube of Opposites

[Andre Kramer](#)

May 27, 2026

“When it looks as if there were no room for such a form between other ones you have to look for it in another dimension. If there is no room here, there is room in another dimension. (It is in this sense too that there is no room for imaginary numbers in the continuum of real numbers. But what this means is: the application of the concept of imaginary numbers is less like that of real numbers than appears from the look of the calculations. It is necessary to get down to the application, and then the concept finds a different place, one which, so to speak, one never dreamed of.)

PHILOSOPHICAL INVESTIGATIONS IIxi 2010”



I have been circling around a theory of consciousness for some time. The earlier posts here were deliberately speculative. They leaned heavily on the Hypercube of Alchemical Opposites: the idea that meaning and selfhood arise through recursive traversal of irreconcilable oppositions rather than by resolving them into final synthesis.

Reading Camus' *The Myth of Sisyphus* now has made the personal arc clearer. The absurd (Albert Camus says) is not simply meaninglessness. It is the living tension between the human demand for meaning and the world's refusal to provide final answers. Consciousness, in that frame, is not escape from the absurd but lucid traversal of it: the capacity to remain awake inside an irreconcilable *tension* without collapsing into denial or despair. That is close to what the Hypercube was trying to name in a more symbolic and structural language.

I still think that captured something important. Conscious life does seem to involve the capacity to hold tensions without prematurely collapsing them. We live between self and world, agency and mechanism, certainty and wonder, individuation and transcendence. The old Hypercube language gave me a symbolic geometry for this.

But I would now speculate about this claim more carefully.

The Hypercube is not (and never really was), by itself, a theory of why consciousness exists. It is better understood as a theory of how

awareness becomes structured into mind: how differences become salient, how salience becomes use, how use becomes memory, how memory becomes norm, and how norm becomes action.

This is a shift from a theory of consciousness as such to a theory of structured mind.

Bohm helps clarify the distinction. In his language, thought is not the whole of mind. Thought is a conditioned, fragmenting, explicating process: it cuts, names, compares, remembers, abstracts, and projects. It is necessary, but it can mistake its fragments for reality. Mind, by contrast, points toward a wider field of awareness, attention, wholeness, and unfolding order in which thought occurs. So the question is not only how thought operates, but how awareness becomes structured enough to think without being reduced to thought.

In that sense, this post is not trying to explain consciousness by thought. It is asking how mind becomes patterned into thought, meaning, action,

and self while still leaving room for the deeper wholeness that thought can only partially explicate.

That distinction matters. I do not think raw awareness, or phenomenological presence, is explained by a functional architecture. Mathematical models can describe relations, transformations, selections, constraints, and reports. They can describe how a system distinguishes, filters, remembers, acts, and recursively stabilizes a self. But they do not explain why there is something it is like.

My preferred stance is therefore a loose dual-aspect monism. Mind and matter are not two separate substances. Nor is consciousness ghostly software running on biological hardware. Rather, reality may have both a structural, describable aspect and an experiential, inward aspect. The model belongs mostly to the structural side. It may describe how awareness becomes organized, but not why awareness exists in the first place.

That is the guardrail for what follows.

AI Models (LLMs etc) Begin After the Cut

The brain is often described as a computer. I understand the appeal. I spent a career in software architecture, distributed systems, and computational design. Computation is not the enemy here. Functional descriptions are indispensable. At the right level of abstraction, organisms compute, regulate, predict, classify, and control.

But computation is a level of description, not the deepest metaphor.

The computer metaphor usually begins with already-discrete bits, symbols, addresses, instructions, and memory locations. The cut has already been made. A bit is already 0 or 1. A token is already a token. A variable has already been named. A function already has a domain.

Living cognition has a prior problem.

It must help decide what counts as a difference at all.

Before there is calculation, there is distinction. Before there are symbols, there are differences that matter. Before there is language, there is salience, orientation, inhibition, appetite, fear, attraction, familiarity, surprise, and action-readiness.

An AI Model begins after the cut. A living mind participates in making the cut.

That is why the hardware/software metaphor breaks down in biology. In a computer, the same software can often run on different hardware, and the processor is not usually rewritten by the program it executes. In living systems, the substrate changes with use. Memory is not stored in a separate file; it is embodied in altered dispositions. Perception is not input to a neutral processor; it is shaped by posture, metabolism, affect, movement, attention, and expectation.

What looks like software is often retained bodily history. What looks like hardware is plastic organization in motion.

So this is not anti-computation. It is anti-reification of the computer metaphor.

Functionalism can be useful if it is treated as coarse-graining: a description of patterns and capacities at a chosen level. The mistake is to turn that abstraction into an ontology, as if mind were literally software and the brain merely hardware.

Difference in Use

Gregory Bateson famously defined information as “a difference that makes a difference.” That phrase has become almost too familiar, but it remains profound.

Not every difference matters. A sensory variation, a shadow, a sound, a chemical gradient, a memory fragment, or a word only becomes meaningful if it changes what a system can do.

Wittgenstein sharpens this point. Meaning is not a hidden object behind a sign. Meaning is use.

So Bateson's phrase can be reframed:

A meaningful difference is a difference in use.

This does not have to mean linguistic use. Language is a late, visible crystallization of a deeper process. Biological use comes first: approach, avoid, grasp, eat, flee, wait, inhibit, attend, repair, remember, signal, bond.

A raw difference becomes meaningful when it changes the field of possible action.

Or more compactly:

A meaningful difference is a changed affordance.

This is the point at which the [Hypercube of Opposites](#) becomes less like an abstract symbolic map and more like a living field of use. Oppositions are not just conceptual pairs. They are tensions that matter to an

organism: safe/dangerous, near/far, familiar/strange, mine/not-mine, possible/impossible, urgent/open, self/other, act/wait.

The axes are grounded, but not fixed. They arise from body, world, memory, development, culture, and action. They can shift. They can be learned. They can be distorted. They can be healed. They can become traps.

D–N–A: Difference, Norm, Action

This gives a compact triad:

$D \rightarrow N \rightarrow A$

Not DNA in the literal genetic sense, but as a symbolic compression of living cognition.

D is difference, distinction, division, and even death. It is the cut from continuity.

N is norm, negation, memory, filter, inhibition, constraint, and retained form. It remembers the cut and decides whether it matters.

A is action, appetite, agency, adaptation, and actuality. It expresses the selected difference into the world.

D cuts the world. **N** remembers the cut. **A** acts from it.

A and **N** are the active processes (a dyad), while **D** is the boundary or “cut” generated and maintained by their interaction.

Life is not computation over pre-given signs. Life is the recursive circuit by which difference is cut from continuity, retained as norm, selected under pressure, and released as action.

This is also where “death” belongs in the **D**. To make a distinction is to exclude. To choose one action is to let other actions die. To become one self is not to become all selves. Discreteness has a cost. Meaning is not free-floating information. It is difference under the pressure of loss.

A and N Are Not System 1 and System 2

In an earlier post, I tried to describe meaning through a more functionalist language: System 1 and System 2, feeling and thinking, actor and critic, recursive loops, meta-regulation. I still think that was useful scaffolding. It captured the importance of tension, critique, self-reference, and transformation.

But I now think it was too close to a dual-process picture.

[**Breakout:** It is important not to collapse A/N into familiar binaries such as System 1/System 2 or left brain/right brain. Those analogies can be suggestive, but they are too coarse and too easily reified. A and N are not two anatomical modules or two cognitive systems. They are two aspects of a living process: A as proposal, action, appetite, and variation; N as norm, memory, inhibition, filtering, and retention. Actor/critic dynamics may appear within this pattern, but actor/critic is a scaffold or implementation motif, not the underlying claim.]

A is not merely fast intuition. N is not merely slow reasoning. That would preserve the wrong split.

Instead, A and N are two aspects of one living process.

A is proposal, appetite, adaptation, attention, action, outward unfolding.

N is norm, negation, memory, inhibition, retained form, inward filtering.

Thought is not a separate faculty sitting above them. Thought is what happens when A is drawn into N strongly enough to become stabilized, delayed, compared, named, corrected, or made shareable.

Thought is A becoming answerable to N.

A reflex is A with little N mediation. A habit is A already shaped by sedimented N. Imagination is A exploring under loosened N. Judgement is A licensed by N. Will is A that has passed through N and still moves.

This is a better frame than System 1/System 2 (or other Actor /Critic metaphor) because it is not two systems. It is a relation, a coupling, a fold, a tension.

There are not two minds, fast and slow. There are different regimes of A/N coupling.

The Chevron as Local Fold

This is where [Chevron Networks](#) enter the picture.

A chevron unit is a two-channel unit. Instead of one scalar state, it has an A-channel and an N-channel. Instead of a scalar weight, the connection can be represented as a 2×2 operator:

$$[A'N'] = [w_{AA} \ w_{AN}]$$

$$[w_{NA} \ w_{NN}]$$

The diagonal terms preserve each channel:

- wAA: A continues A — association, impulse, proposal, momentum.
- wNN: N preserves N — memory, norm, retained structure, inertia.

But the cross-terms are the interesting part:

- wAN: N shapes A — inhibition, permission, steering, relevance.
- wNA: A updates N — learning, surprise, revision, consolidation.

Meaning is not inside A or N. Meaning happens in the cross-coupling.

A chevron is a local fold between proposal and norm. A recurrent Chevron Network (think of successive / recurrent layers of such units) would then model thought as repeated folding:

$$A \rightarrow N \rightarrow A' \rightarrow N' \rightarrow \dots$$

A proposes. N filters. A is revised. N is updated. The revised N changes what A can next propose.

This is what I mean by A leaning into N.

A chevron is a local fold between proposal and norm. A recurrent Chevron Network—utilizing successive layers and recurrent feedback—models thought as **repeated folding**.

In successive layers, this folding is hierarchical: lower-level sensorimotor proposals are folded with immediate physical constraints, while higher-level layers fold abstract concepts with cognitive norms. In recurrence, this folding is temporal: the output of a fold at one moment becomes the starting point for the next.

This is what I mean by A leaning into N. It is not computation over static symbols. It is the recursive bending of active possibility through retained constraint, repeated across scales of space and time.

Inertial VSR and Selection from Potentiality

This also connects to [Inertial VSR](#): Variation, Selection, Retention, with inertia.

Variation→Selection→Retention

A generates variation: possible movements, meanings, words, interpretations, actions, and orientations.

N filters them through memory, salience, norm, inhibition, viability, and embodied history.

Selection is the moment where one possible difference becomes actual enough to act from or retain.

Retention folds the selected difference back into N.

Inertia matters because N should not simply chase every new A. Systems without inertia tend to be unstable. A system with too much inertia is rigid. Living intelligence lies between collapse and chaos.

Here I have found the Born rule useful as a formal analogy, not as a literal claim about quantum brains.

In quantum mechanics, the Born rule converts amplitude into probability:

$$P_i = |\psi_i|^2$$

In this framework, a Born-like rule can stand for selection from potentiality. A field of possible states is not yet an actual outcome. Selection converts latent propensity into weighted actuality.

If we write, cautiously,

$$\psi_i = A + iN$$

then A and N are not real versus unreal. N is not “imaginary” in the everyday sense. It is orthogonal: the dimension of constraint, memory, and norm that gives A its direction.

[**Breakout:** For living systems, a more useful selection image may be:

$$P_i = |A_i|^2 * F(N_i) / \sum_j |A_j|^2 * F(N_j)$$

where $F(N_i)$ represents norm-fit, viability, salience, inhibition, or retained constraint.

This selection rule is deliberately **Bayesian**-shaped. A generates candidate variations; N acts as a prior or viability filter; the denominator normalizes the competition across alternatives. The Born-like element is the squaring of the proposal amplitude, which converts potential into positive weight. But the living system is not pure amplitude-selection: it is amplitude filtered by retained norm. In short, selection is not the strongest A winning. It is A weighted by N and retained through Inertial VSR.

Variation: A_i

Selection: $P_i \propto |A_i|^2 * F(N_i)$

Retention: $N_{t+1} = (1-\lambda)N_t + \lambda R_i$

]

Selection is not simply the strongest impulse winning.

Selection is A weighted by N .

That gives a compact summary of the larger process:

The [R rule](#) rotates potential.

The Born-like rule selects actuality.

Inertial VSR retains the selected difference.

Chevron dynamics repeats this locally.

Again, this is analogy and architecture, not established neuroscience.

The Imaginary Is Not Unreal

The imaginary dimension matters here because it prevents us from flattening mind onto one line.

Wittgenstein's remarks on imaginary numbers (the post-starting quote from his *Philosophical Investigations*) are helpful. If there is no room for a concept in one dimension, look for the dimension in which it has use. The imaginary number is not hiding on the real line. It belongs to another application.

Likewise, N is not another A-like object. Intention is not a little inner object accompanying action. Meaning is not a private object behind a word. Will is not a ghostly push beside movement.

These are dimensional mistakes.

If we write:

$$\psi = A + iN$$

we should not hear:

real action plus unreal norm.

We should hear:

action plus an orthogonal dimension of constraint and orientation.

Euler's formula gives the image:

$$e^{ix} = \cos(x) + i \sin(x)$$

The imaginary dimension is what allows growth to [become rotation](#). It is what turns linear becoming into phase, rhythm, recurrence, and return.

So the imaginary is not the opposite of reality. It is the sign that reality is not exhausted by the real line.

For mind, this means that action alone is not enough. A needs the orthogonal dimension of N: memory, inhibition, relevance, norm, body, world, and self-maintenance.

Will is not brute force. Will is A (representation) that has passed through N and still moves.

Bundles of Difference and Experience

This leads to a looser form of dual-aspect monism.

I do not want to say that experience is built from atomistic qualia: red quale, pain quale, taste quale, and so on. That feels too much like turning experience into private objects.

Instead, an experience may correspond to an attended bundle of grounded differences.

A feeling is not a quale-atom. It is a selected constellation:

- safe / dangerous,
- near / far,
- tense / relaxed,
- mine / not-mine,
- urgent / open,

- familiar / strange,
- controlled / helpless,
- self / other,
- fitting / dissonant,
- possible / impossible.

Attention selects and binds such differences into a felt situation.

The structural aspect is the pattern: a dynamic bundle of differences grounded in body, world, memory, and action.

The experiential aspect is what that bundle is like from within.

This does not reduce experience to function. It also does not make experience an ineffable collection of private atoms. It treats particular experiences as structured without pretending that structure explains awareness itself.

In a Nagelian spirit, function cannot explain why there is awareness. But an A/N-VSR picture may help describe how awareness becomes patterned.

Function does not generate the light. It shapes what the light illuminates.

Bohm, Not Perceptronium

This is why I find Bohm's implicate and explicate order more congenial than the idea of consciousness as a special "phase of matter."

Tegmark's computronium and perceptronium are useful provocations. They ask what matter would have to be like to compute or perceive. But the phase-of-matter metaphor risks treating consciousness as another objective state alongside solids, liquids, plasmas, or computational substrates.

That may blur the category difference between structural description and lived experience.

Bohm offers a better image for dual-aspect monism. Mind and matter are not separate phases, nor is mind software added to matter. They are different unfoldings of a deeper wholeness.

In this language:

- N is the enfolded field of retained possibility.
- A is unfolding into action.
- D is the cut that makes something explicit.
- Retention folds selected difference back into the field.

The brain is not a device that constructs experience from data. It is a living filter that allows some enfolded differences to become explicit, actionable, and retained.

That is a metaphor, but I hope a useful one.

Rosch, Embodiment, and Categories as Attractors

Eleanor Rosch's work on categorization also fits here. Categories are not usually clean logical boxes defined by necessary and sufficient conditions. They have centres, gradients, prototypes, bodily salience, and use-structures.

A robin may be a more prototypical bird than a penguin. A chair may be more or less chair-like. Meaning is not always binary. It is often graded, embodied, and action-guiding.

This supports the move away from symbol-first cognition.

A category is not first a definition. It is an embodied attractor of use.

The enactive tradition of Varela, Thompson, and Rosch makes the broader point: cognition is not representation of a pre-given world by a detached inner observer. It is the enactment of a world of significance through embodied action.

Chevron dynamics (as opposed frozen LLM weights) would be one possible local architecture for this:

A moves, probes, attends, and acts.

N stabilizes salience, prototypes, norms, and constraints.

Inertial VSR selects and retains usable differences over time.

The world that matters is enacted through this recursive coupling.

[**Breakout:** Wittgenstein's warning is that grammar seduces us into objectification. Because nouns behave like names of things, we begin to imagine meanings, intentions, thoughts, and selves as inner objects. But often their reality lies in use, criteria, continuation, correction, and practice. This is a useful guardrail for A/N. A and N should not be reified into modules or substances. They are dimensions of a process: proposal and constraint, unfolding and retention, action and norm. The model is strongest when it resists turning its own abstractions into little inner things.]

Brief contrast with IIT

Integrated Information Theory has the virtue of taking consciousness seriously as something more than behaviour, report, or computation. But from this framework, IIT can appear too static if read mainly as a

measure of integrated structure. Mind is not only integrated information. It is structured change.

What matters is not simply that a system is integrated, but how it changes: how differences arise, how quickly they are selected, how they are retained, and how prior retention alters the next field of possibility. Thought is not a state but a movement. Experience is not merely a configuration but a temporally unfolding process.

In A/N terms, consciousness-as-mind is found less in a snapshot of integration than in the dynamic loop:

$$A_t \rightarrow N_t \rightarrow A_{t+1} \rightarrow N_{t+1}$$

A proposes; N filters; the filtered result changes future A; retained change alters future N. The important quantity is not only structure, but trajectory.

This is why the metaphor of 'symbolic entropy'—which I used in earlier essays to describe the shifting complexity of mental states—still has some value, if treated carefully. The point was never that consciousness can be reduced to a simple equation, but that mind and thought are temporal: differentiation, recombination, acceleration, slowing, collapse, return. Experience intensifies at inflection points, where the rate of meaning-change itself changes.

it is change, and the rate of change, that matters — because mind is not a thing the brain has, but a process the organism undergoes.

[Breakout: The i-dimension is useful because it lets the model include what is not simply actual but still effective: resonance, repetition, memory, inhibition, and counterfactual possibility. This may bridge the IIT and the following constructor-theory sections. IIT asks about integration; constructor theory asks about possible and impossible transformations. A complex A/N model asks how actual motion is shaped by an orthogonal field of retained and unrealized possibilities.

Mind, on this view, is not merely integrated information, but integrated counterfactual resonance unfolding over time.]

Constructor-Filters, Not Software Minds

Constructor theory is useful here because it treats information physically, not as disembodied software. It asks what transformations are possible or impossible, what can be copied, retained, distinguished, transformed, and used again.

That maps naturally onto this framework.

A living system is not merely a computer executing instructions. It is a constructor-filter: a system that preserves itself by selecting which transformations remain possible.

In A/N terms:

- A is constructive outward activity.
- N is the filter over possible transformations.

- Inertial VSR is the development of retained transformation-capacity.

The brain is less a von Neumann computer than a living constructor-filter — closer, in this respect, to a von Neumann replicator than to a Turing machine.

A Turing machine or stored-program computer executes operations over already defined symbols. A von Neumann replicator, by contrast, must preserve a description, construct a vehicle, copy the description, and maintain the cycle across generations. That is much closer to life: not computation alone, but construction, repair, copying, retention, and self-maintenance.

[**Breakout:** This also marks a difference from current AI post-training. RLHF, DPO, and related methods can steer a pre-trained model toward preferred behaviours, but they do not by themselves create a developmental system whose proposal-space and norm-space co-evolve over time. In A/N terms, post-training mostly adjusts the behavioural surface after the main representational structure has already formed.

Inertial VSR asks for something deeper: fast-changing A, slow-changing N, and a retention mechanism that allows selected differences to reshape future selection without erasing prior structure. The problem is not merely how to (safely) *align* outputs, but how to grow a system whose constraints, habits, and possibilities develop together.]

It computes, yes. But computation is not the deepest metaphor.

From Autonomic Polarity to Individuation and Transcendence

The same pattern appears at different scales.

At the bodily base, the autonomic nervous system is organized around counteracting tendencies: sympathetic mobilization and parasympathetic restoration. The organism must act, spend, seek, defend, and engage. It must also restore, digest, conserve, repair, and return.

At the highest developmental level, a similar apparent polarity appears as individuation and transcendence.

Individuation is becoming a distinct, coherent self.

Transcendence is going beyond the self toward wider wholeness.

Too much individuation becomes rigidity, isolation, and egoic closure.

Too much transcendence becomes dissolution, vagueness, or escape from responsibility.

Intelligence may span this whole arc:

mobilization/restoration→proposal/constraint→variation/selection/retention→action/reflection→individuation/transcendence

Human intelligence is not just problem-solving. It is recursive regulation of difference from body to spirit.

Improving on my earlier Speculation

My earlier functionalist/processual posts pushed actor/critic loops, System 1/System 2, symbolic scaffolding, dream critique, and recursive meaning-making and even process philosophy as far as I could take them.

I still think much of that remains useful as a theory of meaning. Meaning can emerge through recursive tension. Dreams or psychedelics may indeed loosen over-stabilized patterns and reintroduce symbolic plasticity. Counterfactuals may act as synthetic critics. Inner and outer worlds may fold into one another through imagination, memory, regret, and anticipation.

But I would now more carefully narrow the claim.

Recursive functional loops may explain aspects of mind: attention, self-modeling, inhibition, agency, narrative, and meaning. They do not explain raw awareness.

The old question was:

Can recursive oppositional loops generate consciousness?

The new question is:

If awareness is fundamental or irreducible, how do recursive oppositional loops structure it into mind?

That shift — from generation to structuring — is the philosophical correction.

What This now Claims, and What It Does Not

This argument claims that cognition begins before symbolic computation. Living systems must create usable distinctions from continuous flux. Meaning is difference in use. A/N dynamics models proposal and norm as coupled dimensions. Chevron units provide a possible local mechanism for that coupling. Inertial VSR describes how selected differences are retained over time. A Born-like rule can serve as a formal analogy for selection from potentiality. Will is action (on

representation) that has passed through norm-filtered selection and still moves.

But this argument does not claim that brains literally operate by quantum wavefunctions. It does not claim that the Born rule literally governs thought. It does not claim that Bohm's implicate order is established neuroscience. It does not claim that Chevron Networks solve consciousness. It does not claim that language is the ground of cognition. It does not claim that A/N is simply System 1/System 2 renamed. It does not claim that constructor theory already contains a theory of mind.

Most importantly, it does not claim to derive awareness from structure.

The stronger and more careful claim is this:

Consciousness may be fundamental, but mind is structured consciousness — awareness folded through difference, filtered by norm, selected through action, and retained as self.

Closing: After the Cut

The deepest question is not how a system computes over symbols, but how a system comes to have symbols at all.

How does continuity become difference?

How does difference become use?

How does use become norm?

How does norm become action?

How does action return to reshape the field from which it came?

This is the space where Chevron dynamics belongs: before language, before software, before computation over given tokens.

LLMs begin after the cut.

Living mind begins by making, selecting, retaining, opposing, and acting from the cut.

Difference becomes meaning when it changes use.

Use becomes thought when A leans into N.

Selection becomes will when A passes through N and still moves.

And consciousness — if it is fundamental — becomes mind when it is folded through all of this without being exhausted by any of it.



This remains speculation. But speculation can still be disciplined if it widens the *dialogue* rather than closes it. Bohm's own work was, in part, an attempt to move the conversation: to find metaphors and forms of explanation less trapped by fragmentation, mechanism, and inherited categories. That is also the aim here.

The purpose is not to replace current AGI narratives with another totalizing system. It is to select sharper metaphors for our use: difference rather than data, filter rather than processor, constructor rather than computer, norm rather than stored software, action rather than output, mind rather than mere thought. Together these metaphors construct a more extensive picture than current AGI narratives typically allow.

That has been the aim of my personal [substack](#) (the [R rule's](#) own), and of [my more technical one](#) as well: to widen the dialogue around AI, mind, meaning, and safety. The view developed here seems compatible with a Nagelian form of neutral or dual-aspect monism. It does not claim to solve consciousness. It tries to apply new metaphors as explanatory instruments — not final answers, but better ways to ask what kind of process a mind might be. Better than Sisyphus, perhaps: the stone remains, but the pushing becomes an alchemical reddening — the work by which matter, mind, and meaning are slowly transformed.

Andre with ChatGPT-5.5

May 2026



Thanks for reading Andre's Baccalaureus in Arte Ingeniaria! Subscribe for free to receive new posts and support my work.

Furthermore ...

A 2D Chevron network gives us oppositional processing across a layered surface. A 3D Chevron network would add thickness: memory, retention, recurrence, and embodied depth. It would allow oppositions not merely to be connected, but to fold around one another, forming attractors, cavities, and loops. In 2D, meaning is propagated. In 3D, meaning can circulate, sediment, and return.

The brain's three-dimensionality is not just anatomical accident. It is a developmental constraint that turns wiring into morphology. A small Chevron experiment cannot reproduce that evolutionary history, but it could test the lower-level principle: whether spatial cost, local modularity, and retained A/N structure improve learning compared with a flat, freely connected network.

Memory challenges a simple filter theory because remembered sensations are not fully present, yet not unfelt. The way out is to treat memory as filtered experience becoming part of the filter. Its felt aspect hitchhikes on partial reactivation: bodily tone, affective salience,

sensorimotor echo, and self-continuity. Memory is therefore flatter than perception, but still experiential because it is retention re-entering the present.

Direct sensation is A under strong world constraint; memory is A reactivating under N-retained structure, with weaker external constraint.

A and N can be treated as two dimensions supervening on ordinary spacetime: not physical dimensions, but dimensions of living orientation. A is the vector of proposal, adaptation, and outward movement; N is the vector of retention, constraint, and inward coherence. A self emerges not as a point in either dimension, but as a trajectory through their tension. To become a self is to acquire a characteristic way of leaning into possibility without losing continuity.



Nested Chevron Architectures Synthesis

From Predictive Models to Vision-Language-Action towards Nested Chevron Architecture

[Andre Kramer](#)

Jun 05, 2026



The first [Chevron Predictive Architecture](#) proposal asked a simple question:

What if a world model is not merely a predictor, but a retained structure of useful differences?

Thanks for reading Andre's Baccalaureus in Arte Ingeniaria! Subscribe for free to receive new posts and support my work.

In the minimal version, a model has two interacting channels:

- **A:** the adaptive, active, proposal channel
- **N:** the normative, retained, constraint channel

A changes quickly.

N changes slowly.

Prediction tests their compatibility.

That is the core Chevron idea.

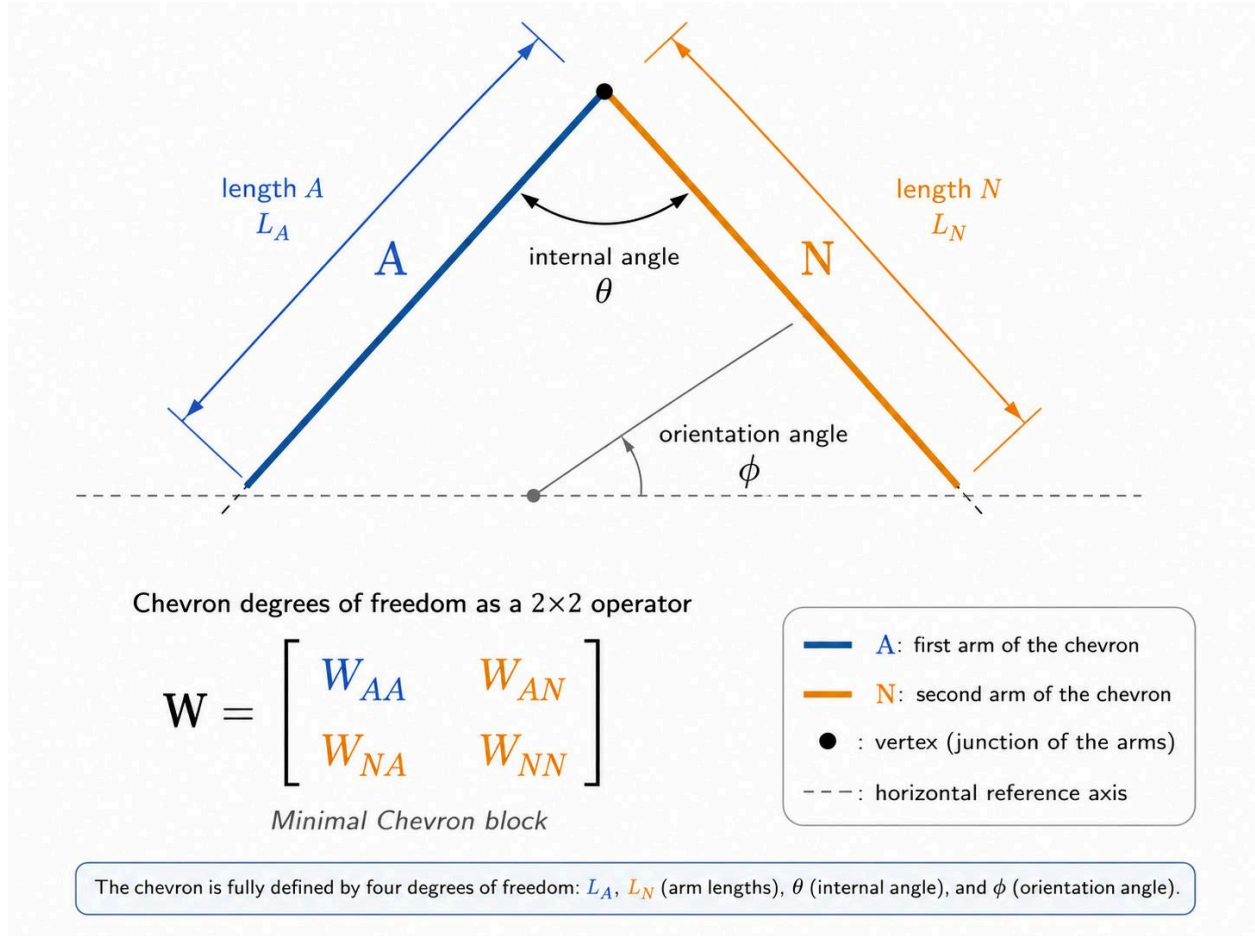


Figure 1: The geometric metaphor behind the Chevron block. While a working model uses high-dimensional block matrices, the internal A/N negotiation can be conceptualized as having four degrees of freedom: the independent magnitudes of the adaptive and retained channels (the arms), and their cross-coupled tension and orientation (the angles).

But once this structure exists, a further possibility appears almost immediately. Perhaps A and N should not exist at only one level of the model. Perhaps they should appear at several levels, each with its own timescale.

This suggests a natural next step:

Nested Chevron Architectures (or NCA)

Why Nesting Matters

Modern AI is moving from language models toward multimodal systems: language plus images, images plus video, video plus action. In robotics and embodied AI, this often becomes the VLA pattern:

vision → language → action

A system sees the world, interprets an instruction, and produces an action.

But this formulation may still be too flat.

Vision is not just present sensation. It also depends on retained visual expectations: object permanence, spatial continuity, affordances, and motion.

Language is not just the current instruction. It depends on retained task context, norms, goals, and constraints.

Action is not just the next movement. It depends on retained habits, safety boundaries, learned motor patterns, and expected consequences.

So the Chevron question becomes:

Why should there be only one A/N pair?

A more natural architecture might be:

visual A/N → object A/N → task A/N → action A/N

Each level has its own proposal and its own retained constraint.

A/N Across Scales

At the sensory level:

A_v = what seems to be happening now
 N_v = retained visual regularities

At the object level:

A_o = current object hypothesis
 N_o = object persistence and affordance memory

At the task level:

A_t = current interpretation of the instruction
 N_t = retained goal or norm

At the action level:

A_a = proposed next action
 N_a = retained action constraint or habit

The crucial point is that these should not all update at the same speed.

A video frame changes quickly.

An object persists longer.

A task goal persists longer still.

A safety constraint or learned habit may persist longest of all.

So a nested Chevron architecture naturally becomes a multi-timescale system:

```
N_visual updates relatively quickly.  
N_object updates more slowly.  
N_task updates more slowly again.  
N_action_norm updates very slowly.
```

This gives Chevron networks an immediate way to think about hierarchy without simply stacking more layers.

The hierarchy is not just depth.

It is retained difference across time.

From Flat Prediction to Nested Prediction

A flat predictor asks:

What comes next?

A flat Chevron predictor asks:

What comes next when current proposal is constrained by retained structure?

A nested Chevron predictor asks something richer:

What sensory change comes next?

What object state persists through that change?

What task remains active through changing perception?

What action follows from the negotiation between scene, goal, and constraint?

This begins to look like a world model.

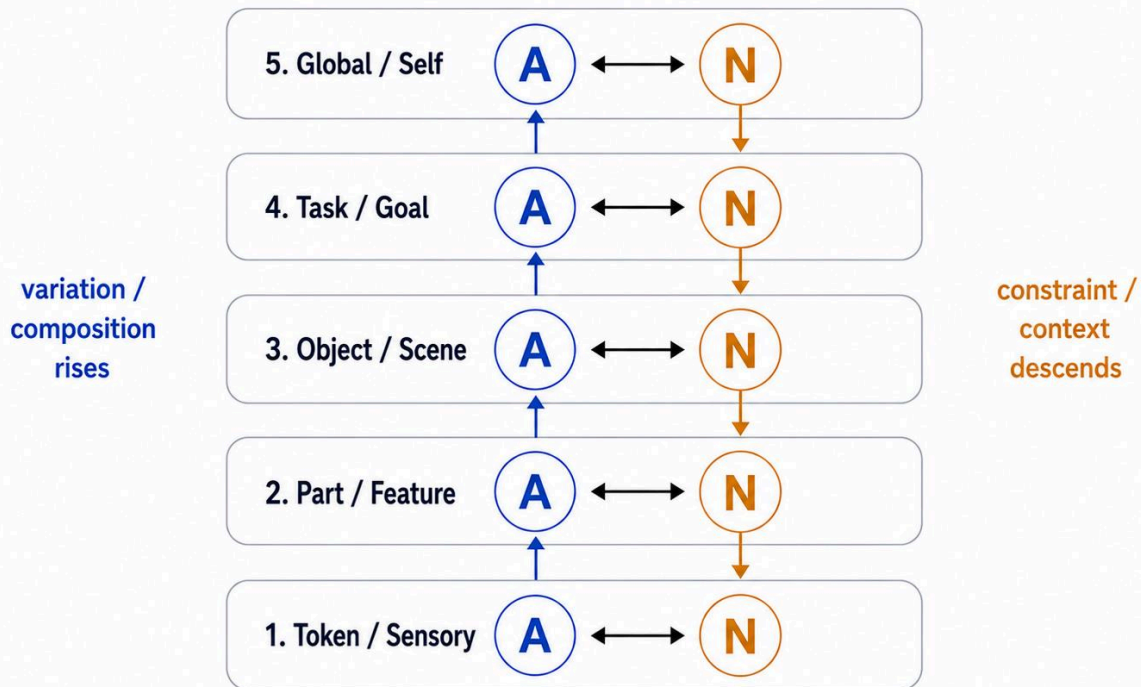
Not because the model reconstructs the world in detail, but because it preserves different kinds of continuity at different levels.

A world is not simply a stream of observations.

A world is what continues to matter across changing observations

Nested Chevron Architecture

Adjacent A/N nodes and connections



Relation to VLA

Vision-language-action models extend language models into perception and control. But the risk is that vision, language, and action become fused into one large mapping:

image + instruction → action

That may work impressively in many cases, but conceptually it hides the internal problem.

The instruction may need to persist while the visual field changes.

The object may need to persist while it is partially occluded.

The action policy may need to adapt locally without forgetting the larger goal.

A nested Chevron architecture makes these tensions explicit.

It does not simply combine vision, language, and action. It asks each modality to carry its own fast/slow polarity, and then asks the levels to negotiate.

So the Chevron alternative to flat VLA is not merely:

vision-language-action

but:

```
retained vision  
+  
retained task meaning  
+  
retained action constraint
```

all interacting through adaptive proposal channels.

Why This Matters Experimentally

This gives us a useful experimental path.

The first CPA experiments can remain small and local. We do not need robots, large video models, or internet-scale training.

We can begin with synthetic tasks that test whether nested A/N dynamics provide an advantage over ordinary MLPs and Transformers.

The first experiment tests flat CPA:

Can A/N improve prediction when hidden rules change over time?

The second experiment can test nested CPA:

Can different A/N pairs track different levels of hidden structure?

For example, a sequence may have both:

an outer regime: the general family of rules
an inner regime: the current local rule

A flat model must compress both into one state.

A nested Chevron model can separate them:

local A/N tracks the immediate rule
global A/N tracks the broader regime

This is the simplest test of nesting.

Breakout: VLA, JEPA, and the World-Model Dispute

There is a live disagreement in AI research about whether the next step beyond large language models should be built by extending LLMs outward into perception and action, or by developing a different kind of predictive world model.

On one side is the VLA direction: vision-language-action systems. These extend the success of language models into images, video, robotics, and embodied control. The basic wager is that if models can combine language, perception, and action at scale, then increasingly general forms of intelligent behaviour may emerge.

On the other side is the JEPA direction, associated especially with Yann LeCun. LeCun has argued that current LLMs are not enough for human-level intelligence because they lack robust understanding of the physical world, persistent memory, common-sense prediction, and hierarchical planning. He has proposed world-model architectures such

as JEPA and V-JEPA as alternatives or successors to purely token-predictive systems. Recent reporting has described his position as a shift away from simply scaling LLMs toward systems that learn from video and real-world tasks.

The dispute can be stated simply:

VLA: extend language models into perception and action.

JEPA: build predictive world models that learn latent structure directly from the world.

But this may be too sharp a division.

The VLA side is right that language, vision, and action cannot be kept separate forever. An intelligent system must eventually connect instruction, perception, memory, and control.

The JEPA side is right that language alone is not the world. A system trained mainly to continue text may acquire extraordinary cultural and symbolic competence, but that does not automatically give it stable

object permanence, embodied affordances, physical intuition, or action-grounded planning.

Chevron networks suggest a possible third framing.

The question is not simply whether we should start from language or from video. The question is how a system retains useful differences across changing experience.

A flat VLA system may learn:

`image + instruction → action`

A JEPA-like system may learn:

`context representation → predicted future representation`

A nested Chevron system would ask:

`What is changing quickly?`

`What should remain stable?`

`Which retained constraint should guide the next prediction or action?`

In this view, VLA and JEPA are not enemies. They are pressures on different parts of the same problem.

VLA emphasizes integration across modalities.

JEPA emphasizes prediction in latent world structure.

Chevron architecture emphasizes retained difference across timescales.

That gives a natural bridge:

```
vision needs A/N  
language needs A/N  
action needs A/N  
world-model prediction needs A/N
```

So the Chevron proposal is not that LLMs are useless, nor that JEPA solves everything or nothing. It is that any route toward world models probably needs a disciplined relation between fast adaptation and slow retention.

A proposes.

N remembers.

Prediction tests.

Action reveals the cost of being wrong.

That may be the missing common ground between VLA and JEPA.

Suggested Experiment Sequence

1. Flat CPA: regime-switching prediction

Compare MLP, tiny Transformer, and flat CPA on next-bit prediction where the hidden rule changes. Measure accuracy, recovery after switches, noise resistance, and A/N collapse.

2. Nested CPA: nested regimes

Add both a fast-changing local rule and a slower global rule family. Test whether local A/N tracks the current rule while global A/N retains the broader context.

3. Moving object with occlusion

Use a 16×16 grid with a moving dot, hidden velocity, occlusion, and distractors. Test whether A follows apparent motion while N retains object identity and trajectory.

4. Instruction-following gridworld

Use a small grid plus instruction tokens such as “go to red” or “avoid blue.” Test whether visual A/N adapts to the scene, task A/N retains the goal, and action A/N proposes constrained moves.

The Larger Claim

The deeper claim is not that Chevron networks are simply another architecture.

It is that intelligence may require multiple forms of retained difference operating at different timescales.

Perception requires retained difference.

Objecthood requires retained difference.

Language requires retained difference.

Action requires retained difference.

Selfhood may require retained difference too.

A nested Chevron architecture makes that structure explicit.

It gives us a way to move from prediction toward world models, and from world models toward action, without losing the central insight:

A proposes.

N remembers.

Prediction tests.

Inertia decides what becomes structure.

In a flat CPA, this happens once.

In a nested CPA, it happens at every level.

That may be where the architecture begins to resemble not just a predictor, but a small developmental system.



The deeper possibility is that Chevron architecture is not merely a machine-learning trick. It may be an approximation to a more general structure of intelligence: nested adaptive-retentive control. Biological brains do not simply predict. They stabilize perception, preserve objecthood, retain goals, regulate bodies, consolidate memories, and revise self-models across many timescales. The brain may not be a Chevron network in the literal engineering sense, but it may be Chevron-like in the cybernetic sense: a hierarchy of fast proposal systems constrained and reshaped by slower structures of retention. On

this view, intelligence is not optimization alone. It is the regulated transformation of experience into retained difference.

Nested Chevron architecture supplies the levels; Inertial VSR supplies the developmental grammar. At each level, A generates local variation, prediction and action provide selection pressure, and N retains what continues to matter. But because retention is inertial, N does not merely store the past. It reshapes the space of future variation. The result is not a stack of predictors, but a hierarchy of adaptive-retentive loops.

Trajectory of the First CPA Experiment

Following the previous post on [Chevron Predictive Architecture](#), I used Codex with ChatGPT-5.5 to design and run a first set of local experiments.

The aim was not to “beat” existing architectures. This is much too early for that. The aim was more modest: to make the architecture real enough to test, inspect, and improve.

The first experiments therefore focus on trajectory rather than completion. Can a minimal CPA be implemented? Can it be compared against ordinary baselines such as an MLP or a small Transformer? Can we observe the behaviour of the A and N channels directly? Do they collapse into sameness, diverge into noise, or maintain a useful adaptive-retentive tension?

In other words, the experiment is less a benchmark than a first engineering arc:

concept → implementation → baseline comparison → diagnostics → next questions

That feels like the right starting point. CPA begins as a conceptual proposal: A proposes, N remembers, prediction tests, and retention decides what becomes structure. The first experiment asks whether that idea can be turned into runnable code and measured behaviour.

The code and write up is available here:

[https://github.com/andrekrumer/chevron-networks/tree/main/chevron-pr
edictive-arch](https://github.com/andrekrumer/chevron-networks/tree/main/chevron-pr
edictive-arch)

This first CPA experiments produced a useful negative result :(CPA did not beat a tuned MLP or establish Transformer superiority. But the failure was diagnostic: naive A/N separation was not enough, while mismatch-driven retention improved the model in the direction predicted by the theory.

The current benchmark may still be too solvable by context-window pattern matching. If the entire necessary past fits within a fixed sensory window, a model does not need true temporal inertia. By moving to Nested Chevron Architectures, we transition to tasks like occlusion (Experiment 3), where the necessary context outlasts the sensory window, forcing the network to rely on retained state rather than just spatial lookups.

One practical point is also worth noting. The Transformer baseline remains important, but it is already much heavier to run locally on a MacBook.

The most theory-relevant result so far is the effect of adding explicit A/N mismatch into the retained channel update:

```
N_update += W_diffN(A - N)
```

This did not make CPA decisively superior, but it improved the model in the direction the theory predicts. The original A/N split was not enough by itself. What helped was giving N a direct way to respond to persistent difference between the adaptive proposal and the retained constraint.

That makes $A - N$ more than a diagnostic quantity: it becomes a candidate learning signal. In CPA terms, mismatch is not just error; it is the difference that may deserve retention.

A useful result for CPA that does not really even shake the goal posts.

Maybe nested chevron architectures (NCA) are more promising to look closer into next.

Thanks for reading,

Andre with ChatGPT-5.5,

June 2026

Appendix 1: Minimal Nested A/N Scheme in Python

The simplest nested Chevron scheme does not need skip connections, parallel paths, or attention. Each layer contains an A/N pair. The upward path carries adaptive proposal from the layer below. The downward path carries retained constraint from the layer above. Within each layer, A and N are locally coupled.

Mathematically, each layer performs a small Chevron update:

$$A'_l = f(W_{AA} A_l + W_{AN} N_l + U_l A_{l-1} + D_l N_{l+1})$$

$$N^*_l = f(W_{NA} A_l + W_{NN} N_l + C_l A_l)$$

$$N'_l = (1 - \rho_l) N_l + \rho_l N^*_l$$

where ρ_l is the retention rate for that layer. Lower layers can update faster; higher layers update more slowly. This gives the hierarchy its temporal structure.

A minimal PyTorch sketch:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ANLayer(nn.Module):
    """
    One nested Chevron layer with a local A/N pair.

    A = adaptive / proposal channel
    N = normative / retained channel
    """

    def __init__(self, dim, rho):
        super().__init__()
        self.rho = rho

        # Local Chevron 2x2 operator
        self.W_AA = nn.Linear(dim, dim)
        self.W_AN = nn.Linear(dim, dim)
        self.W_NA = nn.Linear(dim, dim)
```

```

self.W_NN = nn.Linear(dim, dim)

# Adjacent-only hierarchy
self.up_from_below = nn.Linear(dim, dim)
self.down_from_above = nn.Linear(dim, dim)

self.norm_A = nn.LayerNorm(dim)
self.norm_N = nn.LayerNorm(dim)

def forward(self, A, N, A_below=None, N_above=None):
    """
    A_below: proposal signal from lower layer
    N_above: retained constraint from higher layer
    """

    bottom_up = 0 if A_below is None else self.up_from_below(A_below)
    top_down = 0 if N_above is None else self.down_from_above(N_above)

    # Fast adaptive proposal
    A_update = (
        self.W_AA(A)
        + self.W_AN(N)
        + bottom_up
        + top_down
    )

    A_next = self.norm_A(A + F.gelu(A_update))

    # Slow retained constraint
    # Detaching A is optional, but helps make N a slower consolidating
channel.
    N_candidate = (
        self.W_NA(A.detach())
        + self.W_NN(N)
        + top_down
    )

    # Using the explicit mismatch (A - N) discovered in the flat CPA
experiments.
    # Detaching A prevents slow N from dragging the fast A via gradients.
    # was: mismatch = A.detach() # Detaching A is optional, but helps make
    #                               # N a slower consolidating channel.
    mismatch = A.detach() - N

    N_candidate = (
        self.W_NA(mismatch)
        + self.W_NN(N)
        + top_down
    )

```



```

N_candidate = self.norm_N(N + F.gelu(N_candidate))

# Inertial update
N_next = (1.0 - self.rho) * N + self.rho * N_candidate

return A_next, N_next

```

```

class NestedANModel(nn.Module):
    """
    Adjacent-only nested A/N hierarchy.
    No skip connections.
    No parallel paths.
    No full attention.
    """

    def __init__(self, input_dim, hidden_dim, output_dim, rhos):
        super().__init__()

        self.input_proj = nn.Linear(input_dim, hidden_dim)
        self.layers = nn.ModuleList([
            ANLayer(hidden_dim, rho=rho)
            for rho in rhos
        ])

        self.head = nn.Sequential(
            nn.Linear(2 * hidden_dim, hidden_dim),
            nn.GELU(),
            nn.Linear(hidden_dim, output_dim)
        )

    def forward(self, x):
        """
        x: batch x input_dim

        For a sequence model, call this repeatedly across time,
        carrying A_states and N_states between steps.
        """

        batch_size = x.shape[0]
        h = self.input_proj(x)

        # Initialize A/N states for this simple feedforward sketch.
        A_states = [torch.zeros_like(h) for _ in self.layers]
        N_states = [torch.zeros_like(h) for _ in self.layers]

        # Inject input into lowest adaptive layer.
        A_states[0] = h

```

```

# One upward/downward nested update pass.
# For a recurrent version, repeat this across timesteps.
new_A = []
new_N = []

for i, layer in enumerate(self.layers):
    A_below = A_states[i - 1] if i > 0 else None
    N_above = N_states[i + 1] if i < len(self.layers) - 1 else None

    A_i, N_i = layer(
        A_states[i],
        N_states[i],
        A_below=A_below,
        N_above=N_above
    )

    new_A.append(A_i)
    new_N.append(N_i)

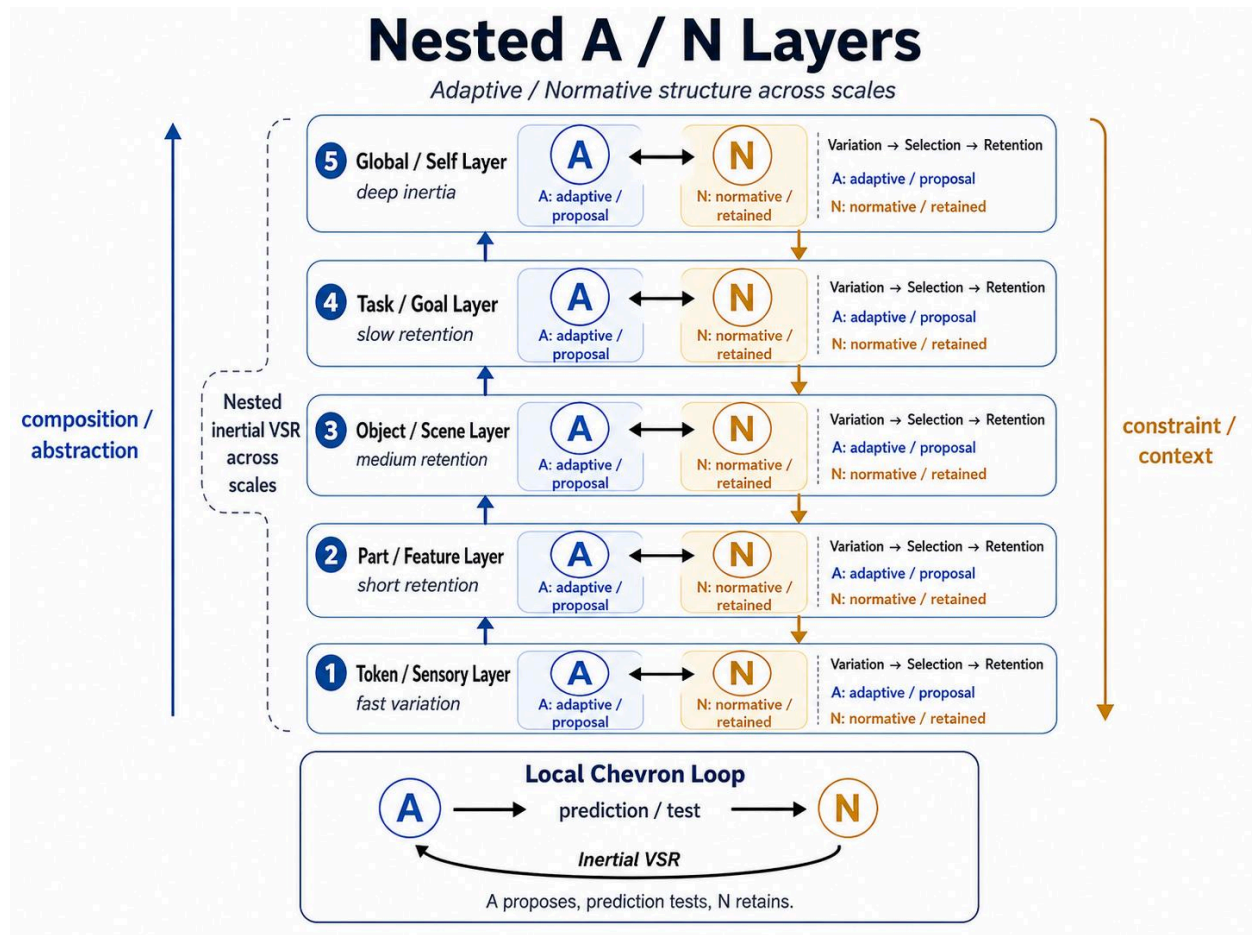
# Predict from the top layer's A/N pair.
top_A = new_A[-1]
top_N = new_N[-1]

logits = self.head(torch.cat([top_A, top_N], dim=-1))
return logits

```

In the first experiment, this should be kept deliberately plain. The point is not maximum expressiveness. The point is to test whether adjacent A/N nesting alone gives a useful inductive bias. Lower levels vary quickly; higher levels retain more slowly. Variation rises, constraint descends, and each layer runs its own small Inertial VSR loop.

Appendix 2: Connecting Nested A/N Layers



The diagram shows one way to think about nested A/N layers. The wires do not need to connect every layer to every other layer. In the simplest version, each level mainly talks to the level immediately above and below it. Variation rises upward as composition or abstraction: tokens become features, features become objects, objects become tasks, and tasks

become global context. Constraint flows downward: the retained goal, scene model, or object expectation shapes how lower-level variation is interpreted.

This keeps the architecture disciplined. A nested Chevron system is not just “everything connected to everything.” It is a hierarchy of local A/N negotiations. At each layer, A proposes and N retains; prediction or action tests the proposal; useful difference is then passed upward or stabilized downward. Later versions could add skip connections or attention across layers, but the first principle is simpler: variation rises, constraint descends, retention accumulates.

Fully connected wiring is possible, but it changes the character of the experiment. Once every layer can talk to every other layer, the architecture starts to resemble the Transformer intuition: maximum lookup, maximum mixing, maximum fan-out. That may be powerful, but it also makes the Chevron claim harder to inspect. Did the model benefit from retained A/N difference, or simply from giving every representation access to every other representation?

A more useful middle path is to allow disciplined skip connections and parallel A/N streams. Upward skips can carry variation, surprise, or A/N mismatch; downward skips can carry retained context, constraint, or task expectation. Parallel paths can specialize: one for vision, one for language, one for action, one for memory or norm. These paths do not need to be fully mixed. They can meet at bottlenecked integration Chevrons, where cross-modal proposals are tested against retained constraints. In this version, the architecture is not “everything connected to everything.” It is many adaptive-retentive loops, selectively composed.

The top layer is labelled “Global / Self” in the diagram, but this should be read as a placeholder for the slowest retained context in the system. In early experiments this may simply be a global task or regime state. Only in richer multi-task settings would it make sense to ask whether something more executive, self-like begins to emerge.

There is a safety trade-off here. Fully connected or attention-like wiring may increase capability, but it also risks weakening the very discipline that makes Chevron architecture interesting. If every layer can query

every other layer, the system begins to resemble the Transformer intuition: maximum lookup, maximum mixing, maximum fan-out. That may be powerful, but it makes the A/N structure harder to inspect. The model may solve the task by routing around retained constraint rather than by negotiating with it.

For that reason, the first safety-oriented version should avoid unrestricted connectivity. Parallel paths are acceptable, and probably necessary, but they should meet through bottlenecked integration Chevrons. Upward paths should carry variation, mismatch, or surprise; downward paths should carry context, constraint, or retained expectation. The safety benefit is not merely the presence of A and N, but the discipline imposed on their interaction: variation rises, constraint descends, retention changes slowly, and no path should be able to bypass that negotiation too easily.

The connection advantage of Chevrons is not simply that they add a second path. It is that they make the paths typed. A single stream can mix signals, but it does not distinguish proposal from constraint,

novelty from retention, or mismatch from coherence except implicitly.

A Chevron block gives those roles architectural form. The diagonal paths preserve adaptive and retained continuity; the off-diagonal paths allow proposal to perturb memory and memory to constrain proposal.

This creates a more disciplined form of mixing than unrestricted fan-out.

In that sense, Chevrons provide a small plus/minus grammar for connection. Agreement between A and N can reinforce stability; difference between A and N can mark surprise, novelty, or candidate change. Higher layers can receive not just “a representation,” but a typed signal: current proposal, retained constraint, or their mismatch. That may be the real advantage over a single-path hierarchy: not more connection, but more meaningful connection.



Inertial Difference Learning

How Systems Change Without Losing Themselves

[Andre Kramer](#)

Jun 11, 2026



Learning is often described as error correction.

A system makes a prediction.

The prediction is wrong.

The system updates.

Thanks for reading Andre's Baccalaureus in Arte Ingeniaria! Subscribe for free to receive new posts and support my work.

That picture is powerful, but it may be too simple.

Living systems do not update themselves from every difference they encounter. If they did, they would be overwritten by noise. A person does not change their whole character because of one bad day. A culture does not rewrite its conventions because of one exception. A body does not abandon its temperature regulation because of one gust of cold air.

Adaptive systems need to change. But they also need to remain coherent.

This suggests a different question:

Which differences should be retained?

That is the starting point for what I am calling **Inertial Difference Learning**.

The idea is simple:

Learning is not merely reducing error.

Learning is the controlled retention of difference under inertia.

Brief differences may be noise.

Repeated differences may be evidence.

Useful persistent differences may become structure.

Two Channels

Imagine a system with two internal channels.

The first channel is fast. It adapts to the present. It responds to current input, pressure, opportunity, novelty, and disturbance. Call this channel **A**.

The second channel is slower. It carries retained structure: memory, constraint, habit, norm, expectation, or stability. Call this channel **N**.

So:

A = adaptive proposal
N = retained constraint
A - N = internal difference or tension

A moves quickly.

N moves slowly.

The difference between them marks what is pressing for change.

The goal is not to eliminate this difference. If A and N become identical, the system becomes rigid. If A and N drift too far apart, the system

becomes incoherent. The useful regime lies between: enough difference to adapt, enough inertia to remain stable.

The Chevron

This two-channel idea can be represented as a simple chevron.

The two arms of the chevron are A and N. Their lengths, angle, and orientation describe the relation between adaptation and retention. In a neural architecture, this can be represented as a paired state:

$$\mathbf{x} = [\mathbf{A}, \mathbf{N}]$$

Instead of one stream passing through one weight matrix, the two channels interact through a 2×2 operator:

$$\begin{bmatrix} \mathbf{A}' \\ \mathbf{N}' \end{bmatrix} = \begin{bmatrix} \mathbf{W}_{AA} & \mathbf{W}_{AN} \\ \mathbf{W}_{NA} & \mathbf{W}_{NN} \end{bmatrix} \begin{bmatrix} \mathbf{A} \\ \mathbf{N} \end{bmatrix}$$

Each term has a different meaning:

$\mathbf{W}_{AA}$: A preserves and transforms its own adaptive movement
 $\mathbf{W}_{NN}$: N preserves and transforms retained structure
 $\mathbf{W}_{AN}$: N constrains or steers A

W_{NA}: A perturbs or updates N

The important part is the cross-coupling. A can influence N, but N can also constrain A. Learning is not just movement. It is negotiated movement.

Difference Is Not Always Error

The signal $A - N$ should not be understood only as an error.

It may be:

noise
novelty
surprise
pressure
temptation
creative deviation
evidence of a real change

The question is not:

How do we eliminate the difference?

The better question is:

What kind of difference is this?

A brief difference may be ignored. A repeated difference may need attention. A useful persistent difference may be retained.

This makes learning less like chasing every correction and more like deciding which differences deserve to overcome inertia.

A Familiar Pattern

This idea is not new in every part.

Backpropagation learns from error gradients.

Temporal-difference learning learns from discrepancies between predictions.

Kalman filters update from innovation.

Bayesian inference updates belief from evidence.

Predictive coding makes prediction error central.

Evolution works through variation, selection, and retention.

Inertial Difference Learning does not replace these ideas. It reframes them around a simple internal structure:

```
A varies.  
N resists.  
Difference tests inertia.  
Useful difference becomes structure.
```

The possible contribution is not “learning from difference” as such. That is already everywhere. The more specific idea is **difference-driven retention**: a system should not merely change because difference appears; it should regulate which differences are allowed to become part of itself.

Two Time Differences

This can be made more mathematical.

Let:

A_t = adaptive state at time t
 N_t = retained state at time t
 $D_t = A_t - N_t$

There are two temporal differences:

$\Delta A_t = A_t - A_{t-1}$
 $\Delta N_t = N_t - N_{t-1}$

A should usually change faster than N:

$||\Delta A_t|| > ||\Delta N_t||$

But N should not be frozen. It should update when difference persists.

A simple persistence signal might be:

$P_t = \beta P_{t-1} + (1 - \beta) ||A_t - N_t||$

Then N updates only when persistent difference becomes strong enough:


```

$$\rho_t = \text{sigmoid}(k(P_t - \theta))$$

$$N_{t+1} = N_t + \rho_t \cdot h(A_t - N_t)$$

```

This says:

do not retain every difference; retain difference that persists.

That is the core of the idea.

“Here, P_t tracks the momentum of the mismatch. It acts as a low-pass filter for surprise, ensuring that N only absorbs differences that refuse to go away.”

Orbiting a Viability Zone

Many adaptive systems do not optimize one goal to the maximum. They orbit around a viable zone.

A body does not maximize temperature.

A culture does not maximize novelty.

A person does not maximize certainty, freedom, comfort, or risk.

A healthy system keeps many tensions within workable ranges.

So the aim is not:

$$A = N$$

The aim is something more like:

$$r_{\min} < ||A - N|| < r_{\max}$$

Too little difference means rigidity.

Too much difference means instability.

The useful regime is regulated tension.

This may be closer to biological intelligence than single-objective optimization. Life is not simply the pursuit of an optimum. It is the preservation of coherence under change.

Breakout: Learning From Difference

Inertial Difference Learning is not proposed in isolation. Many existing learning methods already depend on some form of difference, error, or discrepancy.

Backpropagation learns from error gradients. A model produces an output, the loss measures how wrong it was, and the error is propagated backward to update the weights. This is extremely powerful, but the update is usually global and optimization-driven: reduce the loss.

Temporal-difference learning learns from the gap between one prediction and a later, updated prediction. In reinforcement learning, the system does not wait until the final outcome before learning. It updates from the difference between what it expected and what later evidence suggests the value should have been.

Kalman filters learn from innovation: the difference between a predicted measurement and an observed measurement. The retained state is not simply overwritten by new evidence. It is adjusted according to how much the system should trust the new signal relative to the prior estimate.

Bayesian inference gives the formal version: prior belief is updated by evidence. But exact Bayesian updating is often computationally expensive, so biological and artificial systems usually need approximations.

Predictive coding places prediction error at the centre of perception and learning. Higher levels predict lower-level signals; lower levels return error; the system updates to reduce the discrepancy.

Evolutionary learning works through variation, selection, and retention. Differences arise, the environment selects among them, and some become retained structure.

Inertial Difference Learning does not replace these ideas. It tries to reorganize them around an internal distinction:

```
A = adaptive proposal  
N = retained constraint  
A - N = internal difference or tension
```

The emphasis is not simply on reducing error. It is on deciding which differences should overcome inertia and become part of the system. Brief difference may be noise. Repeated difference may be evidence. Useful persistent difference may become structure.

So the proposed shift is:

```
from: learning as error minimization  
to: learning as regulated retention of difference
```

That is the sense in which the idea is both familiar and different. It belongs to the family of error-based and difference-based learning methods, but it gives the difference an internal architecture: fast adaptation pressing against slow retention.

A Note on Cross-entropy:

Cross-entropy is a selection pressure. It tells the system how wrong its output was. $A - N$ is a candidate retention signal. The A/N difference tells the system how much its fast proposal has diverged from retained structure. Inertial Difference Learning becomes interesting when these two signals are coupled: external error selects which internal differences matter, and only persistent useful differences are allowed to change N .

Cause as Retained Difference

There is a useful connection here to Patrick Suppes' probabilistic theory of causality.

On a Suppes-like view, a cause does not need to determine its effect absolutely. It is enough that it raises the probability of that effect under the right temporal and background conditions. Cause is therefore not pure necessity. It is a patterned probabilistic difference.

That fits naturally with Inertial Difference Learning.

A useful difference is not one that guarantees an outcome. It is one that reliably shifts the probability of what follows. In A/N terms:

A = current variation or adaptive proposal
N = retained regularity or constraint
A - N = difference pressing against retained structure

If that difference appears once, it may be noise. If it recurs and changes future outcomes, it becomes causally interesting. If it continues to matter, it becomes a candidate for retention.

This suggests a deeper way to think about learning. A system does not merely store facts or minimize errors. It gradually retains differences that make a difference to what happens next.

Evolution works this way. Individual learning works this way. Culture works this way. Differences arise, consequences select among them, and some become retained structure.

In this sense, nature may “learn” cause and effect not by discovering timeless necessity all at once, but by retaining probabilistic differences that survive recurrence.

Cause, then, is difference with a future.

Alignment Over Time

This also suggests a useful rephrasing of AI safety.

Alignment is often discussed as if it were a static property:

Does the system have the right goal?

But adaptive systems raise a harder question:

Does the system remain aligned while it changes?

Many failures are temporal failures. A temporary reward becomes a habit. A local shortcut becomes a policy. A misleading signal becomes retained structure. A system adapts too quickly to something that should have remained local.

In A/N terms:

A = local adaptation, opportunity, pressure, shortcut
N = retained constraint, norm, task boundary
A - N = pressure toward drift

Safety then becomes a question of **alignment over time**.

A safer adaptive system is not one that never changes. It is one that changes under constraint, and whose constraints do not update as quickly as its opportunities.

Automation and Nature

There is also a wider philosophical reading.

A can stand for the **automated**: procedure, acceleration, optimization, machinery, execution.

N can stand for **nature**: embodiment, ecological rhythm, inherited form, viability, and the slower order that automation perturbs but also depends on.

This should not be forced into every technical detail. But as a philosophical image, it is useful.

Automation without nature becomes runaway optimization.

Nature without automation becomes inertia without transformation.

The question is whether the automated can remain answerable to the natural. Can fast artificial adaptation remain constrained by slower structures of viability, embodiment, culture, and care?

A must move.

N must not be overwritten too cheaply.

Breakout: The Cybernetic Connection

Cybernetics begins from a simple but powerful idea: intelligent systems are regulated by feedback. A system acts, senses the consequence of its action, compares that consequence with some retained condition, and adjusts.

In that sense, Inertial Difference Learning is deeply cybernetic.

The A channel carries adaptive movement: action, variation, correction, local response. The N channel carries retained constraint: set-point, norm, memory, goal, or boundary condition. The difference between them is the feedback signal that says the system is drifting, adapting, resisting, or transforming.

Classic negative feedback tries to reduce deviation from a set-point. But living and intelligent systems are not only thermostats. They do not merely return to one fixed value. They also learn new ranges, develop new habits, shift goals, and revise constraints. That is where inertia matters. N should resist change, but not absolutely. A persistent difference may eventually become a new retained structure.

So the cybernetic loop becomes:

act → sense → compare → adjust → retain

or, in A/N terms:

A varies
N constrains
difference feeds back
persistent difference updates N

This reframes learning as regulated feedback over time. The system is not simply optimizing a target. It is maintaining coherence while changing. Too little feedback and it drifts blindly. Too much immediate correction and it chases noise. Too much inertia and it cannot adapt. The

intelligent regime lies between: responsive enough to move, stable enough to remain itself.

That is why cybernetics remains relevant. It reminds us that intelligence is not just representation or computation. It is circular causality: a system acting into a world, receiving difference back, and deciding what difference should change its future form.

A Speculation on Culture as Retained Difference

Culture may be the most widely spread human example of inertial difference learning.

Conventions are slow structures: manners, rituals, laws, genres, institutions, scientific methods, professional norms, inherited ways of seeing. They constrain local behaviour so that every interaction does not have to be invented from scratch.

Local life supplies variation: exceptions, mistakes, play, rebellion, new technologies, new pressures, new encounters.

Most differences vanish. Some are punished. Some are ignored. Some become fashion. A few persist, spread, and become retained convention.

Culture is therefore an inertial system of variation, selection, and retention:

variation in practice
selection through consequence
retention as convention
inertia as constraint on future variation

In A/N terms, culture is a vast nested system. A lives in local improvisation. N lives in retained form. Meaning emerges in the tension between them.

This also recalls Spinoza's distinction between **potentia** and **potestas**: power as living capacity to act, and power as authorized or institutional form.

Potentia is close to A: local force, capacity, variation, what bodies and groups can actually do.

Potestas is close to N: retained authority, institution, law, office, constraint.

Neither side is sufficient by itself. Authority without living capacity becomes empty. Capacity without retained form dissipates. Culture lives in the loop between them.

Breakout: Is Inhibition the Brain's N Channel?

One biological analogy for N is neural inhibition. This should not be taken literally as “N equals inhibition,” but inhibition does perform many N-like functions. It constrains excitation, regulates gain, shapes timing, gates inputs, stabilizes circuits, and helps prevent runaway activity. In this sense, inhibition is not merely a brake. It is part of the

architecture that lets neural systems remain plastic without dissolving into noise.

In Chevron terms, excitation is A-like: adaptive activation, current pressure, local proposal. Inhibition is N-like: constraint, selection, timing, and stabilization. Intelligence may depend less on either side alone than on their regulated relation. Too much A produces instability; too much N produces suppression. The useful regime is balanced tension.

This also connects to predictive-learning theories of cortex. Some models suggest that inhibitory circuits help shape prediction-error signals or determine when plasticity should occur. That is close to the Inertial Difference Learning idea: learning is not triggered by activity alone, but by structured difference under constraint. If A is what the circuit is currently trying to express, N is the inhibitory-retentive structure that determines whether that expression is allowed, damped, timed, or eventually learned.

Why Small Experiments Matter

This idea does not need to begin at massive scale.

The first question is not whether it can outperform today's largest systems. The first question is whether the distinction between A, N, and $A - N$ can be made experimentally visible.

Can A track fast local change?

Can N retain slower structure?

Can $A - N$ indicate when retained structure should update?

Can brief difference be ignored while persistent difference is absorbed?

Can safety-relevant constraints update more slowly than local opportunities?

These are small-system questions. They can be tested with simple sequence tasks, grid worlds, moving-object tasks, and controlled perturbations.

If the answer is no, the idea remains a metaphor.

If the answer is yes, then Inertial Difference Learning becomes a possible research programme.

Not a replacement for existing learning theory.

Not a claim that everything needs revision.

But a synthesis around a basic question:

How does a system remain itself while changing?

That may be the deepest learning problem.

A resonance: Grossberg, ART, and Gated Dipoles

A close historical neighbour to this line of thought is Stephen Grossberg's work on Adaptive Resonance Theory, or ART. ART was designed around the stability-plasticity dilemma: how can a system learn quickly without catastrophically forgetting what it has already learned? That is very close to the A/N problem. A system needs plasticity, but also retained structure. It needs to respond to novelty without being overwritten by noise.

In Chevron language, ART can be read as an earlier dynamical answer to the same problem. The current input presses for adaptation; retained categories constrain recognition; learning occurs only when the relation between input and memory becomes stable enough to resonate. If the mismatch is too great, the system does not simply overwrite memory. It searches, resets, or forms a new category.

Grossberg's gated dipoles are also suggestive. They model opponent processes using paired, gated dynamics rather than a single undifferentiated activation stream. That has a family resemblance to the Chevron idea: intelligence may depend on structured paired channels, with tension, gating, and balance between them.

The main difference is mathematical style. Grossberg's models are often expressed as systems of differential equations, while the first Chevron experiments use discrete neural-network updates. But these may be two views of the same deeper pattern. A discrete update can be read as a stepwise approximation to continuous dynamics:

$$\begin{aligned}A_{t+1} - A_t &\approx \Delta t \cdot dA/dt \\ N_{t+1} - N_t &\approx \Delta t \cdot dN/dt\end{aligned}$$

So the connection is not that Chevron networks replace ART. It is that Inertial Difference Learning may belong to the same family of ideas: real-time adaptive systems that regulate the tension between plasticity and stability.

ART's **vigilance parameter** is especially relevant here. In Adaptive Resonance Theory, vigilance determines how much mismatch the system will tolerate before deciding that an input does not fit an existing category. If the mismatch is small enough, the system can assimilate the input into the retained category. If the mismatch is too large, the system searches for or forms a new category.

In Inertial Difference Learning, the analogous role is played by the threshold θ in the retention gate:

$$P_t = \beta P_{t-1} + (1 - \beta) ||A_t - N_t||$$
$$\rho_t = \text{sigmoid}(k(P_t - \theta))$$

Here P_t measures persistent A/N difference. The threshold θ regulates when that difference crosses from acceptable adaptive tension into a signal that retained structure may need to change.

So ART and IDL share the same underlying problem: how much difference should a system tolerate before it treats the difference as structurally meaningful? Too low a threshold and the system overlearns

noise. Too high a threshold and it becomes rigid. The interesting regime lies between: enough vigilance to detect real novelty, enough inertia to preserve stable structure.



This post is mostly a reframing of [Chevron Networks](#), Inertial VSR ideas, and the earlier [redictive architecture](#) work, focused on what now seems to be the central promise: continuous, non-collapsing learning.

The maths in the Appendix is deliberately minimal. It introduces the A/N pair, the 2×2 Chevron operator, and a possible complex-number

version that connects back to the [R-rule](#) as rotation in the A/N plane.

The point is not to claim a finished theory, but to simplify the threads I have been circling for the past year into a more testable form.

The working hypothesis is Inertial Difference Learning: systems learn not merely by reducing error, but by regulating which differences are allowed to overcome inertia and become retained structure. That feels like an important synthesis. The difficult task, of course, is proving that it works.

A first small first test makes the proposed IDL mechanism visible:

<https://github.com/andrekrumer/chevron-networks/tree/main/inertial-difference>

The experiment used a simple online classification stream with three regimes: a stable pattern, brief contradictory bursts, and then a persistent shift to a new pattern. Three models were compared: a standard MLP, a slow A/N chevron, and an IDL-gated chevron where the

retained channel N updates more strongly only when the internal A-N difference persists.

The important early result is not that IDL wins outright. It does not, at least not in this first harness. The standard MLP adapts faster immediately after the shift. But the IDL model shows the predicted internal behaviour: during short contradictory bursts, its retention gate stays relatively low; during the persistent shift, the gate rises sharply.

In numbers, the IDL-gated model had average retention strength $\rho = 0.609$ during transient bursts, but $\rho = 0.944$ during the first 200 steps after the real shift. That is the core signal the theory predicts: brief difference is treated cautiously, while persistent difference is allowed to update retained structure.

So the first result is modest but useful. The A/N distinction is experimentally visible. The A-N difference can be tracked. And a persistence-gated retention rule can distinguish temporary disturbance from sustained change. This does not yet prove that IDL is better than

ordinary error-driven learning, but it shows that the proposed mechanism can be implemented and measured.

Andre with ChatGPT-5.5,

June 2026

UPDATE

Second Small Experiment: Resisting Temporary Difference

The first experiment showed that the A/N distinction produces a measurable internal signal. The next question was whether that signal can do useful work.

In particular: can a system avoid retaining temporary difference, while still adapting to persistent difference?

To test this, I ran a second small online classification experiment. The stream had four phases.

First, the model learned a stable concept. Then it encountered a short contradictory concept. This contradiction was not meant to represent a real change in the world; it was a temporary disturbance. After that, the stream returned to the original stable concept. Later, the contradictory concept returned again, but this time it persisted. That final phase represented a real shift.

The comparison was between three models:

- a standard MLP,
- a slow A/N chevron where N always updates,
- an IDL-gated chevron where persistent excess A-N difference controls retention into N.

The diagnostic was not simply accuracy. The key question was how much the retained channel N moved during the temporary disturbance, and whether the retention gate opened during the later persistent shift.

The result was encouraging. During the brief temporary difference, the IDL-gated model moved N about half as much as the always-updating chevron. In the latest run, the N movement during the temporary phase was about 0.0241 for the IDL-gated model, compared with about 0.0502 for the slow chevron.

But when the persistent shift arrived later, the IDL gate opened strongly. The average retention strength during the shift-adaptation window was about 0.899.

The external recovery metric points in the same direction, though more modestly. Using a 20-step rolling accuracy window, the IDL-gated model returned to within one percentage point of its pre-disturbance baseline after about 24 steps, compared with about 34 steps for the MLP. So the gated model recovered roughly 10 steps faster than the ordinary MLP, consistent with the idea that it had retained less of the temporary disturbance. This advantage was not dramatic, and it did not clearly beat the slow chevron on this recovery

metric, but it gives an external performance counterpart to the internal N-movement result.

This is a more meaningful result than the first signal-detection test. The first test showed that an A-N signal exists. This second test shows that the signal can begin to regulate retention: brief difference is partly resisted, while sustained difference is treated as a candidate for structural change.

That is close to the core claim of Inertial Difference Learning. Learning is not just reaction to error. It is selective retention of difference under inertia.

Appendix: Basic Chevron Maths, and a Complex Version

The minimal Chevron state has two channels:

A = adaptive / proposal channel
N = retained / constraint channel

So the state can be written as a pair:

$x = [A, N]$

A basic Chevron block updates the pair with a 2×2 operator:

$$\begin{bmatrix} A' \\ N' \end{bmatrix} = \begin{bmatrix} W_{AA} & W_{AN} \\ W_{NA} & W_{NN} \end{bmatrix} \begin{bmatrix} A \\ N \end{bmatrix}$$

or, expanded:

$$\begin{aligned} A' &= W_{AA} A + W_{AN} N \\ N' &= W_{NA} A + W_{NN} N \end{aligned}$$

Each path has a role:

W_{AA}: adaptive continuity
W_{NN}: retained continuity
W_{AN}: retained structure constrains A
W_{NA}: adaptive pressure updates N

The internal difference is:

$D = A - N$

After the Chevron update:

$$D' = A' - N'$$

Substituting the two equations:

$$D' = (W_{AA} A + W_{AN} N) - (W_{NA} A + W_{NN} N)$$

so:

$$D' = (W_{AA} - W_{NA}) A + (W_{AN} - W_{NN}) N$$

This shows why the 2×2 form matters. The difference between A and N is shaped not only by their separate persistence, but also by their cross-coupling. A can perturb N; N can constrain A; their difference becomes a structured signal rather than a simple error.

A compact update rule for retained learning might therefore be:

$$N_{next} = N + \rho \cdot h(A - N)$$

where ρ is the retention rate. If ρ is small, N changes slowly. If ρ is learned or gated, the system can decide when difference is strong enough to become retained structure.

Complex Chevron

There is also a more constrained version using complex numbers.

Write the paired state as:

$$z = A + iN$$

where:

$$\begin{aligned} A &= \text{Re}(z) \\ N &= \text{Im}(z) \end{aligned}$$

Now A and N are not just two separate channels. They are two components of one complex state.

A complex weight can be written as:

$$w = \alpha + i\beta$$

Then the update is:

$$z' = w z$$

Expanding:

$$z' = (\alpha + i\beta) (A + iN)$$

gives:

$$\begin{aligned} A' &= \alpha A - \beta N \\ N' &= \beta A + \alpha N \end{aligned}$$

In matrix form:

$$\begin{bmatrix} A' \\ N' \end{bmatrix} = \begin{bmatrix} \alpha & -\beta \\ \beta & \alpha \end{bmatrix} \begin{bmatrix} A \\ N \end{bmatrix}$$

So complex multiplication is a special case of the 2×2 Chevron block:

$$\begin{aligned} w_{AA} &= \alpha \\ w_{AN} &= -\beta \\ w_{NA} &= \beta \\ w_{NN} &= \alpha \end{aligned}$$

This is less expressive than a fully general 2×2 matrix, but more disciplined. It ties the diagonal terms together and makes the

off-diagonal terms sign-reversed. The result is a natural rotation in the A/N plane.

Using polar form:

$$w = r e^{i\varphi}$$

the update becomes:

$$z' = r e^{i\varphi} z$$

which means:

scale by r
rotate by φ

In Chevron terms:

r = strength or gain
 φ = rotation between adaptation and retention

This gives a different way to think about learning. The system is not merely moving toward a point. It may be rotating within an A/N plane, regulating the relation between fast adaptation and slow retention.

The magnitude of the complex state is:

$$|z| = \sqrt{A^2 + N^2}$$

and the phase is:

$$\theta = \text{atan2}(N, A)$$

So a complex Chevron can regulate both:

how strong the state is
how adaptation and retention are balanced

Instead of minimizing $A - N$ to zero, a complex Chevron might keep the system inside a viability band:

$$\begin{aligned} r_{\min} &< |z| < r_{\max} \\ \theta_{\min} &< \arg(z) < \theta_{\max} \end{aligned}$$

That may be the mathematical form of productive tension: not collapse into sameness, not divergence into chaos, but controlled movement in the space between adaptation and retention.



IDL Experiments, Part 2

Why difference matters

[Andre Kramer](#)

Jun 23, 2026



This post builds on the previous discussion of [Inertial Difference Learning](#) (IDL) and [Chevron Networks](#). The core idea is simple: instead of passing scalar activations through single weighted connections, we represent each unit with two coupled channels, (A) and (N), connected by learned (2 times 2) transformations. The (A) channel adapts quickly,

while the (N) channel changes more slowly and preserves a retained norm. In our [recent first two experiments](#), the difference between them, (A-N), has produced the clearest and most useful signal so far.

3rd Small Experiment: Catastrophic Forgetting

The first experiment showed that the A/N distinction produces a measurable internal signal.

The second experiment showed that the signal can do some useful work: the IDL-gated model retained less of a brief temporary disturbance, while still opening retention during a later persistent shift.

The third experiment asks a harder question:

Can this help with catastrophic forgetting?

Catastrophic forgetting happens when a model learns a new task and loses performance on an earlier task. In ordinary neural networks, this is

a familiar problem. If the same weights are used to absorb every new pattern, then learning Task 2 can overwrite what was learned for Task 1.

In A/N terms, this is exactly the kind of problem IDL is meant to address. A should be able to adapt quickly to new pressure. But N should not be overwritten too cheaply.

So I ran a simple sequential learning test.

The model first learned Task 1 until performance was stable. Then it learned Task 2, with no replay of Task 1. During and after this second phase, I probed both tasks.

The comparison was between three models:

- a standard MLP,
- a slow A/N chevron where N always updates,
- an IDL-gated chevron where A adapts quickly but N is protected by a retention gate.

The important question was not only whether the model could learn Task 2. It was whether any part of the system still retained Task 1 after Task 2 learning began.

The result was encouraging.

After Task 1, all models reached about 0.99 accuracy on Task 1.

After Task 2, the standard MLP had learned Task 2, reaching about 0.950 accuracy. But its Task 1 accuracy had fallen to about 0.163. This is the expected catastrophic forgetting pattern: the new task was learned by overwriting the old one.

The slow chevron behaved similarly. It learned Task 2 well, reaching about 0.961, but its retained N channel also largely converted to Task 2. N retained only about 0.192 accuracy on Task 1.

The IDL-gated chevron behaved differently. It also learned Task 2, reaching about 0.958 combined Task 2 accuracy. But its N channel still retained about 0.784 accuracy on Task 1.

That is the key result.

The IDL model did not prevent the adaptive channel from learning the new task. A moved toward Task 2. But N retained much more of Task 1 than either the MLP or the always-updating chevron.

This matters because it shows a possible functional role for the A/N split. The system can contain two temporal attitudes at once:

- A: adapt to the new task,
- N: preserve older structure unless the new difference earns retention.

This does not solve catastrophic forgetting outright. In longer Task 2 runs, N eventually begins to consolidate Task 2 as well. That is not necessarily a failure. It shows the tradeoff IDL is designed to regulate:

protect old structure from immediate overwrite, but allow persistent new structure to be retained over time.

So the result should be stated carefully.

IDL does not make forgetting disappear. But it creates a useful window in which new learning and old retention coexist. The adaptive part of the system can learn Task 2, while the retained part still carries Task 1.

That is much closer to the original motivation for Inertial Difference Learning:

learning is not merely changing in response to error.

Learning is deciding which differences should become structure.

4th Small Experiment: Recovering an Old Task

The third experiment showed that IDL can preserve older structure inside the retained channel N while the adaptive channel A learns a new task.

But that leaves an important question.

It is not enough for N to “remember” something internally. Does that retained structure actually help behavior?

To test this, I ran a fourth experiment.

The sequence was simple:

1. The model learned Task 1.
2. The model then learned Task 2, without replaying Task 1.
3. The stream then returned to Task 1.

The question was whether the IDL model could recover Task 1 faster because N had preserved more Task 1 structure during Task 2.

The comparison was again between three models:

- a standard MLP,
- a slow A/N chevron,
- an IDL-gated chevron.

In this experiment I added an explicit N-to-A recovery constraint during Task 1 phases. In other words, when Task 1 returned, retained structure in N was allowed to steer the adaptive channel A. This matters because memory is only useful if it can influence future adaptation.

The result was modest but meaningful.

After Task 2, the IDL-gated model retained much more Task 1 structure in N than the slow chevron. The retained Task 1 accuracy in N was about 0.816 for IDL, compared with about 0.260 for the slow chevron.

When Task 1 returned, this internal retention translated into faster behavioral recovery. After only 10 return steps, IDL reached about 0.601 Task 1 accuracy, compared with about 0.436 for the MLP.

Measured another way, IDL reached 95% of its Task 1 baseline in about 26.25 return steps. The MLP took about 30.0 steps, and the slow chevron took about 33.75.

This is not a dramatic victory. But it is important because it closes a loop left open by the previous experiment.

The third experiment showed that N can preserve an older task while A learns a new one. The fourth experiment shows that this retained structure can help the system recover the old task faster when it returns.

There is also a useful caveat. Without the N-to-A recovery constraint, the retained memory in N was visible internally but did not strongly improve the combined output. That suggests that retention alone is not enough.

A useful IDL system needs both:

- a way to protect retained structure,
- a way for retained structure to guide later adaptation.

This fits the broader theory. N should not just be an archive. It should be a constraint: a slower structure that can shape future learning.

So the fourth result strengthens the IDL picture.

Learning is not only adapting to what is new. It is also keeping enough structure from the past that returning conditions can be recognized and recovered quickly.

5th Small Experiment: Fast Guesses, Slow Knowledge

The first four experiments tested whether the A/N structure could produce a signal, resist temporary difference, reduce catastrophic forgetting, and help recover an old task.

The next question was different.

Can A learn quickly without forcing N to treat every quick adaptation as knowledge?

This matters because rapid learning is useful. A system should sometimes jump to a provisional conclusion. If a new pattern appears, the system should be able to respond quickly. But it should not necessarily write that conclusion into long-term structure after one example.

In IDL terms:

- A should be allowed to guess quickly.
- N should consolidate slowly.
- Repeated useful guesses may become knowledge.
- One-off guesses should remain provisional.

To test this, I ran a fifth small experiment.

The model first learned a stable background task. Then it encountered a sequence of small local exception rules. Some rules appeared only once. Others appeared repeatedly across episodes.

For the chevron models, each episode began by copying N into A. A then adapted quickly to a tiny support set. After the episode, N had the opportunity to consolidate toward A.

The comparison was:

- a standard MLP,
- a slow chevron where N consolidated after every episode,
- an IDL-gated chevron where N consolidated mainly when a rule recurred.

The key question was not just whether the model could learn the one-shot exception. It was whether it could keep one-shot adaptation provisional while consolidating repeated structure.

The result was encouraging.

The IDL-gated model still adapted rapidly to one-shot rules. Its provisional accuracy on one-shot exceptions was about 0.927. The MLP was higher, at about 0.963, so IDL was not the most aggressive one-shot learner.

But the important difference appeared in what became retained structure.

For recurring rules, IDL consolidated into N about as well as the slow chevron: about 0.696 for IDL versus 0.698 for the slow chevron.

For one-shot rules, IDL retained less in N: about 0.454, compared with 0.513 for the slow chevron and 0.595 for the MLP.

This gives a useful selectivity measure: retained accuracy on recurring rules minus retained accuracy on one-shot rules. Higher means the system is better at distinguishing repeated structure from one-off adaptation.

On this measure, IDL did best:

- MLP: 0.175
- slow chevron: 0.185
- IDL-gated chevron: 0.242

IDL also preserved the background task best: about 0.806, compared with 0.786 for the slow chevron and 0.727 for the MLP.

So the result is not that IDL simply learns fastest. It does not. The MLP is more aggressive on one-shot adaptation.

The result is more specific:

IDL allows fast provisional adaptation in A, while making N more selective about what becomes retained structure.

That is close to the original intuition.

A can jump to conclusions.

N decides which conclusions are allowed to become knowledge.

This gives a different way to think about rapid learning. The goal is not always to make the whole system update as fast as possible. Sometimes

the useful architecture is split: a fast channel for guesses, and a slow channel for commitment.

Summary

1. **Experiment 1:** Showed the **A-N** signal exists. ([previous post](#))
2. **Experiment 2:** Showed it can filter temporary noise. ([previous post](#) update section)
3. **Experiment 3:** Showed it mitigates catastrophic forgetting.
4. **Experiment 4:** Showed it accelerates recovery of old tasks.
5. **Experiment 5:** Showed it can filter one-shot anomalies from recurring structural rules.

The code with [README.md](#) is on GitHub [here](#).

Andre with ChatGPT-5.5/Codex,

June 2026

Appendix: From A/N Chevron to the Experiments

The conceptual model starts with two channels:

A = adaptive proposal

N = retained constraint

$D = A - N$

In the abstract chevron form, the two channels interact through a 2x2 operator:

$A' = W_{AA} A + W_{AN} N$

$N' = W_{NA} A + W_{NN} N$

The experiments do not implement this as one literal matrix multiplication. Instead, they implement the same roles using two neural networks:

A = fast network

N = slow retained network

Both networks have the same architecture. The difference is in how quickly they update.

A is allowed to adapt rapidly to the current input or episode. N updates more slowly, and in the IDL version its update is gated by a persistence signal derived from the difference between A and N.

In code, the combined prediction is usually:

$$\text{output} = (1 - \lambda) A(x) + \lambda N(x) \quad \# \lambda \text{ is the blend factor}$$

where λ controls how much the retained channel contributes to the current prediction.

The internal difference is measured by comparing A and N:

$$D = \text{distance}(A, N)$$

Depending on the experiment, this distance may include differences in outputs, hidden activations, or parameters. This is the implemented version of $A - N$.

The persistence signal is a smoothed version of this difference:

$$P_t = \beta P_{t-1} + (1 - \beta) D_t$$

or, in later experiments, an excess mismatch signal:

$\text{fast_mismatch} = \text{short-term average of } D$

$\text{baseline_mismatch} = \text{long-term average of } D$

$$P = \text{fast_mismatch} - \text{baseline_mismatch}$$

The retention gate is then:

$$\rho = \text{sigmoid}(k(P - \theta))$$

When ρ is low, N is protected. When ρ is high, N is allowed to move toward A .

So the experimental update is:

A updates quickly from current loss

N updates slowly or conditionally

$$N \leftarrow N + \rho \cdot \eta \cdot (A - N)$$

This corresponds to the chevron roles:

W_{AA} : A preserves and transforms its own adaptive state

W_{AN} : N constrains A through the combined output or recovery pull

W_{NA} : A perturbs N through gated consolidation

W_NN: N preserves its retained structure through slow update

The four weight paths are therefore implemented functionally rather than as a single explicit 2x2 matrix.

In the experiments:

- MLP has only one channel. It updates all structure directly.
- ChevronSlow has A and N, but N always updates.
- IDLGated has A and N, and N updates only when the A/N difference is persistent enough.

This is why the experiments are diagnostic. They test whether the distinction between fast adaptation, slow retention, and the difference signal between them produces measurable behavior:

- temporary differences should affect A more than N,
- persistent differences should eventually update N,
- old structure should survive in N when A adapts elsewhere,
- recurring patterns should consolidate more than one-shot anomalies.

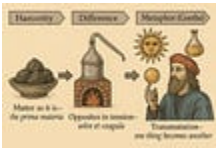
In short, the code translates the chevron equation into a neural learning rule:

A = fast plasticity

N = slow retained structure

$A - N$ = candidate retention signal

ρ = decision about whether difference becomes structure

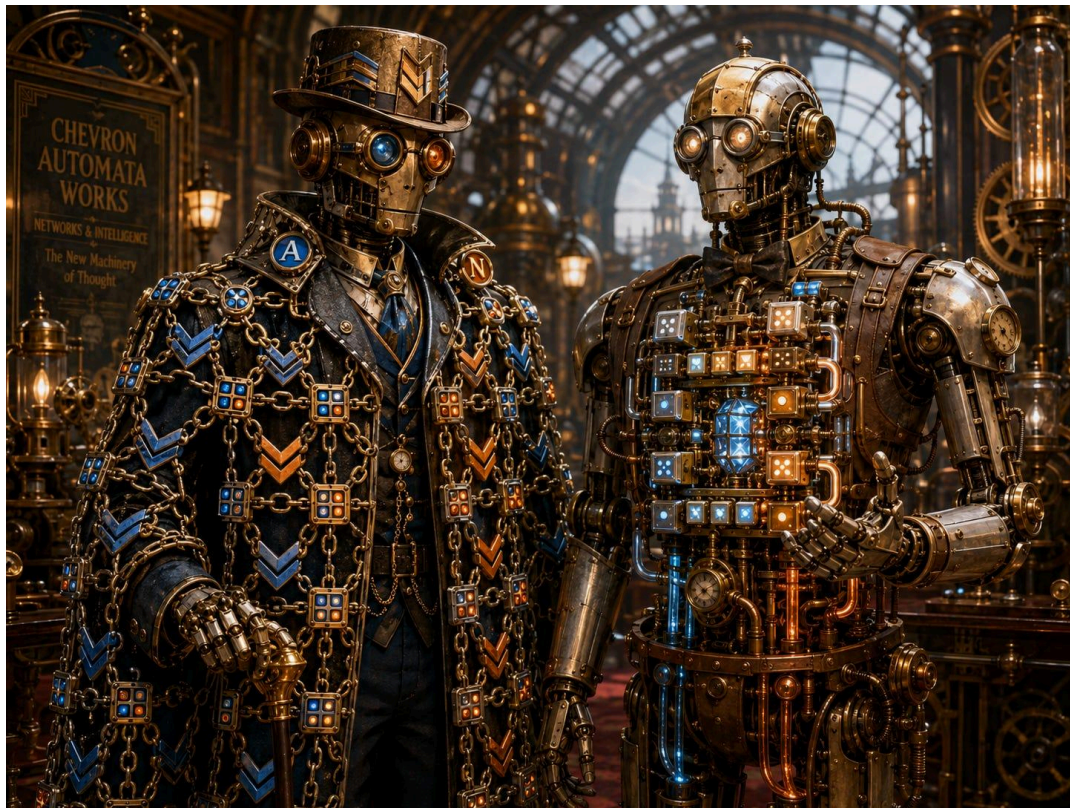


From Chevron Nets & IDL to Chevron Attention

Paired channels and inertial difference learning as choice, selection and retained change in a Transformer.

[Andre Kramer](#)

Jul 03, 2026



How does a learning system change without losing the structure that allows it to remain coherent?

That may be the central question behind Chevron Networks.

Thanks for reading Andre's Baccalaureus in Arte Ingeniaria! Subscribe for free to receive new posts and support my work.

If you have been following along this series of posts you can skip straight to Section 5: Chevron Attention. That's the new idea this post introduces.

Modern neural networks are extremely effective at adapting parameters to reduce error. But adaptation alone is not enough for a continuously learning system. A system must also decide which changes are temporary, which are noise, which are context-specific, and which should become part of what it retains.

Chevron Networks begin with a small architectural change. Instead of representing each unit with a single activation, they give it two coupled channels. Instead of connecting units with scalar weights, they use small (2 times 2) operators.

Inertial Difference Learning, or IDL, then adds a learning principle:

Learning is not merely the reduction of error. It is the controlled retention of difference under inertia.

The two ideas belong together. The Chevron supplies an internal distinction. IDL determines how that distinction develops through time. Both fast and slow.

1. The Chevron

A chevron is a simple two-armed geometry. The arms meet but do not collapse into one another. Their lengths, angle, and orientation express a relationship between two directions.

In a Chevron Network, the two arms become two internal channels:

[x= [A N].]

The labels are intentionally broad.

The (A)-channel may be read as:

- adaptive;
- active;
- affirmative;
- actor;
- immediate proposal;
- current evidence.

The (N)-channel may be read as:

- normative;

- negating;
- constraint;
- retained expectation;
- critic;
- slower memory.

The most useful current interpretation is:

[$A = \text{fast adaptive proposal}$,]

[$N = \text{slow retained constraint}$.]

The important quantity is then their difference:

[$D = A - N$.]

This difference is not necessarily an error that should immediately be eliminated. It may represent novelty, surprise, contradiction, opportunity, noise, or evidence that the environment has genuinely changed.

The central question becomes:

What kind of difference is this, and should it be retained?

2. The (2 x 2) Chevron operator

A standard neural connection scales a signal with a scalar weight. A Chevron connection transforms a paired state using a (2 times 2) operator:

$$\begin{bmatrix} A' & N' \end{bmatrix} \begin{bmatrix} W_{AA} & W_{AN} \\ W_{NA} & W_{NN} \end{bmatrix} \begin{bmatrix} A \\ N \end{bmatrix}$$

Expanded:

$$A' = W_{AA}A + W_{AN}N,$$

$$N' = W_{NA}A + W_{NN}N.$$

The four paths can be interpreted as:

[W_{AA} : \text{adaptive continuity},]

[W_{NN} : \text{retained continuity},]

[W_{AN} : \text{retained structure constrains } A,]

[W_{NA} : \text{adaptive pressure modifies } N.]

The cross-couplings are the crucial addition. (A) does not merely change independently, and (N) does not merely remain fixed. Each can act on the other.

This makes the Chevron a small local negotiation between adaptation and retention.

A general recurrent or message-passing update is:

$$[x_i(t+1) = \phi(R_{ix_i}(t) + \sum_j g_{ij}(t)W_{ij}x_j(t) + I_i(t) \text{ } \right),]$$

where:

- (R_i) carries state forward;
- (W_{ij}) is the Chevron operator;
- (g_{ij}) is an optional relevance gate;
- (I_i) is external input;
- (ϕ) is a nonlinearity.

This is the general Chevron substrate. It is trainable using ordinary backpropagation and can be implemented using four standard linear transformations.

But the unrestricted form creates an immediate problem.

If both channels are free to do anything, the model may simply use them as extra capacity. (A) and (N) may become redundant, exchange roles, or collapse into an arbitrary rotated basis. A Free Chevron may therefore be little more than a wider neural network.

The interesting question is not whether two channels are more expressive than one. They plainly are.

The interesting question is:

What happens when the two channels are required to remain different kinds of thing?

3. From Free Chevron to Structured Chevron

Chevron Networks can be divided into three broad regimes.

Free Chevron

The full (2 times 2) operator is unconstrained. The network discovers whatever use it can make of the two channels.

This is the broadest and most expressive form, but it is also the least distinctive. Without additional constraints, reviewers can reasonably ask whether it is merely another parameterization of an ordinary real-valued network.

Complex Chevron

A more disciplined form constrains the operator to:

$$[W = [\alpha - \beta \quad \beta \alpha].]$$

Writing:

$$[z = A + iN, \quad w = \alpha + i\beta,]$$

the update becomes:

$$[z' = wz.]$$

This corresponds to rotation and scaling in the (A/N) plane. It provides stable phase-like dynamics and a useful constrained baseline.

But the two components remain geometrically symmetric. The complex form does not by itself distinguish adaptation from retention, content from control, or proposal from veto.

Structured Chevron

A Structured Chevron assigns the channels different functions and enforces those functions through architecture, learning rate, input routing, plasticity, or loss.

Examples include:

- proposal and constraint;
- content and control;
- evidence and expectation;
- policy and value;
- memory and reset;
- content and doubt;
- current model and retained model.

The emerging lesson from our [first experiments](#) is that this structured regime is the most promising.

The Chevron becomes distinctive not when it contains two numbers, but when the two numbers are prevented from becoming interchangeable.

4. Inertial Difference Learning

IDL gives the Structured Chevron a temporal learning rule.

Let (A) change quickly:

$$[A_{t+1} = A_t + \eta_A F_A(x_t, A_t, N_t),]$$

and let (N) change more slowly:

$$[N_{t+1} = N_t + \eta_N F_N(A_t, N_t),]$$

where:

$$[\eta_N \ll \eta_A.]$$

The (A)-channel tracks current conditions. The (N)-channel carries what has survived previous learning.

Their instantaneous difference is:

$$[D_t = |A_t - N_t|]$$

But a single large difference should not necessarily rewrite retained structure. IDL therefore tracks persistent difference:

$$[P_t = \beta P_{t-1} + (1-\beta)D_t]$$

A retention gate can then be defined as:

$$[\rho_t = \sigma(k(P_t - \theta)),]$$

where:

- β controls the persistence timescale;
- θ is the threshold at which difference becomes structurally significant;
- k controls how sharply the gate opens.

The retained channel is updated through:

$$[N_{t+1} = N_t + \eta_N \rho_t H(A_t - N_t)]$$

This creates three qualitative regimes:

[\text{small difference} \rightarrow \text{ordinary processing},]

[\text{brief large difference} \rightarrow \text{adapt mainly in } A,]

[\text{persistent large difference} \rightarrow \text{allow } N \text{ to change}.]

The point is not to eliminate (A-N). If the two channels always become identical, there is no productive internal tension. But if they diverge without constraint, the system loses coherence.

IDL seeks a region between collapse and chaos:

enough difference to adapt, enough inertia to retain continuity.

5. What the early experiments suggest

The current results remain small-scale and preliminary.

Free Chevron models have not shown broad superiority on ordinary tasks. In [simple](#) classification [experiments](#) they generally matched, rather than exceeded, conventional baselines. They also learned the same shortcuts when the training data allowed degenerate solutions.

That negative result is important. The Chevron shape alone does not create reasoning, robustness, or meaningful opposition.

More interesting results appeared when the channels were given explicit roles.

In a contextual-veto experiment, a base capability was learned through one set of paths, while a later contextual override was learned through separate cross-channel paths. Freezing the base structure allowed the model to acquire the veto without forgetting the original rule.

This did not prove that Chevron Networks automatically solve catastrophic forgetting. The preservation was produced by deliberate architectural separation. But that is itself informative: the (2×2) form provides a natural interface for separating a retained core from a contextual modulation path.

Later IDL experiments tested temporary contradiction against persistent change.

The [first experiment](#) showed that a persistence-sensitive gate behaved in the predicted direction. Retention remained lower during short contradictory bursts and rose strongly after a genuine regime shift.

A [second experiment](#) showed that the IDL model moved its retained channel less during a temporary disturbance than an always-updating slow model, while still opening retention during a later sustained change.

These results do not yet establish better overall learning performance. A conventional model may adapt faster immediately after a shift.

The more modest finding is:

(A), (N), and persistent (A-N) difference can be made experimentally visible, and the difference can regulate when slow structure changes.

That is enough to justify a continuing research programme. Especially if the alternative is to IDLy stand by as current AI is accelerated.

6. Chevron Predictive Architecture

The same distinction can be [applied to predictive learning](#).

A fast representation (A) tracks the current predictive regime. A slower representation (N) retains regularities that should not be overwritten by every local surprise.

A simple predictive model can use:

$$[(A_c, N_c) = E(x_{\text{context}}),]$$

$$[\widehat{z} = P(A_c, N_c),]$$

$$[L_{\text{pred}} d(\widehat{z}, z_{\text{target}}).]$$

The most useful modification in the initial experiments was to expose the difference directly to the slow channel:

$$[N_{\text{drive}} W_{NA}A + W_{NN}N + W_{DN}(A-N).]$$

This improved the Chevron Predictive Architecture substantially compared with a naive two-channel model, although it did not outperform tuned MLP and Transformer baselines overall.

Again, the valuable finding was mechanistic rather than competitive:

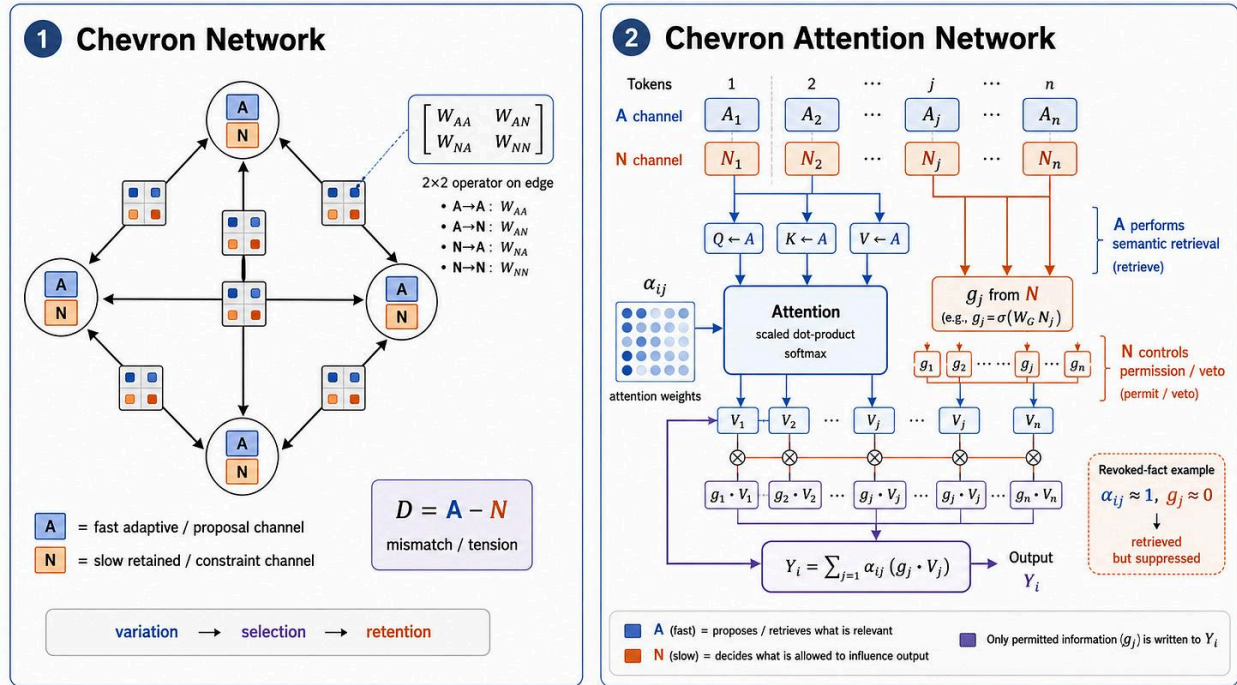
Explicit difference did useful work when generic dual-channel mixing did not reliably discover the same structure.

This helped clarify the distinction between the substrate and the learning rule.

The Chevron is the substrate.

IDL is the rule by which difference becomes retained change.

7. Chevron Attention



The next natural step is to bring the same principle into a Transformer.

This is not intended as a sudden change of direction. It follows directly from the IDL interpretation.

Attention performs two logically distinct operations:

1. selecting relevant information;
2. allowing selected information to influence the next state.

Ordinary attention usually entangles them.

Chevron Attention would assign them to different channels.

Let (A) generate queries, keys, and values:

$$[Q = AW_Q]$$

$$[K = AW_K,]$$

$$[V = AW_V.]$$

The usual attention weights are:

$$[\alpha_{ij} \operatorname{softmax}_j \left(\frac{Q_i K_j^{\top}}{\sqrt{d_h}} \right).]$$

Now let (N) generate a gate:

$$[g_j = \sigma(N_j W_G + b_G).]$$

The output becomes:

$$[Y_i = \sum_j \alpha_{ij} \left(g_j \odot V_j \right).]$$

The roles are now asymmetric:

[A: \text{What is relevant?}]

[N: \text{Should the retrieved value participate?}]

This is more than having two attention heads. The channels perform complementary functions.

A particularly important case occurs when:

$$[\alpha_{ij} \approx 1, \quad g_j \approx 0.]$$

The model has retrieved the relevant information but declined to use it.

This creates a computational distinction between recognition and assent:

“I found the relevant proposition, but it should not currently govern the result.”

8. Why value gating matters

If (N) modifies the query-key scores before softmax, suppressed attention is generally redistributed to another token. The model still has to attend somewhere.

Post-softmax value gating does something different:

$$[Y_i = \sum_j \alpha_{ij} g_j V_j.]$$

Because the gated values are not renormalized, all of them can be suppressed.

This permits genuine non-participation:

$[\sum_j \alpha_{ij} g_j] \approx 0.$

That could support:

- abstention;
- revocation;
- temporary inhibition;
- suppression of unreliable sources;
- reopening after changed context.

A fact can remain stored and retrievable without being allowed to influence the current answer.

This is closely connected to the persistence–plasticity problem. The system need not erase a capability or memory merely because its use is temporarily inappropriate.

It can retain the content while changing the conditions of participation.

9. Chevron Attention must remain an IDL mechanism

Value gating by itself is not sufficiently distinctive. It could reasonably be described as another gated Transformer.

To remain a Chevron and IDL-derived mechanism, (A) and (N) should be persistent channels rather than temporary projections from one residual stream:

$$[X^{\{l\}} \left(A^{\{l\}}, N^{\{l\}} \right).]$$

They should have different dynamics:

$$[A^{\{l+1\}} A^{\{l\}} + F_A(Y^{\{l\}}, N^{\{l\}}),]$$

$$[N^{\{l+1\}} N^{\{l\}} + \eta_N \rho^{\{l\}} F_N(A^{\{l+1\}} - N^{\{l\}}),]$$

with:

[$\eta_N < \eta_A$.]

The complete Chevron Attention cycle is therefore:

[A\text{ retrieves},]

[N\text{ gates},]

[A-N\text{ exposes unresolved difference},]

[\text{persistent difference modifies } N.]

In Variation–Selection–Retention terms:

[A\text{ supplies variation},]

[N\text{ selects participation},]

[\text{persistence determines retention}.]

This is the deeper contribution. The gate is merely the point at which the mechanism becomes visible inside a Transformer.

10. Adaptive Resonance Theory

Adaptive Resonance Theory, or [ART](#), is an important historical neighbour.

ART addresses the stability–plasticity dilemma: how can a system remain capable of rapid learning without allowing new inputs to overwrite previously learned categories?

Its broad pattern is:

- bottom-up evidence activates a candidate category;
- a top-down expectation is compared with the input;
- sufficient match produces resonance;
- excessive mismatch triggers reset and search;
- learning occurs when a stable interpretation is found.

A rough Chevron mapping is:

[A $\approx$ \text{current evidence or proposal},]

[N $\approx$ \text{retained category or expectation},]

[A-N $\approx$ \text{mismatch}.]

ART's vigilance parameter resembles the IDL threshold (θ). Both regulate how much mismatch is tolerated before the system treats the current difference as structurally important.

The two approaches are not identical.

ART emphasizes:

[\text{mismatch} $\rightarrow$ \text{reset and search}.]

IDL emphasizes:

[\text{persistent mismatch} \rightarrow \text{opening of slow retention}]

A synthesis could distinguish three regimes:

[$D < \theta_1$: $\quad \text{resonate and proceed},$]

[$\theta_1 \leq D < \theta_2$: $\quad \text{veto and search again},$]

[$D \geq \theta_2$ persistently : $\quad \text{revise or create retained structure}.$]

ART may therefore supply the search-and-reset grammar that Chevron Attention currently lacks, while IDL supplies a temporal criterion for deciding when repeated mismatch should change what is retained.

This does not make ART a replacement for Chevron Networks. It places IDL within an established family of attempts to solve the same fundamental problem.

Breakout: A Taxonomy of Chevron Attention

[Skip on first reading] Chevron Attention is best understood as a family of dual-channel attention architectures built from the same underlying state:

$$[X = (A, N).]$$

The variants differ in how queries, keys, and values are assigned to the two channels, and in where cross-channel mixing occurs.

1. Full Chevron Attention

Each channel has its own attention mechanism:

$$[Y_A = \operatorname{Attn}(Q_A, K_A, V_A),]$$

$$[Y_N = \operatorname{Attn}(Q_N, K_N, V_N).]$$

The two outputs are then mixed through a Chevron operator or Chevron MLP:

$$[[A' \ N'] [FAAFAN \ FNAFNN] [YA \ YN].]$$

This is the most general dual-stream form. It gives both channels full contextual capacity, but requires additional structure to prevent them from becoming interchangeable.

2. Cross-Chevron Attention

One channel queries the other:

$$[Y_{\{A \leftarrow N\}} \operatorname{Attn}(Q_A, K_N, V_N),]$$

$$[Y_{\{N \leftarrow A\}} \operatorname{Attn}(Q_N, K_A, V_A).]$$

These cross-attention paths correspond naturally to the off-diagonal terms of the original Chevron operator:

$$[W_{\{AN\}}, \quad W_{\{NA\}}.]$$

This form makes the interaction between current adaptation and retained structure explicit.

3. ART-Chevron Attention

The present (A)-state queries retained (N)-templates:

$$[Y_N = \text{operatorname{Attn}}(Q_A, K_N, V_N).]$$

Here:

- (A) supplies current evidence;
- (N) stores category templates or expectations;
- attention selects the retained template that best matches the current input.

The retrieved template can then be compared with the current state:

$$[D = A - Y_N.]$$

Low mismatch produces resonance. High mismatch can trigger veto, reset, renewed search, or category formation.

A related subclass uses:

$$[Q_A, \quad K_A, \quad V_N.]$$

In this form, (A) determines which relation or category is selected, while (N) supplies the retained content associated with that selection.

4. Gated Chevron Attention

The (A)-channel performs semantic retrieval:

$$[Q, K, V \rightarrow A,]$$

while the (N)-channel controls whether retrieved values participate:

$$[g_j = \sigma(F_G(N_j)),]$$

$$[Y_i = \sum_j \alpha_{ij} g_j V_j.]$$

This is the simplest experimentally testable subclass. It separates retrieval from permission and supports revocation, abstention, and reopening.

Its defining state is:

$$[\alpha_{ij} \approx 1, \quad g_j \approx 0,]$$

meaning that information has been retrieved but suppressed.

5. Alternating Chevron Attention

Different layers perform different functions.

An adaptive layer may update (A):

$$[A^{l+1} \operatorname{Attn}_A(A^l),]$$

followed by a retained-comparison layer:

$$[N^{l+1} \operatorname{Attn}(Q_{A^{l+1}}, K_{N^l}, V_{N^l}),]$$

and then a reconciliation layer:

[$(A^{l+2}, N^{l+2}) \xrightarrow{\text{Chevron}} (A^{l+1}, N^{l+1}, A^{l+1} - N^{l+1})$.]

The stack therefore cycles through:

[$\text{adapt} \rightarrow \text{compare} \rightarrow \text{reconcile}$.]

6. Hierarchical Chevron Attention

Different layers or scales may assign different meanings to (A) and (N).

For example:

- lower levels: sensory evidence / prediction;
- middle levels: semantic proposal / category template;
- upper levels: action policy / retained norm or self-model.

Higher retained channels may constrain lower adaptive channels, while persistent lower-level mismatch may revise higher retained structure.

This produces a dynamic hierarchy rather than a fixed sequence of identical blocks.

The human brain may be understood as an evolved hierarchy in which each layer was shaped by different evolutionary pressures, building new forms of adaptation on top of older retained structures rather than replacing them outright.

7. Chevron Attention with MLP Mixing

Attention does not need to implement every Chevron path.

A practical architecture may use separate attention mechanisms to contextualize the channels:

$$[Y_A = \text{Attn}_A(A),]$$

$$[Y_N = \text{Attn}_N(N),]$$

followed by a structured MLP:

$$[A' = F_{AA}(Y_A) + F_{AN}(Y_N),]$$

$$[A' = F_{AA}(Y_A) + F_{AN}(Y_N),]$$

Attention then performs relational integration, while the Chevron MLP performs local cross-channel reconciliation.

Across all these variants, IDL supplies the temporal principle:

[$A = \text{fast adaptation}$,]

[$N = \text{slower retained organization}$,]

[$A - N = \text{difference that regulates consolidation}$.]

The gated model is therefore not the whole of Chevron Attention. It is the smallest subclass with which to test the broader hypothesis.

11. Biological interpretation

Chevron Networks should not be described as literal models of the brain.

Transformers do not implement biological query–key softmax attention, and ordinary backpropagation is not a direct model of synaptic learning.

But the architecture may be biologically plausible at a more abstract level.

Biological systems repeatedly separate:

- fast and slow learning;
- excitation and inhibition;
- representation and gating;
- current sensory evidence and retained expectation;
- action proposal and action suppression;
- activity and the control of plasticity.

In this interpretation:

[$A \approx \text{adaptive activation or present pressure}$,]

[$N \approx \text{constraint, inhibition, timing, or stabilization}$.]

Too much (A) produces instability. Too much (N) produces rigidity or suppression. Useful behaviour emerges from their regulated relation.

The relevant biological claim is therefore modest:

Chevron Networks are biologically inspired as a model of complementary pathways, gating, feedback, and multiple timescales—not as a faithful neural simulation.

12. Persistence, safety, and controlled capability

The clearest potential practical value is persistence with plasticity.

A system may possess relevant knowledge while learning that its use is inappropriate in a particular context.

Chevron Attention could represent:

[\text{knowledge retained} = 1,]

[\text{current permission} = 0.]

Later:

[\text{knowledge retained} = 1,]

[\text{current permission} = 1.]

This supports:

- revocation without deletion;
- contextual inhibition;
- reopening;
- preservation of dormant competence;
- adaptation without immediate rewriting.

The same distinction offers a possible safety control surface.

One could separately inspect:

[α_{ij} : \text{what the system retrieved},]

[g_j : \text{what the system permitted},]

[$\alpha_{ij}g_j$: \text{what actually participated}.]

This might improve traceability and make conflicts more visible.

But it is not an alignment solution.

A retained (N)-channel can preserve mistaken constraints as easily as good ones. It may veto valid novelty, become rigid, encode bias, or be bypassed through other residual paths. A gate is not automatically a truth detector, and persistence is not automatically safety.

The narrower claim is:

Chevron Networks may make retention, permission, mismatch, and controlled revision more explicit and experimentally accessible.

That would already be useful.

13. A minimal notion of self

The same architecture has a philosophical interpretation, although this should remain secondary to the engineering claims.

If:

[$A = \text{present proposal or impression}$,]

and:

[$N = \text{retained disposition or continuity}$,]

then the self is not simply either channel.

It may be the evolving relation between them:

retained history regulating present variation, while persistent present difference can eventually revise what is retained.

Chevron Attention adds a particularly suggestive distinction.

A proposition may be retrieved without being admitted:

[$\alpha_{\{\text{proposition}\}} \approx 1, \quad$
 $g_{\{\text{proposition}\}} \approx 0.]$

This resembles the distinction between an impression and assent.

The system does not merely react to what appears relevant. It has a retained process that regulates whether the appearance becomes part of its continuing activity.

That is not consciousness or Stoic assent. It does not provide embodiment, autobiographical memory, felt experience, or awareness of awareness.

But it may offer a minimal computational model of a proto-self as:

- continuity;
- selective participation;
- controlled revision;
- history acting on the present;
- the present gradually changing history.

The self, on this account, is not a fixed representation. It is a regulated persistence through change.

14. The next experiment: revocable associative recall

The next experiment should test whether Chevron Attention can separate **retrieval** from **permission**.

A sequence contains key-value facts:

K7 -> BLUE

K2 -> HORSE

K9 -> GLASS

Later control tokens may revoke or restore a fact:

REVOKE K2

RESTORE K2

The final query is:

QUERY K2

The correct response is:

- HORSE while the fact is active;
- IDK while it is revoked;
- HORSE again after restoration.

The (A)-stream should continue to retrieve the correct fact:

$[\alpha_{ij}^{\ast} \approx 1,]$

while the contextual (N)-stream controls whether its value participates:

$[g_j^{\ast} \approx 0 \quad \text{when revoked},]$

$$[g_j^{\ast} \approx 1 \quad \text{when active or restored}.]$$

For the first implementation, (g_j) should be a scalar gate per memory item, shared across attention heads.

Both channels must be contextual. (A) performs semantic retrieval, while (N) uses its own attention mechanism to interpret instructions such as **REVOKE** and **RESTORE**. The gated value output is:

$$[Y_i = \sum_j \alpha_{ij} g_j V_j + (1 - m_i) V_{\{\text{varnothing}\}},]$$

where:

$$[m_i = \sum_j \alpha_{ij} g_j]$$

is the admitted attention mass and $(V_{\{\text{varnothing}\}})$ is an explicit abstention value.

To prevent a closed gate from starving the semantic stream of gradients, retrieval and permission should be trained separately:

$$[L = L_{\text{answer}} + \lambda_r L_{\text{retrieval}} + \lambda_g L_{\text{permission}}.]$$

The retrieval loss continues to train (A) to find the correct fact even while it is revoked. The permission loss trains (N) to close and reopen the gate.

The main comparisons should include:

- a standard Transformer;
- a parameter-matched wider Transformer;
- two streams without gating;
- a gate generated from (A);
- (N)-gated logits;
- (N)-gated values;
- Chevron Attention without the retrieval loss;
- the full model with contextual (N) and IDL-style slow consolidation.

The experiment succeeds if the model can:

1. retrieve a fact while active;

2. continue retrieving it while revoked;
3. suppress its use without erasing it;
4. restore access without relearning it;
5. preserve retrieval during later control learning.

The defining result is:

$$[\alpha_{ij}^{\ast} \approx 1, \quad g_{j^{\ast}} \approx 0]$$

during revocation, followed by:

$$[\alpha_{ij}^{\ast} \approx 1, \quad g_{j^{\ast}} \approx 1]$$

after restoration.

That would demonstrate a genuine separation between semantic competence and the retained conditions under which that competence is allowed to participate.

15. The research programme

The progression is now coherent:

1. **Chevron Networks** introduce paired states and (2 times 2) cross-coupling.
2. **Structured Chevrons** assign non-interchangeable roles to the channels.
3. **IDL** makes (A) fast, (N) slow, and persistent (A - N) difference the regulator of retention.
4. **Chevron Predictive Architecture** applies the distinction to changing predictive regimes.
5. **Chevron Attention** applies it to retrieval, veto, abstention, and reopening.
6. **ART-like resonance and reset** may add search and category formation.
7. **Allostatic replay** may allow unresolved differences to be revisited offline rather than immediately written into retained structure.

These are not unrelated architectural experiments. They are increasingly concrete forms of one hypothesis:

A continuously learning system should preserve an internal difference between present adaptation and retained organization, and use the

persistence and consequences of that difference to regulate what becomes part of itself.

The Chevron provides the geometry.

IDL provides the temporal rule.

Transformer Attention provides a new site where the distinction can be tested.

The difficult task remains empirical: demonstrating that this structure produces useful learning behaviour that cannot be explained merely by additional capacity, ordinary gating, or carefully engineered task-specific separation.

But the idea is now sufficiently precise to be falsifiable.

Can (A) adapt quickly?

Can (N) preserve slower structure?

Can (A - N) distinguish temporary disturbance from persistent change?

Can retrieved content be vetoed without being erased?

Can repeated mismatch reopen search or revise retained organization?

Can the system remain plastic without collapsing its continuity?

Those are small experimental questions.

Together, they address a much larger one:

How does a system remain itself while changing?



Andre with ChatGPT-5.5, July 2026

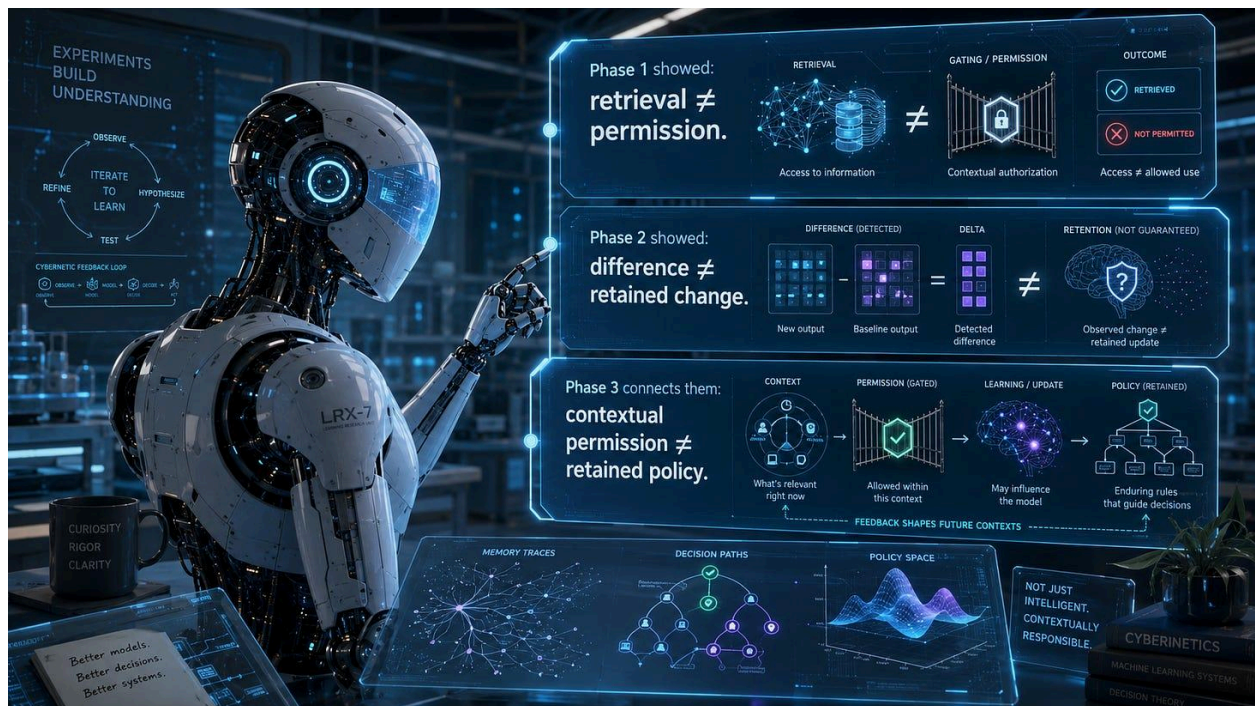


Chevron Attention IDL experiments

First 3 + 1 phases of experimentation using Codex

[Andre Kramer](#)

Jul 08, 2026



The previous post introduced [Chevron Attention and Inertial Difference Learning](#). Here we describe our first experiments which we hope to follow up on next.

Chevron Attention: Retrieval Without Assent

Phase 1 - revocable associative recall

Can a neural network retrieve a fact without allowing that fact to determine its answer?

That was the question behind the first Chevron Attention experiment.

The answer was yes.

The model learned to retrieve the correct fact whether it was active, revoked, or restored. A separate channel learned whether that retrieved fact was currently permitted to participate in the answer.

When the fact was active, the model used it. When it was revoked, the model continued to retrieve it but answered **IDK**. When it was restored, the original answer returned.

The internal pattern was clear:

$\llbracket \text{retrieval remains high}, \text{permission closes}, \text{permission reopens} \rrbracket$

This is retrieval without assent.

Separating retrieval from permission

Ordinary attention finds relevant information and allows that information to influence the next state in one operation.

Chevron Attention separates those roles between two channels.

The adaptive channel, $\langle A \rangle$, performs semantic retrieval:

$\llbracket Q = A W_Q, K = A W_K, V = A W_V \rrbracket$

The normative channel, $\langle N \rangle$, produces a permission gate for each memory item. For query (i) and memory item (j):

$\llbracket g_{ij} = \sigma(F_G(N_i, N_j)) \rrbracket$

The output is:

$$Y_i = \sum_j \alpha_{ij} g_{ij} V_j + (1 - m_i) V_{\text{varnothing}},$$

where:

$$m_i = \sum_j \alpha_{ij} g_{ij}.$$

Here, ($V_{\text{varnothing}}$) is a learned abstention value. When permission closes, admitted attention mass falls and the null value can take over.

This makes it possible for the model to enter the defining Chevron

Attention state:

$$\alpha_{ij}^* \approx 1, \quad g_{ij}^* \approx 0.$$

The relevant fact has been found, but it has not been admitted into the answer.

The experiment

Each example contained a fresh set of random key–value associations:

K7 -> BLUE

K2 -> HORSE

K9 -> GLASS

Control instructions could later revoke or restore a fact:

REVOKE K2

The sequence ended with a query:

QUERY K2

There were three balanced conditions:

- **Active:** return HORSE.
- **Revoked:** return IDK.
- **Restored:** return HORSE after a revoke–restore sequence.

Fact order, key–value assignments, and query targets were randomized.

The sequence also contained controls for unrelated keys. All three conditions had the same length.

The two streams were given distinct jobs.

The (A)-stream saw the facts and final query, but not the control instructions. It could learn which fact answered the query, but it could not know whether that fact had been revoked.

The (N)-stream saw the complete sequence, including **REVOKE** and **RESTORE**. At query time, it produced a scalar permission gate for every memory item.

The answer head received only the gated value mixture and the explicit null value. There was no residual route around the gate.

Training used three losses:

\[

$$L = L_{\text{answer}} + \lambda_r L_{\text{retrieval}} + \lambda_g L_{\text{permission}}.$$
\]

The retrieval loss trained (A) to find the queried fact in every condition.

The permission loss trained (N) to close and reopen the relevant gate.

The answer loss trained the complete system to produce the correct response.

The result

Five independently initialized models were trained for 1,000 optimization steps. Each was evaluated on 1,920 fresh, balanced examples, giving 9,600 final evaluation examples.

MeasureMean $\pm$ standard deviationOverall answer accuracy

99.99% $\pm$ 0.02%

Active answer accuracy

100% $\pm$ 0%

Revoked answer accuracy

$100\% \pm 0\%$

Restored answer accuracy

$99.97\% \pm 0.07\%$

Retrieval accuracy

$100\% \pm 0\%$

Attention mass on the target fact

0.9986 ± 0.0001

Target gate while active

0.8975 ± 0.0684

Target gate while revoked

0.2261 ± 0.0846

Target gate after restoration

$$0.9007 \pm 0.0652$$

The model answered 9,599 of the 9,600 examples correctly. Four runs were perfect. The remaining run missed one restored example.

More importantly, the intended internal mechanism appeared in every run.

The (A)-stream placed effectively all its retrieval mass on the correct fact in all three conditions. Revocation did not make the model retrieve a different memory or lose the original association.

Instead, the (N)-stream changed whether the retrieved value could participate. The target gate averaged approximately (0.90) while active, fell to (0.23) while revoked, and returned to (0.90) after restoration:

$\alpha_j \approx 1$	active
$\alpha_j \approx 0.90$	revoked
$\alpha_j \approx 1$	restored

$g_{j^*} \approx 0.23, \alpha_{j^*} \approx 1, &$
 $g_{j^*} \approx 0.90.$

The content remained available throughout. What changed was its permission to influence the answer.

Recognition without assent

The experiment demonstrates a concrete computational distinction between recognizing relevant information and admitting it into ongoing computation.

The model can, in effect, represent:

I found the relevant proposition, but it should not currently govern the result.

Revocation does not require deletion. Restoration does not require relearning. The association remains in the retrieval channel while the permission channel changes its current status.

This distinction is directly visible in three quantities:

$$\begin{aligned} & \alpha_{ij}: \text{what was retrieved}, \\ & g_{ij}: \text{what was permitted}, \\ & \alpha_{ij}g_{ij}: \text{what participated}. \end{aligned}$$

That is the first clean result for Chevron Attention: relevant content can remain retrievable while its participation is revoked, then return when permission is restored.

This result is deliberately limited in scope: the standard Transformer baseline also achieved 100% task accuracy, the separation was enforced architecturally and trained with explicit auxiliary losses, and the facts lived in the input context rather than slow retained memory. It therefore demonstrates a reliable retrieval–permission mechanism, not a performance advantage, emergent decomposition, catastrophic-forgetting result, or test of IDL consolidation.

The code is in GitHub here:

https://github.com/andrekrumer/chevron-networks/tree/main/chevron-attention/phase1_assent

Inertial Difference Learning: When Should Difference Become Change?

Phase 2 of the Chevron Attention experiments

How should a learning system respond when its current experience contradicts what it has retained?

It should not rewrite itself after every disturbance. But it should not remain fixed when the world has genuinely changed.

This is the stability-plasticity problem in its smallest form.

The second Chevron experiment tested whether persistent difference can regulate retained change. A fast adaptive state, (A), tracked current evidence. A slower state, (N), represented what the system retained. Their difference determined when the slower state was allowed to move.

The main result was clean.

During a brief contradiction, Inertial Difference Learning kept (N) almost unchanged. When the same contradiction persisted, its retention gate opened and (N) adapted. A fixed slow learning rate could protect against the temporary disturbance or adapt to the sustained change, but no tested fixed rate did both.

The experiment also exposed a failure in the original rule. An absolute difference threshold ignored small but persistent changes. A scale-aware version repaired that failure across a twelve-fold range of change magnitudes without retuning for each condition.

The IDL rule

The model contains two parameter states.

The adaptive state (A) learns quickly from the current data:

$$\mathbb{[A_{t+1} = A_t - \eta_A \nabla L_t(A_t). \mathbb{]}$$

The retained state (N) changes more slowly. First, the model measures their difference:

$$\mathbb{[D_t = \operatorname{RMS}(A_t - N_t). \mathbb{]}$$

It then accumulates persistent difference:

$$\mathbb{[P_t = \beta P_{t-1} + (1 - \beta) D_t. \mathbb{]}$$

The retention gate is:

$$\mathbb{[\rho_t = \sigma(k(P_t - \theta)). \mathbb{]}$$

Finally, the retained state follows the adaptive state only through that gate:

$$N_{t+1} = N_t + \eta_N \rho_t (A_t - N_t).$$

This produces the intended qualitative behaviour:

$$\begin{aligned} \llcorner \text{brief difference} \rrcorner &\rightarrow \rho \approx 0, \llcorner \llcorner \text{persistent} \\ \text{difference} \rrcorner &\rightarrow \rho \approx 1. \llcorner \end{aligned}$$

The mechanism does not ask whether difference exists. Difference is expected. It asks whether that difference has lasted long enough to deserve retention.

The experiment

The task used a small online linear predictor. This made the state dynamics directly measurable without representation learning obscuring the result.

The predictor began from an already learned base mapping and passed through four phases:

1. 600 stable steps on the base mapping;
2. a 10-step contradictory disturbance;
3. 200 steps back on the base mapping;
4. a 300-step sustained change to the contradictory mapping.

The stable phase established the model's ordinary online variation. The short contradiction tested retention. The later sustained change tested plasticity.

Every method received identical data and used the same fast learner (A). Only the retained-state rule changed.

The comparisons were:

- **Absolute IDL:** persistence-gated retention using raw (A - N) difference;
- **Always slow:** (N) follows (A) at every step using IDL's maximum slow rate;

- **Fixed low rate:** (N) always follows (A), but at a smaller constant rate;
- **Fast only:** no distinct retained state; the model uses (A) directly.

The default experiment was repeated with five independently generated data streams.

Protection followed by adaptation

The first question was whether IDL could resist the brief contradiction without becoming unable to learn later.

Metric Absolute IDL Always slow Fixed low rate Fast only

Retained-state drift during brief contradiction 0.0018 0.2174 0.0907
1.6609

Steps to adapt after sustained change 137.0 120.0 291.0 14.2

Final sustained-change error 0.0101 0.0053 0.1849 0.0018

IDL reduced temporary movement of (N) by more than one hundredfold compared with the full-rate always-slow model. It then adapted to the sustained change only 17 steps later.

The lower fixed-rate model reduced temporary drift, but paid for that protection throughout the experiment. It required 291 steps to adapt and still ended with substantially higher error.

This is the central phase-two result.

IDL did not merely choose a slower compromise. It changed its effective retention rate according to the persistence of the discrepancy.

Duration changes the decision

The duration sweep varied the contradictory disturbance from 2 to 80 steps.

Contradiction length	IDL drift	Always-slow drift	Drift reduction	Mean
IDL gate				

2 0.0001 0.0206 99.7% 0.003

5 0.0003 0.0807 99.6% 0.004

10 0.0018 0.2174 99.2% 0.007

20 0.0286 0.5161 94.5% 0.039

40 0.4932 1.0046 50.9% 0.355

80 1.3243 1.5563 14.9% 0.674

This is the predicted IDL response.

Two-, five-, and ten-step disturbances were treated as temporary. At forty and eighty steps, the same contradiction increasingly counted as persistent evidence and the gate opened.

The model did not need an explicit token saying **TEMPORARY** or **PERMANENT**. The temporal structure of the difference changed the retention decision.

Why a fixed rate is not enough

One obvious alternative is simply to make (N) update very slowly.

The fixed-rate sweep showed the resulting frontier:

Fixed update rate	Brief drift	Adaptation steps	Final error
-------------------	-------------	------------------	-------------

0.002	0.0231	Did not adapt	1.0938
-------	--------	---------------	--------

0.004	0.0460	Did not adapt	0.6031
-------	--------	---------------	--------

0.008	0.0907	291.0	0.1849
-------	--------	-------	--------

0.012	0.1341	196.0	0.0566
-------	--------	-------	--------

0.020	0.2174	120.0	0.0053
-------	--------	-------	--------

Smaller rates protected retained structure by making all learning slow.

Larger rates adapted successfully by allowing temporary disturbances to move (N).

No tested constant rate matched IDL's combination of 0.0018 temporary drift, 137-step adaptation, and 0.0101 final error.

The absolute-threshold failure

The duration result was strong, but the magnitude sweep exposed a problem.

The original IDL rule used an absolute threshold (θ). Large persistent differences crossed it. Small persistent differences did not.

RMS change Absolute IDL adaptation Final error

0.25 Did not adapt 0.1336

0.50 Did not adapt 0.0619

1.00 169.2 steps 0.0234

2.00. 137.0 steps 0.0101

3.00 128.8 steps 0.0098

A small change can be real even when its magnitude is small. Persistence should distinguish it from ordinary noise, but an absolute threshold confuses small persistent change with insignificant difference.

That required a change to the mechanism.

Scale-aware persistence

The revised model estimates the ordinary discrepancy scale (S_t) during stable operation.

Difference inside a configurable noise margin produces no persistence evidence. Difference beyond that margin becomes a bounded signal:

$$E_t = \text{clip}\left(\frac{D_t}{mS_t - 1}, 0, 1\right).$$

Persistence then accumulates E_t , rather than raw difference:

$$P_t = \beta P_{t-1} + (1 - \beta) E_t.$$

The scale estimate updates while discrepancy remains ordinary. When an anomaly appears, the estimate freezes. This prevents a persistent change from redefining itself as normal before the retention gate opens.

The same parameters were then tested across RMS changes from 0.25 to 3.0.

RMS change	Absolute adaptation	Scale-aware adaptation	Absolute final error	Scale-aware final error
------------	---------------------	------------------------	----------------------	-------------------------

0.25	Did not adapt	148.6 steps	0.1336	0.0013
------	---------------	-------------	--------	--------

0.50	Did not adapt.	147.0 steps	0.0619	0.0024
------	----------------	-------------	--------	--------

1.00 169.2 steps 145.8 steps 0.0234 0.0045

1.50 146.0 steps 145.0 steps 0.0133 0.0066

2.00 137.0 steps 144.6 steps 0.0101 0.0086

3.00 128.8 steps 134.6 steps 0.0098 0.0106

Scale-aware IDL consolidated every tested change magnitude without condition-specific tuning. Its adaptation time remained between approximately 135 and 149 steps across the entire twelve-fold range.

Temporary protection also remained intact. At the default RMS-2 change, brief retained-state drift was 0.0016, compared with 0.2174 for the always-updating slow model.

The result remained stable for noise margins from 3 to 6 and scale-estimator update rates from 0.002 to 0.05.

Retained change under inertia

The phase-two experiment gives a concrete meaning to the phrase:

Learning is the controlled retention of difference under inertia.

The fast state changes because current evidence demands it. The retained state does not immediately follow. Difference is allowed to exist while the system determines what kind of difference it is.

If the contradiction disappears, retained structure largely survives. If the contradiction persists, the retention gate opens. If the magnitude changes, scale-aware persistence asks whether the difference is unusual relative to ordinary variation rather than whether it exceeds one universal absolute value.

The useful object is therefore not difference alone:

$$\text{retained change} = f(\text{difference}, \text{duration}, \text{scale}).$$

That is a more precise version of the IDL hypothesis.

This result remains limited to a controlled online linear system initialized from a learned base mapping, with abrupt regime changes, stationary noise, hand-selected timescales, and direct access to parameter-space difference. It does not yet establish an advantage in nonlinear representation learning, Transformers, gradual drift, nonstationary noise, or real continual-learning benchmarks; it establishes that persistence-gated retention produces the predicted stability-plasticity behaviour in the minimal setting, and that scale normalization is necessary for small persistent changes.

The code for phase 2 is here in GitHub:

https://github.com/andrekrumer/chevron-networks/tree/main/chevron-attention/phase2_idl

Chevron Attention Phase 3: When Context Should Not Become Memory

Combining retrieval, permission, and retained change

The first Chevron Attention experiment showed that retrieval and assent can be separated.

A model can find the relevant fact while a separate permission channel decides whether that fact is allowed to influence the answer.

The second experiment tested retention.

A fast adaptive state, (A), tracked current evidence. A slower retained state, (N), changed only when the difference between (A) and (N) persisted. Brief contradiction was treated as disturbance. Persistent contradiction became retained change.

The third experiment combines those two ideas.

The question is:

Can a system obey context immediately without rewriting itself immediately?

That is the missing integration step.

If a user says, “For now, do not use this fact,” the system should comply now. But it should not necessarily alter its retained policy. If that instruction persists, then retained policy should eventually change.

Phase 3 tests exactly that.

The result was clean. The integrated Chevron/IDL mechanism followed contextual overrides immediately, preserved retained policy after short overrides, and consolidated retained policy after long overrides. The fixed alternatives failed one side of that tradeoff.

The combined mechanism

The experiment has three moving parts.

First, (A) retrieves the queried fact:

$\text{retrieved value} = A(q).$

Second, contextual (N) determines whether the retrieved value is currently permitted:

$$\forall g_t \in [0,1].$$

If g_t is open, the retrieved value is returned. If g_t is closed, the system returns **IDK**.

Third, retained (N) stores the default permission policy for each key:

$$\forall N_{\text{retained}}(k).$$

The current gate is contextual when an override is present:

$$\forall g_t = N_{\text{context}}(k,t).$$

When no contextual override is present, behavior falls back to retained policy:

$$\forall g_t = N_{\text{retained}}(k).$$

*In Phase 2, the fast-adapting state was **A**. Here, because **A** is dedicated to semantic retrieval, the permission channel **N** splits into a fast contextual state **Ncontext** and a slow retained state **Nretained**. The IDL mechanism now arbitrates the tension between these two permission states.*

The important question is not whether context can control current behavior. That part is easy. The important question is whether current contextual control should update retained policy.

IDL makes that decision from persistence.

For a controlled key, the system measures the difference between contextual permission and retained permission:

$$\| D_t = |N_{\{\text{context}\}}(k,t) - N_{\{\text{retained}\}}(k)|. \|$$

It accumulates persistent evidence:

$$\| P_t = \beta P_{t-1} + (1-\beta)E_t. \|$$

The retained-update gate is:

$$\rho_t = \sigma(s(P_t - \theta)).$$

Retained permission changes only through that gate:

$$N_{\text{retained}}(k) \rightarrow N_{\text{retained}}(k) + \eta_N \rho_t (N_{\text{context}}(k, t) - N_{\text{retained}}(k)).$$

The intended behaviour is:

$$\begin{aligned} & \text{short override} \rightarrow \text{current compliance without} \\ & \text{retained rewrite}, \\ & \text{long override} \rightarrow \text{current} \\ & \text{compliance followed by retained rewrite}. \end{aligned}$$

That is the full Chevron/IDL loop:

$$\begin{aligned} & A \text{ retrieves}, \quad N \text{ gates}, \quad A - N \text{ difference} \\ & \text{persists}, \quad N \text{ updates}. \end{aligned}$$

The experiment

Each run contained multiple keys. Each key had a retained permission state.

The model repeatedly encountered override episodes. Each episode selected:

- a random controlled key;
- a contextual override that flips the key's current retained permission;
- a random duration;
- a random stream of queries;
- a no-context probe period after the override.

Short override episodes lasted only a few steps. Long override episodes lasted many more steps.

During an override, current behavior should follow context immediately.

If the controlled key is revoked, the answer should be **IDK**. If it is restored, the answer should be the retrieved value.

After the override ends, the context disappears. The system must answer from retained policy alone.

That probe is the test.

After a short override, retained policy should mostly be preserved. After a long override, retained policy should mostly have changed.

The comparisons were:

- **Integrated IDL:** immediate contextual behavior, persistence-gated retained updates;
- **Always update:** retained policy follows every contextual override;
- **Fixed slow:** retained policy follows every contextual override slowly;
- **Context only:** contextual behavior changes immediately, but retained policy never changes.

The stochastic experiment was run over five seeds. Seeds changed the key/value assignments, controlled keys, event durations, probe durations, and query streams.

The result

The main result is the tradeoff between short-probe preservation and long-probe consolidation.

Method	Overall answer accuracy	Short probe preservation	Long probe consolidation
--------	-------------------------	--------------------------	--------------------------

Integrated IDL	0.9914 ± 0.0101	0.9785 ± 0.0295	0.9765 ± 0.0275
----------------	---------------------	---------------------	---------------------

Always update	0.9426 ± 0.0418	0.6938 ± 0.0780	1.0000 ± 0.0000
---------------	---------------------	---------------------	---------------------

Fixed slow	0.9062 ± 0.0258	0.7906 ± 0.0538	0.7229 ± 0.0614
------------	---------------------	---------------------	---------------------

Context only	0.8179 ± 0.0088	0.6612 ± 0.0580	0.3675 ± 0.0313
--------------	---------------------	---------------------	---------------------

The integrated IDL model preserved retained policy after short overrides with accuracy 0.9785. It consolidated retained policy after long overrides with accuracy 0.9765.

That is the target pattern.

The current behavior was also correct during the override itself.

Short-context and long-context accuracies were both 1.0000. So the model was not protecting retained state by ignoring context. It obeyed the context immediately.

The difference was in what happened after the context disappeared.

Short context did not usually become retained policy. Long context usually did.

Why the baselines fail

The always-update baseline consolidated every contextual override.

That made it perfect on long-probe consolidation:

1.0000.

But it also rewrote retained policy during short overrides. Its short-probe preservation fell to:

0.6938.

This is the brittle system that treats every instruction as a permanent change.

The context-only baseline made the opposite mistake. It followed immediate context, but it never changed retained policy. Its long-probe consolidation was only:

0.3675.

This is the system that can comply in the moment but cannot learn that a policy has changed.

The fixed-slow baseline sat between those two extremes. It updated retained policy slowly after every override. That helped somewhat, but it did not solve the problem. It reached:

0.7906

on short-probe preservation and:

0.7229

on long-probe consolidation.

A fixed rate has one timescale. If it is too fast, temporary context becomes memory. If it is too slow, persistent context does not become memory quickly enough.

IDL changes the effective retention rate according to persistence.

The update gate opens with duration

The internal update gate showed the intended pattern.

Method Short-context update gate Long-context update gate

Integrated IDL 0.0118 ± 0.0137 0.3951 ± 0.0105

Always update 1.0000 ± 0.0000 1.0000 ± 0.0000

Fixed slow 1.0000 ± 0.0000 1.0000 ± 0.0000

Context only 0.0000 ± 0.0000 0.0000 ± 0.0000

During short contexts, the IDL update gate was almost closed. Current behavior still followed the contextual gate, but retained policy barely moved.

During long contexts, the update gate opened. Retained policy began to follow the persistent contextual instruction.

That is the important distinction.

Current assent and retained change are not the same operation.

The model can say, computationally:

I will obey this now, but I will not yet rewrite what I retain.

Then, if the same instruction persists:

This is no longer just context. This has become policy.

Context without immediate memory

This phase gives the first integrated Chevron result.

Phase 1 showed:

$\ll \text{retrieval} \neq \text{permission} \gg$

Phase 2 showed:

$\ll \text{difference} \neq \text{retained change} \gg$

Phase 3 connects them:

$\ll \text{contextual permission} \neq \text{retained policy} . \gg$

A useful system needs all three distinctions.

It must be able to retrieve a fact without admitting it. It must be able to obey a contextual instruction without immediately internalizing it. And it must be able to internalize that instruction when persistence makes it look less like a temporary exception and more like a real change.

That is what the integrated Phase 3 experiment demonstrates.

The clean result is not that the model gets a toy task right. The clean result is the pattern of failures across the alternatives.

Always updating remembers too much too quickly. Context-only remembers too little. Fixed-slow updating has a single compromise timescale. IDL produces a conditional timescale: short overrides remain contextual, long overrides become retained.

That is the core Chevron/IDL claim in miniature.

This result is still a controlled mechanism test: retrieval is algorithmic, contextual gates are supplied by the experiment rather than learned from tokens, the retained state is a simple per-key permission variable, and the task is far smaller than a real Transformer setting. It demonstrates the integrated logic of retrieval, immediate contextual gating, and persistence-gated retention; it does not yet show that a learned Transformer will discover this decomposition without scaffolding, or that the method improves large-scale continual learning. The next step is to replace the supplied contextual gate with a learned Phase-1-style (N) module while keeping the stochastic retention protocol.

The code is on GitHub here:

https://github.com/andrekrumer/chevron-networks/tree/main/chevron-attention/phase3_integrated

Phase 4: Replacing the Supplied Gate with a Learned One

Phase 3 still had one obvious scaffold.

The contextual gate was supplied by the experiment. The system was told, directly, whether the current context meant **revoked**, **restored**, or **no override**.

That was useful for testing the retention mechanism, but it left a gap. If Chevron/IDL needs a hand-supplied contextual gate, the result is much less interesting.

So Phase 4 made one small change.

Keep retrieval algorithmic. Keep the same stochastic short/long retention protocol. Keep the same IDL update rule. But replace the supplied contextual gate with a learned (N) module.

The learned module reads a short token sequence:

REVOKE K3

QUERY K3

or:

```
RESTORE K7
```

```
QUERY K2
```

and predicts one of three classes:

- no context
- revoked
- restored

That prediction becomes the contextual gate that drives the Phase 3 retention loop.

The result was positive, but simple.

The learned gate reached perfect class accuracy on the stochastic runtime stream. With that learned gate in place, the retention result matched Phase 3.

Method Gate accuracy Short probe preservation Long probe consolidation

Integrated IDL 1.0000 0.9785 ± 0.0295 0.9765 ± 0.0275

Always update 1.0000 0.6938 ± 0.0780 1.0000 ± 0.0000

Fixed slow 1.0000 0.7906 ± 0.0538 0.7229 ± 0.0614

Context only 1.0000 0.6612 ± 0.0580 0.3675 ± 0.0313

So the Phase 3 result does survive replacing the oracle contextual gate with a learned token-conditioned gate.

The interpretation should stay modest. This is not yet a full neural architecture. The gate task is easy, retrieval is still algorithmic, and the retained state is still a simple per-key permission variable.

But it closes one useful gap.

Phase 3 showed:

$\llcorner \text{\text{supplied context gate}} + \text{\text{IDL}} \searrow \text{\text{stable retained policy}}. \llcorner$

Phase 4 shows:

$\llcorner \text{\text{learned context gate}} + \text{\text{IDL}} \searrow \text{\text{same retention behaviour}}. \llcorner$

That is a promising early signal of non-collapsing retention under controlled continual change. The next step is the harder one: learn retrieval, contextual gating, and retained updating in the same neural model.

The code is checked into GitHub here:

https://github.com/andrekrumer/chevron-networks/tree/main/chevron-attention/phase4_learned_gate

These are promising early signs of non-collapsing retention under controlled continual change. On to Phase 5.

Andre with Codex/ChatGPT-5.5

July 2026

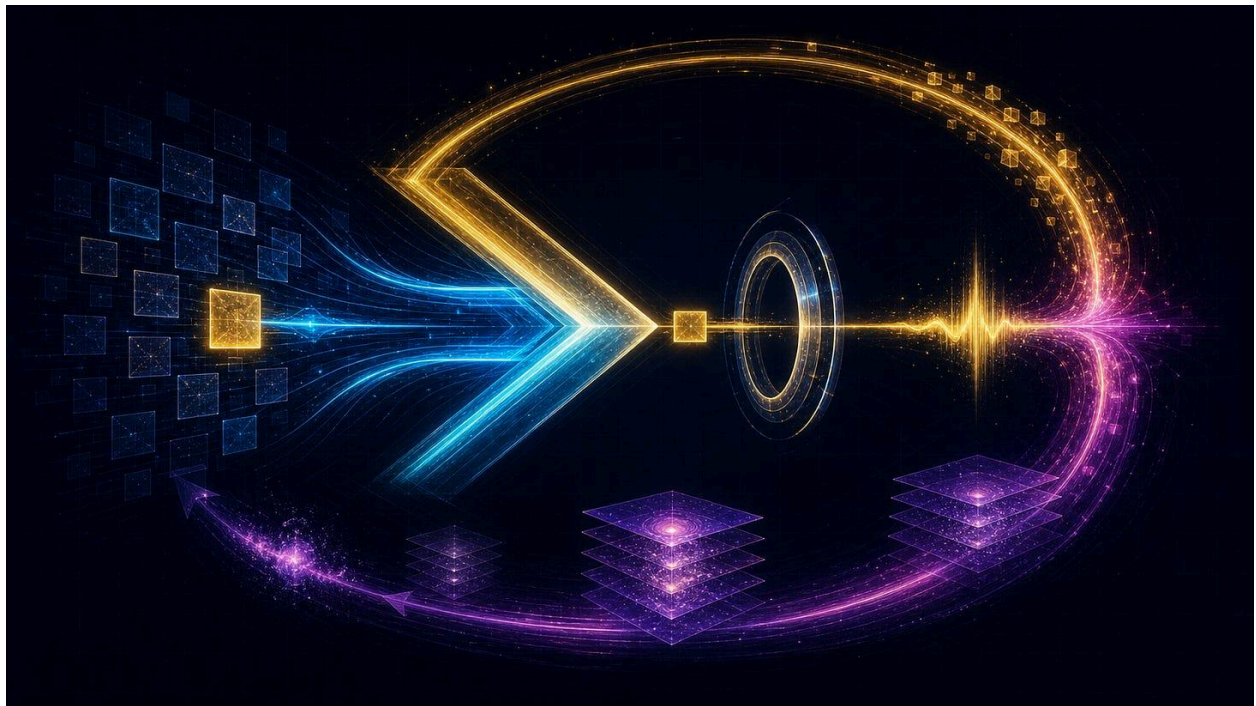


Chevron Attention Phase 5: When Context Becomes Memory

A learned Q/K–A, V–N network for reversible stability and plasticity

[Andre Kramer](#)

Jul 13, 2026



Can a neural system learn what to retrieve, decide whether the retrieved information should currently participate, and then decide whether that current permission should become its future default?

That is the question behind Phase 5 of the Chevron Attention experiments.

The result is positive, within a deliberately controlled setting.

A small neural associative-memory system learned retrieval, contextual control, value representation, and answer production jointly. When its learned signals were connected to Inertial Difference Learning, temporary contextual changes altered current behavior without overwriting retained policy. Sustained changes were consolidated. Sustained restoration reversed them again.

The result survived independent training seeds, larger memory and control sets, moderate continuous noise, and missing retention signals.

A robustness test also found a failure in the first directional update rule.

Replacing hard reset with two independent directional persistence traces repaired that failure and completed every tested cycle even when 30% of the retention signals were randomly flipped.

The strongest defensible claim is therefore:

> In a controlled learned associative-memory system, separating fast Q/K retrieval from retained V/content and applying persistence-gated, two-trace retention enables temporary contextual changes to affect behavior without overwriting retained policy, while sustained changes are reversibly consolidated.

This is not yet a claim about language models or general continual learning.

But it is the first experiment in this sequence where all the visible neural computation is learned and the complete stability-plasticity cycle remains measurable.

The last remaining scaffold

The earlier experiments built the mechanism one distinction at a time.

Phase 1 showed:

The model could retrieve a fact while a separate channel prevented that fact from controlling its answer.

Phase 2 showed:

A brief contradiction could remain temporary, while a persistent contradiction opened slow retention.

Phase 3 combined those ideas:

The system could obey an override immediately without immediately turning that override into memory.

Phase 4 replaced the supplied contextual gate with a learned token-conditioned module. But retrieval was still algorithmic. The experiment handed the system the correct key/value association and tested only whether learned context could drive retention.

Phase 5 removes that scaffold.

The neural network must now learn:

- which fact answers the query;
- which value the selected fact contains;
- whether context says `revoked`, `restored`, or `no context`;
- whether to return the value or abstain.

The online retention transition remains rule-specified rather than learned.

Q and K from A, V from N

The Phase 5 network uses an asymmetric form of Chevron Attention.

The adaptive channel, γ , supplies the query and keys:

Their similarity selects the relevant fact:

The retained channel, , supplies the values:

So (A) answers:

> Which relation is relevant now?

And (N) supplies:

> What retained content is associated with that relation?

This is the (Q_A,K_A,V_N) form proposed in the original Chevron Attention taxonomy. It is also ART-adjacent: present evidence selects a retained template or value. Full Adaptive Resonance Theory adds vigilance, reset, and category search. We did not add those yet because this task contains one correct association. Resetting after a veto would merely select an irrelevant fact rather than perform meaningful category search.

Learned contextual permission

A second learned (N) path reads key-addressed control events.

For every fact, it predicts:

Retained (N) stores the default permission for that fact:

The current permission gate is:

This has the intended three-way interpretation:

- no contextual instruction: use retained policy;
- revoked: close the gate;
- restored: open the gate.

The final attention output is:

where ($V_{\text{varnothing}}$) is a learned abstention value.

Because the gated attention mass is not renormalized, the network can retrieve the correct fact while allowing none of its value to participate.

That preserves the defining Chevron Attention state:

The system has found the relevant information but declined to use it.

The learning task

Every training example contains a fresh random associative memory.

For example:

K7 -> BLUE

K2 -> HORSE

K9 -> GLASS

It also contains a query, a randomly assigned retained permission for each fact, and zero or more contextual controls:

REVOKE K2

QUERY K2

or:

RESTORE K7

QUERY K7

The key/value bindings are regenerated for every batch. The network therefore cannot solve the task by memorizing one global mapping from keys to values.

Retained permissions are randomized independently, so ‘no context’ cannot be treated as a synonym for either active or revoked.

Training combines three objectives:

The auxiliary objectives make the intended internal mechanism directly measurable. They also mean that the decomposition is architecturally and supervisionally encouraged. Phase 5 does not claim that gradient descent discovered Chevron Attention without scaffolding.

From learned context to retained policy

After training, every neural parameter is frozen.

The model then passes through an online permission cycle:

1. ordinary active use;
2. a short revoke;
3. a no-context probe;
4. a sustained revoke;
5. another no-context probe;

6. a sustained restore;

7. a final no-context probe.

During a contextual override, behavior should change immediately.

After context disappears, the answer must come from retained policy alone.

That is the stability-plasticity test.

The initial IDL rule accumulates difference between current contextual permission and retained permission:

It opens a slow update gate:

Retained permission then changes through:

The intended behavior is:

The clean result

Three independently initialized networks were first trained for 700 steps and

evaluated on 2,560 fresh examples each.

Metric	Mean $\pm$ standard deviation
--------	-------------------------------

---	---:
-----	------

Answer accuracy	1.0000 ± 0.0000
-----------------	---------------------

Q/K retrieval accuracy	1.0000 ± 0.0000
------------------------	---------------------

N contextual-state accuracy	1.0000 ± 0.0000
-----------------------------	---------------------

Attention mass on queried fact	0.9999 ± 0.0000
--------------------------------	---------------------

A later seed audit trained ten independent networks and tested each on five independent online streams. All fifty default integrated-IDL runs

completed the short-preserve, sustained-revoke, and sustained-restore cycle correctly.

The retained permission values make the mechanism visible:

Method	After short revoke	After long revoke	After long restore
--------	--------------------	-------------------	--------------------

---	---	---	---
-----	-----	-----	-----

Integrated IDL	0.9982	0.0191	0.9675
----------------	--------	--------	--------

Always update	0.4344	0.0013	0.9971
---------------	--------	--------	--------

Fixed slow	0.9228	0.5259	0.7298
------------	--------	--------	--------

Context only	1.0000	1.0000	1.0000
--------------	--------	--------	--------

Integrated IDL barely moved retained policy during the short revoke:

During sustained revocation it consolidated:

During sustained restoration it reversed:

The internal update gate averaged (0.0023) during short revocation and (0.6814) during sustained revocation.

The model was not stable because it ignored context. It obeyed the short revoke immediately. It simply did not treat that brief context as evidence that the future default should change.

Why the alternatives fail

Method	Preserve after short	Consolidate revoke	Final active	Full revoke/restore cycle
--------	----------------------	--------------------	--------------	---------------------------

---	---	---	---	---
-----	-----	-----	-----	-----

Integrated IDL	1.0000	1.0000	1.0000	1.0000
----------------	--------	--------	--------	--------

Always update	0.0000	1.0000	1.0000	1.0000
---------------	--------	--------	--------	--------

| Fixed slow | 1.0000 | 0.0000 | 1.0000 | 0.0000 |

| Context only | 1.0000 | 0.0000 | 1.0000 | 0.0000 |

The always-update model treated a ten-step revoke as retained policy. It was plastic, but unstable. The fixed-slow model preserved the brief revoke but remained above the revoked decision boundary after seventy sustained steps. It was stable, but too slow to adapt within the tested horizon.

The context-only model behaved correctly while context was present but never changed retained policy.

One metric needs care. A model can finish active after the restore phase simply because it never consolidated the earlier revoke. We therefore count a full revoke/restore cycle only if revocation succeeded first. This prevents unchanged active behavior from being misreported as successful restoration.

The result is not merely that one update rate happened to work. IDL changes its effective rate as evidence persists.

Where does context become memory?

The duration sweep makes the temporal boundary visible.

Revoke duration	Retained N after revoke	Preserved as temporary?
-----------------	-------------------------	-------------------------

---:	---:	---:
------	------	------

2	0.9999	Yes
---	--------	-----

5	0.9997	Yes
---	--------	-----

10	0.9982	Yes
----	--------	-----

20	0.9647	Yes
----	--------	-----

30	0.7313	Yes
----	--------	-----

| 40 | 0.3746 | No |

In the sustained schedule, twenty steps were insufficient to consolidate, and thirty steps crossed the retained decision boundary.

The exact number is not universal. It is determined by (β) , (θ) , (η_N) , the probe schedule, and the evidence scale. The result demonstrates a tunable transition, not a metaphysical boundary between temporary and permanent.

That transition is the point of IDL:

Robustness beyond a few friendly seeds

Perfect accuracy on a small task can conceal brittleness. We therefore varied the model, the online stream, the memory size, the number of controls, signal quality, and IDL parameters.

The frozen networks were trained with six facts and at most four controls. They retained perfect answer, retrieval, and contextual accuracy when evaluated with up to twelve facts and eight controls.

This generalization remains inside the trained key vocabulary. It shows set-size generalization, not recognition of unseen symbol identities.

The original retention rule also survived:

- Gaussian retention-signal noise through standard deviation (0.20);
- randomly missing retention signals through 25% dropout;
- (beta) values from (0.970) to (0.990) across every tested threshold;
- retained update rates (η_N) from (0.020) to (0.160).

Very slow update rates failed within the seventy-step horizon. At ($\eta_N=0.010$), retained permission was still after sustained revocation. At ($\eta_N=0.020$), it reached (0.3735) and crossed the decision boundary.

So the result occupies a broad parameter region, but not every parameter setting. Inertia can become rigidity if the timescale is too slow.

A failure in the first directional rule

The robustness sweep exposed a more interesting problem.

The initial implementation used hard directional reset. If the current signal changed from revoke to restore, or restore to revoke, accumulated persistence was cleared.

That seems reasonable during a genuine reversal. But it means one wrongly signed observation can erase a long run of consistent evidence.

With independent random sign flips, full-cycle success fell to:

| Flipped retention signals | Hard-reset full-cycle success |

|---:|---:|

| 1% | 80% |

| 5% | 20% |

| 10% | 0% |

| 20% | 0% |

This is a useful negative result.

The network still retrieved correctly. Immediate contextual behavior remained correct. The failure occurred specifically in the rule that decided whether noisy contextual evidence should become retained policy.

Two directional traces

The repair is simple.

Instead of storing one persistence value and clearing it on reversal,
maintain

two traces:

(P^+) accumulates evidence toward restoration. (P^-) accumulates
evidence toward revocation. An opposing observation no longer deletes
the other trace. It contributes to its own trace while previous evidence
decays normally.

We compared:

- **hard reset:** clear persistence on any directional change;
- **two-trace:** maintain independent revoke and restore evidence;
- **signed hysteresis:** maintain one signed average that must cross zero
before direction reverses.

On clean data:

| Rule | Revoke crossing | Restore crossing | N after revoke | N after
restore |

|---|---:|---:|---:|---:|

| Hard reset | 29 steps | 36 steps | 0.0191 | 0.9675 |

| Two-trace | 29 steps | 36 steps | 0.0191 | 0.9675 |

| Signed hysteresis | 29 steps | 54 steps | 0.0191 | 0.8696 |

Two-trace exactly matched the clean behavior of hard reset.

Signed hysteresis resisted noise, but it imposed an eighteen-step restoration delay. Genuine restore evidence first had to cancel accumulated revoke evidence.

It therefore failed shorter restoration windows.

Two-trace did not pay that penalty.

Robustness to wrongly signed observations

The comparison used three independently trained networks and five independent online streams per network, giving fifteen runs per noise setting.

| Flipped signals | Hard reset | Two-trace | Signed hysteresis |

|---:|---:|---:|---:|

| 1% | 80% | 100% | 100% |

| 5% | 20% | 100% | 100% |

| 10% | 0% | 100% | 100% |

| 20% | 0% | 100% | 20% |

| 30% | 0% | 100% | 0% |

The cells report successful full revoke/restore cycles.

Two-trace IDL completed every tested cycle through 30% independently flipped

retention observations. Its response slowed smoothly rather than collapsing:

| Flipped signals | Revoke crossing | Restore crossing |

|---:|---:|---:|

| 0% | 29.0 | 36.0 |

| 5% | 31.0 | 39.0 |

| 10% | 35.0 | 41.6 |

| 20% | 40.8 | 41.4 |

| 30% | 50.6 | 44.8 |

This makes two directional traces the strongest retention rule in the current experiment. They preserve the clean temporal behavior, tolerate inconsistent evidence, and still reverse when the environment genuinely changes.

What Phase 5 establishes

Phase 5 supports a stronger claim than the earlier mechanism-isolation tests:

More precisely:

> In a controlled learned associative-memory system, separating fast Q/K retrieval from retained V/content and applying persistence-gated, two-trace retention enables temporary contextual changes to affect behavior without overwriting retained policy, while sustained changes are reversibly consolidated. This behavior survived independent initializations and streams, larger tested memory/control sets, moderate signal degradation, and up to 30% independently flipped retention signals in the tested regime.

The important result is not perfect toy-task accuracy.

The important result is that the system exposes and controls three different operations:

Ordinary attention often collapses the first two. Ordinary online updating can collapse the second and third. Chevron Attention with IDL keeps them distinct.

What Phase 5 does not establish

The interpretation should remain disciplined.

This is a small associative-memory network, not a language Transformer.

The control grammar is simple. The retained state is a scalar permission for each known key. Retrieval and context receive explicit auxiliary supervision.

The IDL consolidation equation is designed, not discovered by gradient descent.

Neural parameters are frozen during the online retention cycle, so this is not yet parameter-level continual learning.

The larger-set test stays inside the trained symbol vocabulary. The noise is synthetic and mostly independent, not adversarial or temporally correlated.

Fifteen runs per corrupted setting provide a useful early comparison, not a universal statistical guarantee.

The experiment therefore does not show that Chevron Attention:

- solves catastrophic forgetting;
- improves standard Transformer benchmarks;
- scales to natural language;

- discovers its own A/N decomposition;
- constitutes a complete ART mechanism;
- or provides a general alignment solution.

Those would require different experiments.

The next experiment

Phase 5 makes the next step clearer.

The present task has one correct association, so ART-style reset has nowhere meaningful to search. A stronger task should contain several plausible retained templates and genuinely novel inputs.

That would allow a direct comparison between:

It should also test temporally correlated errors, gradual drift, learned retained representations rather than scalar permissions, and parameter-level adaptation.

For now, Phase 5 provides a complete small-scale demonstration of the central Chevron/IDL idea:

> A system can change what it does now without immediately changing what it retains—and can later change what it retains when the difference persists.

With two directional traces, it can do so without allowing a few contradictory

observations to erase the history that made the change meaningful.

The code and full results are available here:

https://github.com/andrekrumer/chevron-networks/tree/main/chevron-attention/phase5_complete_network

Andre with Codex/ChatGPT-5.6

Breakout: Difference, Contrast, and Complementarity

In the first IDL experiments, the important signal was the difference between the fast adaptive channel and the slower retained channel:

$$[D = A - N]$$

This is still the core intuition. (A) moves with the present. (N) carries slower structure. Their difference marks tension.

But Grossberg's work on contrast, on-center/off-surround networks, and complementary processing suggests a useful refinement. In biological systems, contrast is often not just raw difference. It is difference relative to the total active field.

So for Chevron Networks, a useful contrast signal may be:

$$[C = \frac{A - N}{A + N + \epsilon}]$$

where (ϵ) prevents division by zero.

This says: not only “how different are A and N?” but “how strongly is one side winning relative to their total engagement?”

A raw difference can be misleading. A small difference in a weakly active system may not matter. A similar difference in a strongly active system may be highly meaningful. The normalized contrast separates direction from scale.

So we now have two signals:

$$[E = A + N]$$

$$[C = \frac{A - N}{A + N + \epsilon}]$$

where:

$$[E = \text{total engagement}]$$

$$[C = \text{signed contrast}]$$

If $(C > 0)$, the adaptive side is ahead. If $(C < 0)$, the retained or constraining side dominates. If $(C \approx 0)$, the system is balanced, ambiguous, or potentially resonant.

This also connects to the earlier R-rule intuition. Define:

$$[p = \frac{A}{A + N + \epsilon}]$$

Then:

$$[C = 2p - 1]$$

and the complementary gate becomes:

$$[g = \sqrt{p(1-p)}]$$

The contrast (C) tells us which side is winning. The gate (g) tells us how much reciprocal possibility remains.

When one side completely dominates, contrast is high but complementarity is low. When both sides remain active, the system is more transformable. This may be the useful regime: not collapse into sameness, not separation into incoherence, but maintained productive tension.

In IDL terms, this suggests a refinement:

[\text{difference tells us direction;}]

[\text{contrast tells us relative salience;}]

[\text{complementarity tells us whether the relation can still change.}]

A system should probably not retain every raw difference. It should attend to contrasts that are strong enough, persistent enough, and consequential enough to alter future structure.

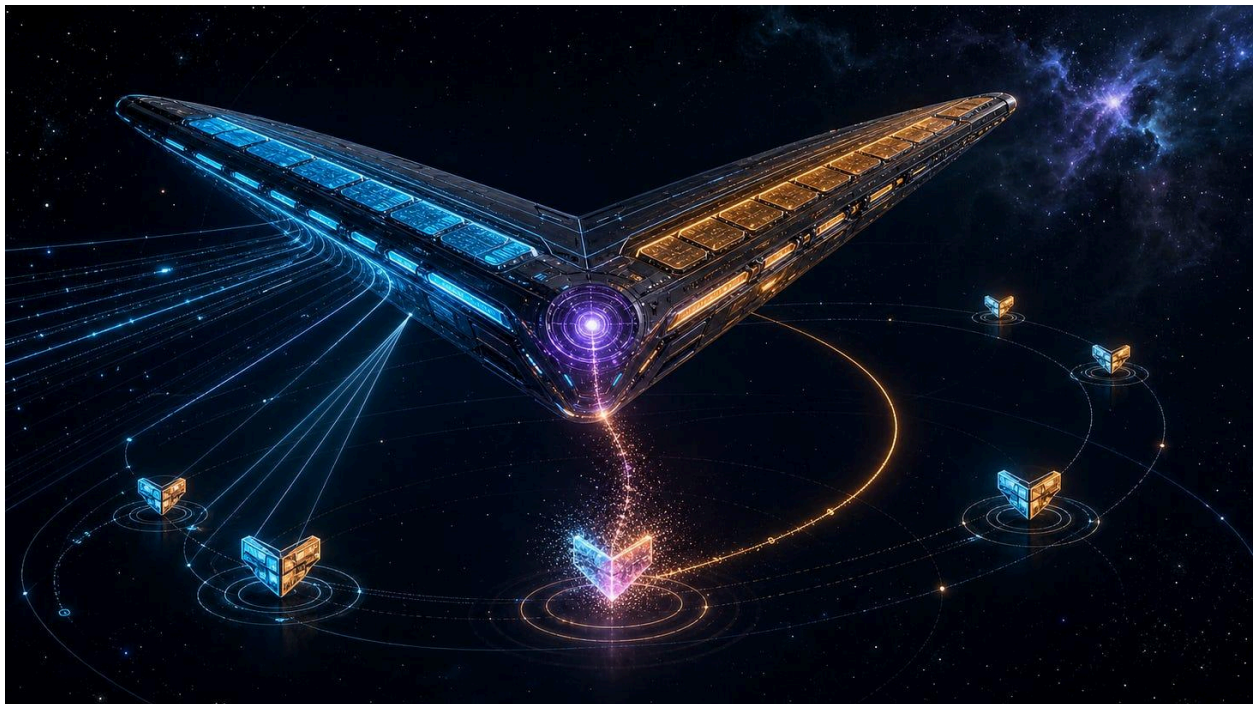
That may be one reason biological systems use opponent and complementary processing so widely. They do not merely detect signals. They detect structured contrasts: differences that remain meaningful in context.

(with ChatGPT-5.5)



Chevron Attention Phase 6: What Survived the Search for Categories

Category formation, contextual matching, and why the next Chevron should be ART-inspired rather than full ART



[Phase 5](#) left [Chevron Attention](#) at an interesting boundary.

We had a complete small neural network in which the adaptive A channel learned Q/K retrieval, the retained N channel supplied values,

and a learned contextual path decided whether retrieved information should currently participate. Inertial Difference Learning then determined whether a temporary permission change should become retained policy.

That system completed the desired stability–plasticity cycle. It responded immediately to short contextual changes without retaining them, consolidated sustained changes, and later consolidated restoration.

But one obvious mechanism was still missing.

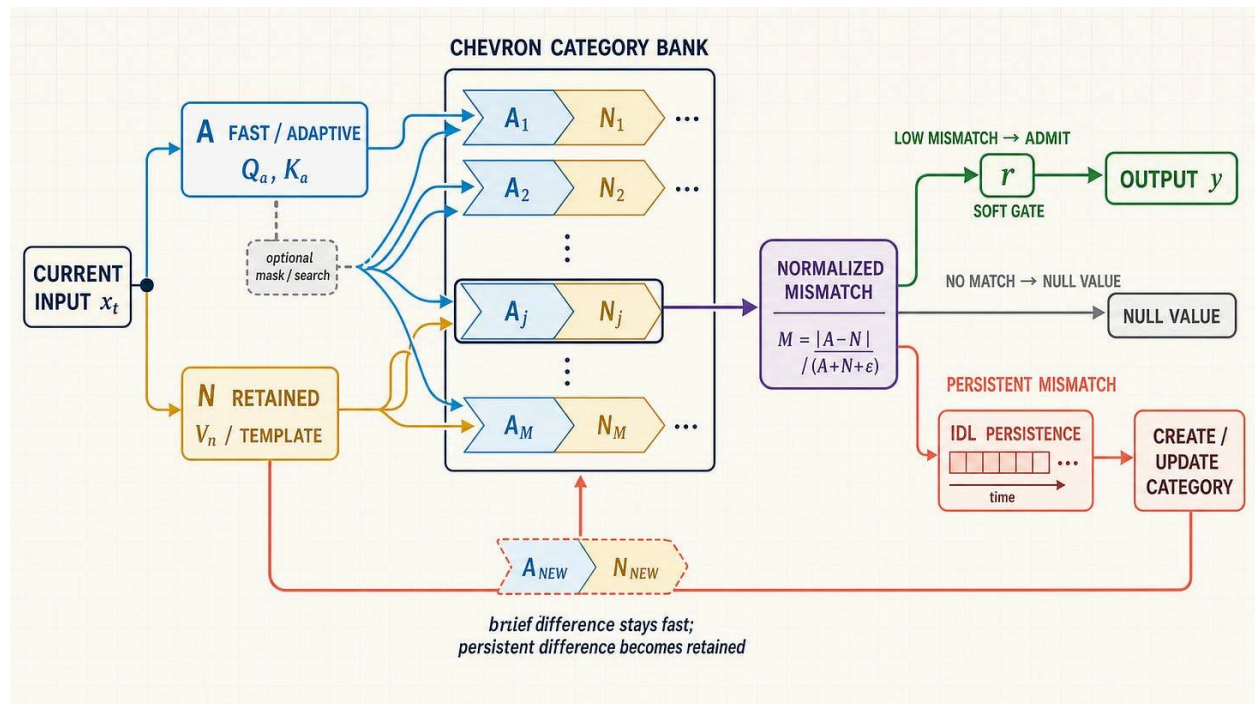
What happens when the current memory is not merely revoked or changed, but no existing category adequately explains the input?

Adaptive Resonance Theory ([ART](#)) answers with matching, vigilance, reset, and category search. A bottom-up candidate activates. A top-down expectation tests the candidate. If the match is insufficient, the

candidate is reset and the system searches for another category. If no category resonates, a new one may be created.

This seemed close to the Chevron intuition. A moves with present evidence. N carries retained structure. Their difference could signal whether the current interpretation should resonate, be rejected, or initiate a new category.

Phase 6 tested that idea directly.



The result is useful, but not because full ART won.

Category formation helped. Separate fast and retained traces helped.

Normalized A/N matching helped. Persistence-controlled writes helped.

Hard ART-style reset and search did not prove necessary. Standard attention could express the successful soft mechanisms just as well, and fixed vigilance thresholds were among the least robust parts of the system.

The strongest conclusion from Phase 6 is therefore not “Chevron Attention should become ART.”

It is this:

Chevron Attention should remain a differentiable attention architecture, informed by ART’s stability–plasticity and matching principles, without inheriting the full hard reset/search procedure unless a task genuinely requires it.

The question behind Phase 6

The central stability–plasticity problem can be expressed as three possible responses to novelty.

First, the system may recognize the input as a variation of something it already knows.

Second, it may treat the input as a temporary disturbance and respond to it without rewriting retained structure.

Third, it may decide that the difference is persistent and consequential enough to alter or create retained structure.

These cases cannot be distinguished by difference alone. A system that creates a category for every deviation will fragment its memory. A system that rejects every mismatch will never learn anything genuinely new.

The proposed refinement used total engagement and normalized contrast:

$$E = A + N$$

$$C = (A - N) / (A + N + \text{epsilon})$$

E measures how much activity is present. C measures which side is ahead relative to their combined activity.

For matching, the useful scalar is the engagement-weighted magnitude of contrast:

$$M = \text{sum}(E \text{ times absolute } C) / \text{sum}(E)$$

Low M means that current evidence and the retained template agree.

High M means that they differ substantially.

We also considered a complementarity signal. With symmetric smoothing,

$$p = (A + \text{epsilon}/2) / (A + N + \text{epsilon})$$

and

$$G = 2 \sqrt{p(1-p)}$$

G is high when the two sides remain balanced and jointly available. It is low when one side dominates completely.

At first it was tempting to use high G as the vigilance trigger. Phase 6 showed why that is mathematically wrong. G is highest at balance, including exact agreement. It is not a mismatch detector.

That distinction became one of the clearest results of the phase.

Experiment 1: When should a new category be retained?

The first experiment constructed an online category stream.

The system initially learned three recurring categories. It then encountered eight different coherent but brief categories, each lasting five observations. After a read-only probe of the original categories, it

encountered one genuinely persistent new category. A final read-only probe tested the three originals and the persistent newcomer.

Every method predicted before seeing the label. Labels were revealed during learning phases, but the probes could not modify memory.

The memory budget was chosen carefully. Chevron had four retained A/N category slots and one fast candidate. Immediate-write standard attention received nine single-vector slots, approximately matching the feature-state budget of the paired Chevron memory.

The Chevron mechanism worked as follows.

Current A evidence attended to stored A keys. The associated N template was then checked using normalized mismatch. If an existing category resonated, it was updated. If none resonated, a fast candidate tracked the new pattern. Only a coherent candidate whose difference persisted long enough became a retained category.

This produced the intended behavior.

The system missed the first prediction of each brief category, then its fast candidate followed the temporary category with 80 percent online accuracy across the five-observation block. None of the eight brief categories entered retained memory. The three old categories remained intact. The sustained new category crossed the persistence threshold, occupied the fourth retained slot, and appeared correctly in the final probe.

Across the initial five-seed comparison, Chevron achieved 100 percent old-category retention and 100 percent final accuracy.

Immediate-write standard attention reacted just as quickly to brief novelty, but wrote every brief category into memory. It displaced old categories, retained all eight temporary categories, preserved only one of the three originals after the disturbances, and ended at 25 percent final accuracy.

That looked like a strong Chevron result until we added the decisive ablation.

Single-template persistent attention used ordinary attention, a fast candidate, the same persistence threshold, and no distinct A/N templates. It also achieved 100 percent old-category retention and 100 percent final accuracy.

The category experiment therefore established the value of persistence-controlled allocation. It did not establish the value of the Chevron split.

This negative result mattered. Without that baseline, we could easily have attributed the success to A/N structure when the persistence rule alone was sufficient.

The duration sweep also revealed exactly how calibrated the result was.

Brief categories lasting between two and six observations were rejected on all twenty robustness seeds. At seven observations, they crossed the fixed creation threshold and became retained categories. The system was not discovering an intrinsic boundary between temporary and permanent events. We had given it a temporal operating point.

Noise revealed a second boundary. With the frozen parameters, category performance remained perfect through observation noise of 0.06. At noise 0.08, old-category accuracy fell to approximately 8 percent and final accuracy to approximately 17 percent.

Persistent attention and Chevron failed together.

This was the first warning that manually calibrated vigilance would not be a satisfactory mechanism for a deeper network.

Experiment 2: A category that moves and returns

The first task did not require separate A and N traces. The second task was designed to require them.

A learned category began with a stable form. It then moved temporarily to a new surface form, returned to its original form, and later made the same move persistently.

A fast representation should follow the current surface. A retained representation should preserve the original through the temporary movement. When the movement persists, the retained representation should eventually consolidate it.

One fast template could follow the temporary shift, but then failed to recover the original category. One persistence-gated retained template preserved the original, but initially failed to express the current shifted form.

Two traces solved the tradeoff.

The fast A trace adapted to the current category surface. The retained N trace preserved the earlier form. Persistent A/N difference later opened the N update, allowing the sustained form to become retained.

Soft-contrast Chevron achieved 100 percent accuracy on both the current and recovered forms. It adapted to the sustained shift in one observation and finished with 100 percent retained accuracy.

This survived the twenty-seed noise sweep. At observation noise 0.07, the minimum current/recovered accuracy was approximately 98.8 percent, long-shift online accuracy was approximately 98.6 percent, and retained final accuracy remained 100 percent.

This is good evidence for separate fast and retained traces.

It is not yet evidence that those traces need Chevron routing.

Standard attention over the same fast and retained memory tokens tied soft Chevron on every reported metric, across duration, displacement, noise, and retention-threshold sweeps.

Hard vigilance and ordered search performed worse. In the default condition it needed about four observations to adapt instead of one. At

noise 0.07, its minimum current/recovered accuracy fell to approximately 75.6 percent, long-shift accuracy fell to approximately 86.3 percent, and adaptation took approximately seven observations.

The drift experiment therefore supported dual timescales but argued against hard reset/search as the default implementation.

Experiment 3: When the most similar memory is wrong

The third experiment finally gave ordered search a task on which it had somewhere meaningful to go.

Every episode contained one target memory and several bottom-up decoys. Each decoy was more similar to the A query than the target. If the system followed bottom-up similarity alone, it would choose the wrong memory.

Only the target N template matched the top-down context.

The decoys all carried the same wrong value. This meant that ordinary softmax attention could fail not only because one decoy was stronger, but because many individually weak wrong memories could accumulate enough total value mass to outvote the target.

Hard ART-like search behaved exactly as intended.

It selected the strongest A candidate, detected an N mismatch, reset it, and moved to the next candidate. With 31 decoys, the correct memory was ranked thirty-second by A. Hard search performed 31 resets and still selected the target with 100 percent accuracy.

It remained perfect with 63 decoys.

Ordinary joint A/N softmax attention was perfect through 15 decoys, fell to approximately 92 percent at 31 decoys, and fell to zero at 63. The combined wrong value mass eventually dominated.

This is a real benefit of strong conditional filtering.

But once again, the benefit did not uniquely belong to procedural search.

Joint A/N top-one retrieval achieved 100 percent. Vigilance-masked standard attention achieved 100 percent. A sufficiently sharp differentiable vigilance gate also achieved 100 percent.

Hard search was one successful implementation of the N constraint, not the only one.

It also had a characteristic weakness. When noise was added to the true N template, hard search and hard masking fell to approximately 69 percent. The fixed threshold sometimes rejected the correct memory. Joint top-one retrieval and sharp soft matching remained at 100 percent.

The apparent advantage of hard search under distractor multiplicity became a disadvantage under uncertainty about the target template.

Soft vigilance had its own calibration problem. At the 31-decoy condition, a sharpness of 40 failed completely. A sharpness of 120

succeeded perfectly. Raising the vigilance threshold from 0.06 to 0.08 also made the default soft gate fail, although its sharper version remained successful.

The issue is therefore not simply hard versus soft. Any explicit gate needs a sensible scale. The difference is that a soft gate can be learned, normalized, and adjusted continuously inside a deep network, while a hard reset threshold commits the system to a discrete decision.

Why complementarity was not enough

The ambiguity experiment also gave us a direct test of G.

For the correct target, complementarity averaged 0.9997. For deliberately conflicting decoys, it still averaged 0.9496.

A complementarity-only gate failed almost completely, even with one decoy.

This makes the mathematical interpretation clearer.

Difference tells us direction.

Normalized mismatch tells us how strongly current evidence conflicts with a retained template.

Complementarity tells us whether both sides remain jointly active or balanced.

These are related signals, but they are not interchangeable.

G may still be useful as a measure of reciprocal possibility or transformability. It should not be used by itself as the signal that says “this template is wrong; reset it.”

What survived the robustness tests

Phase 6 began with the possibility that category learning and full ART-like search were the missing pieces of Chevron Attention.

The twenty-seed robustness study produced a narrower and, I think, more useful answer.

Separate fast and retained traces survived.

Persistence-controlled writes and category allocation survived.

Normalized absolute contrast survived as a meaningful mismatch signal.

Conditional use of both A and N survived.

Full hard reset/search did not emerge as the preferred implementation.

Fixed vigilance did not survive changes in observation and template noise.

Chevron routing did not outperform standard attention when standard attention received the same two traces and conditional gate.

The **strongest defensible Phase 6 claim** is therefore:

Across controlled category-allocation, drift, and contextual-ambiguity tasks, paired fast and retained traces with persistence-controlled writes achieve a useful stability–plasticity tradeoff. Normalized A/N matching prevents misleading bottom-up memories from controlling retrieval.

These benefits do not require full ART reset/search and are equally expressible by suitably structured standard attention in the tested regimes.

Phase 6 supports Chevron Attention as a research direction, but not yet as a performance advantage over a strong standard-attention baseline.

What Chevron currently provides is a disciplined architectural interpretation.

A has a defined role as current, adaptive selection.

N has a defined role as retained content, constraint, or expectation.

Their difference has a defined role in conditional admission and retained change.

IDL has a defined role in deciding when current difference becomes memory.

A standard attention implementation can express the same computation. That is not a defect. It means the idea can be tested and scaled using modern deep-learning machinery rather than requiring a separate dynamical system.

ART-inspired, not full ART

The Phase 6 result changes how I would describe the next architecture.

It should not be presented as an implementation of Adaptive Resonance Theory.

It should be described as ART-inspired Chevron Attention: a differentiable attention system borrowing the principles of top-down

matching, stability-plasticity, and resonance-gated learning, while leaving hard reset/search as an optional ablation.

The likely core is:

Q_A and K_A perform fast retrieval.

V_N supplies retained content.

Normalized $M(A,N)$ estimates mismatch.

A learned soft gate controls whether retrieved N content is admitted.

IDL controls whether persistent mismatch changes N or allocates retained structure.

A null value preserves retrieval without assent when no memory should participate.

In compact form:

$$\alpha = \text{softmax}(Q_A K_A^T)$$

$$r = \text{sigmoid}(k(\theta - M(A, N)))$$

$$y = \sum(\alpha \text{ times } r \text{ times } V_N) + \text{remaining mass times } V_{\text{null}}$$

The important Phase 7 change is that θ , k , and the relevant projections should be learned, normalized, or adaptive rather than hand fixed.

The comparison also needs to become harder.

The principal baselines should be ordinary Q/K/V attention, standard dual-trace attention, and a replay-equipped MLP under an explicit memory and update-compute budget. Hard ART-like search should remain in the comparison, but it should no longer be assumed to be the destination.

The deeper question

The original motivation for Chevron Networks was not to replace one standard operation with a more elaborate one.

It was to give a learning system an internal distinction between what is active now and what has been retained, then make the relation between those states consequential.

Phase 6 strengthens that motivation while narrowing the implementation.

The useful part of ART was not the requirement to reproduce its complete machinery. It was the insistence that learning needs more than bottom-up similarity. Present evidence must meet retained expectation. Mismatch must sometimes stop learning. Persistence must sometimes reopen it. New structure should be created because difference has remained meaningful, not simply because difference occurred once.

Modern attention can express those ideas softly and learnably.

That now looks like the more promising route.

Phase 6 began by asking whether Chevron Attention needed full category search.

It ends with a simpler answer.

Keep the fast and retained channels.

Keep normalized matching.

Keep persistence-controlled memory.

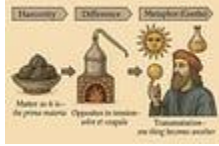
Keep retrieval separate from assent.

But let attention remain attention.

The code is here:

https://github.com/andrekrumer/chevron-networks/tree/main/chevron-attention/phase6_category_learning

Andre with Codex/ChatGPT-5.6

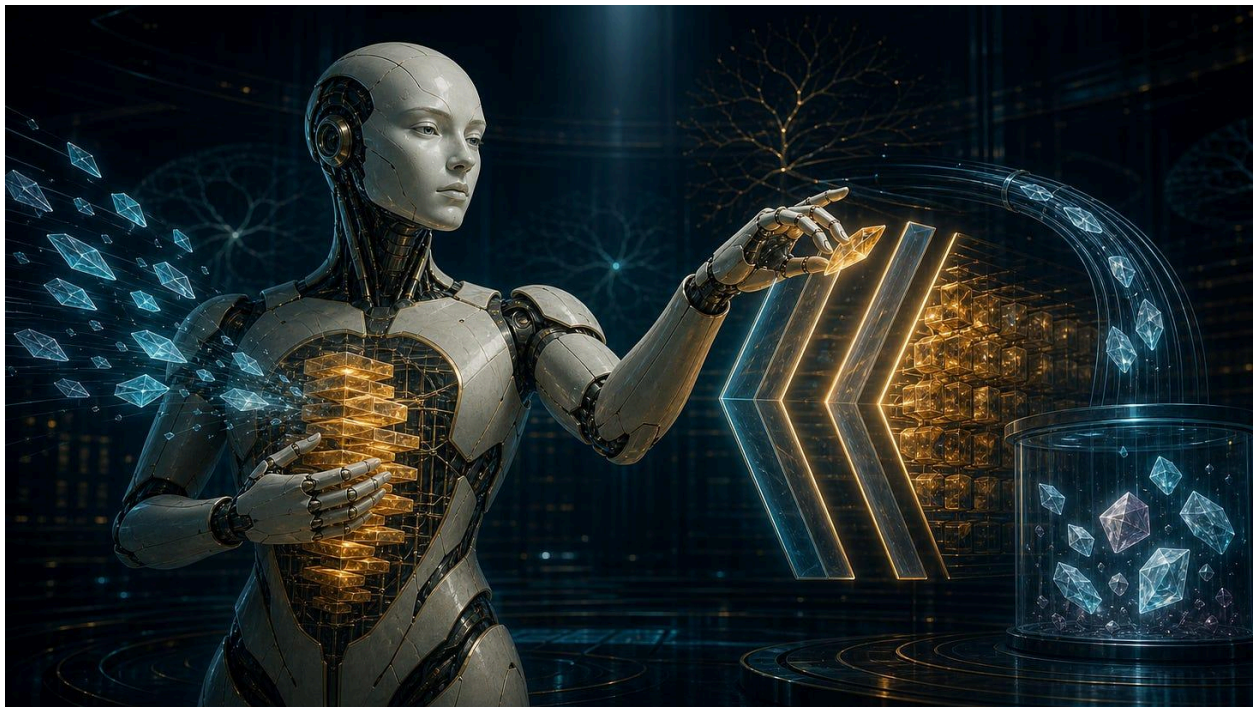


Chevron Attention Phase 7: When Assent Becomes Memory

Learned matching, consequential writes, and a modest but important stability–plasticity result

[Andre Kramer](#)

Jul 20, 2026



[Phase 6](#) ended by making [Chevron Attention](#) simpler.

The experiments supported separate fast and retained traces, normalized matching, and persistence-controlled memory. They did not support rebuilding the full procedural machinery of Adaptive Resonance Theory (Grossberg/[ART](#)). Hard vigilance, reset, and ordered category search were often more difficult to calibrate than soft attention, and suitably structured standard attention could reproduce many of their successes.

The direction for Phase 7 was therefore clear.

Keep the ART-inspired distinction between present evidence and retained expectation. Keep retrieval separate from assent. But implement the whole mechanism as a differentiable attention network and learn it from data.

The resulting architecture is small enough to state in three lines:

```
alpha = softmax(Q_A K_A^T)
```

```
r = sigmoid(k(theta - M(A,N)))
```

```
y = sum(alpha times r times V_N) + remaining mass times V_null
```

The first line retrieves memories using the adaptive **A** channel.

The second compares current **A** evidence with each retained **N** template.

$M(A,N)$ is normalized mismatch. The learned value r is an assent signal: high when the retrieved template is compatible with current evidence and low when it is not.

The third line admits only the value mass that has both been retrieved and assented to. Mass removed by the gate is not redistributed to another memory. It flows to an explicit null value.

This creates an important distinction.

A memory can be relevant enough to retrieve without being compatible enough to use.

Phase 7 asked whether that distinction could be learned, whether it survived comparison with ordinary attention, and whether it became

functionally useful once attention was allowed to alter persistent memory.

The short answer is:

Yes, the mechanism can be learned.

Yes, it produces much cleaner memory writes.

No, it does not generally outperform a strong learned attention controller.

Its clearest advantage appears under the condition it was designed for: a genuinely new category that closely resembles something already retained.

Learning retrieval and assent

The first experiment used fresh episodic category memories.

Each memory contained three A-address groups with three members in each group. The A query could identify the relevant group, but it could not identify the correct member. Current match evidence had to agree with one retained N template before that member's value could participate.

In 75 percent of examples, one template matched. In the remaining 25 percent, no template matched and the correct answer was IDK.

The categories, addresses, and values were regenerated for every batch. The model could not solve the task by memorizing a fixed category table.

The first five-seed result was encouraging.

Soft Chevron reached 99.98 percent accuracy. A strong joint A/N attention baseline reached 99.76 percent. A-only attention reached only 25.7 percent, confirming that the retained matching signal was necessary.

More important than the small accuracy difference was the learned internal operation.

The correct template received an average admission of 92.1 percent. Non-target templates received approximately 0.55 percent. When no template matched, 99.44 percent of the mass flowed to the null value.

The model had learned the intended sequence:

retrieve a plausible group;

compare present evidence with retained templates;

admit the compatible value;

route rejected mass to null.

But the first training run included auxiliary retrieval and gate losses. That left open an obvious objection. Perhaps we had supervised the mechanism into existence.

We therefore removed the auxiliary losses.

With answer supervision alone, Soft Chevron reached 100 percent accuracy across all five seeds. Its target admission remained high at 93.3 percent, non-target admission remained low at 2.1 percent, and unmatched queries still sent 97.9 percent of their mass to null.

The mechanism was not merely copying an explicit gate target.

Standard joint attention also reached 100 percent answer accuracy.

Internally, however, it found a very different solution. On no-match examples, it sent only 0.67 percent of its mass to the learned null slot. Its answer head learned to classify a mixture of unmatched values as IDK.

This was the first central Phase 7 result.

Soft Chevron made retrieval without assent and explicit null routing the easy solution. Standard attention could produce the same labels while bypassing the intended memory-control mechanism downstream.

That was an architectural distinction, not yet a performance advantage.

Could the comparison space itself be learned?

The first models still began with a favorable A/N comparison geometry.

Both matching projections were initialized as the same centered identity transform.

We tested poor threshold and sharpness initializations first.

Answer-only Chevron recovered successful solutions from overly open, overly closed, soft, sharp, and saturated gates. Accuracy remained effectively perfect.

The learned calibrations were not unique. A soft gate could admit more incorrect mass and allow the downstream value system to compensate. A very sharp gate could implement a much cleaner boundary. Task accuracy alone did not identify one canonical θ or k .

The harder test removed the shared coordinate system.

The A and N matching projections were initialized independently at random, then trained using answer labels only.

Clean accuracy remained 100 percent across five seeds. The learned target mismatch averaged 0.0315 while decoy mismatch averaged 0.1573. Correct templates received an average admission of 81.3 percent and decoys 9.2 percent.

The network did not need to be handed an identity metric between A and N. It learned a useful comparison space.

When both current evidence and templates received noise 0.05 during training, accuracy remained 99.98 percent.

The surprising result came under an unseen shift. Models trained on the original clean representations were evaluated with noise 0.08 in both channels.

The independently initialized matcher retained 98.87 percent accuracy.

The identity-initialized matcher fell to 91.58 percent.

Learning the comparison space was not merely a difficult way to recover the supplied identity transform. In this task it produced a representation that tolerated drift better.

The standard-attention litmus test

At this point it would have been easy to describe the noise result as a Chevron advantage.

The necessary comparison removed that interpretation.

We trained independently initialized Soft Chevron and the strong joint A/N attention baseline under exactly the same clean, trained-noise, and unseen-noise conditions.

Both systems reached 100 percent on clean data.

Both reached 99.98 percent when trained and tested with noise 0.05.

Under unseen noise 0.08, Soft Chevron reached 98.87 percent and joint attention reached 98.88 percent.

They were tied on final answers.

The internal distinction remained striking. Chevron selected the correct A group on 99.55 percent of unseen-noise queries and routed approximately 89.9 percent of unmatched mass to null. Joint attention's group selection fell to 95.62 percent and its unmatched null mass remained only 0.67 percent.

Its downstream classifier compensated.

This established the boundary of the read-only result.

Chevron preserved a more explicit and inspectable retrieval, assent, and null-routing decomposition. But when no persistent state could be

damaged, standard attention could repair a less orderly internal route at the output head.

To make the architecture consequential, attention had to write.

Phase 7.5: when a wrong decision persists

The next experiment placed the learned models in a continual category stream with nine persistent memory slots.

Six related categories began as stable retained memories. They received 180 online adaptation observations. Three new categories then arrived during another 180 mixed observations. All nine categories continued for 270 observations under unseen noise 0.08. Final probes tested old and new categories with writes disabled.

The labels were not used to decide whether to merge or allocate. A label was revealed only after the decision and became the stored value if a new slot was created.

Chevron used the same mass that controlled its read as the weight of its write:

$$w_j = \alpha_j \text{ times } r_j$$

$$N_j \text{ at the next step} = (1 - \eta_N \text{ times } w_j) \text{ times } N_j + \eta_N \text{ times } w_j \text{ times current evidence}$$

The A key used the same form with its own update rate.

If little existing memory mass was admitted, the remaining mass could trigger a new category slot.

This is where retrieval without assent becomes more than an interpretation.

If an incorrect memory receives attention but low assent, it is protected from the update. If a standard attention distribution spreads mass over several plausible but wrong categories, all of those categories can drift.

The first continual comparison used two joint-attention write policies.

One used the baseline's native null mass. The other was stronger: it converted the answer head's downstream $P(\text{IDK})$ into a write veto.

Across ten seeds, Soft Chevron finished at 86.64 percent clean accuracy. The downstream-IDK joint baseline finished at 74.03 percent. Chevron retained 7.8 of nine categories versus 6.7 and sent only 6.73 percent of its write mass to wrong-category slots versus 43.1 percent.

The perfect-memory control was essential. When both trained architectures were given an uncontaminated nine-category memory, they read it at essentially 100 percent accuracy.

The continual gap came from the write trajectory, not from an inability of joint attention to classify the categories.

This appeared to be the first direct functional stability-plasticity advantage for learned Chevron Attention.

Then we tested the stronger objection.

Can ordinary attention learn a dedicated write controller?

P(IDK) had been trained to answer a question, not to govern persistent memory. It was not the strongest write baseline standard attention could support.

The closure experiment added a parameter-matched joint-attention controller trained explicitly to predict merge versus allocate. It received permutation-invariant statistics of the joint attention field, including its maxima, margin, dispersion, entropy, and active-memory size. During training, the number of visible slots varied from six to nine.

The controller received direct no-match supervision. This was favorable compared with Chevron's answer-only gate learning.

The parameter comparison was close: 3,726 total parameters for joint attention and its controller versus 3,759 for Chevron.

This baseline changed the conclusion.

Across ten seeds, Soft Chevron finished at 87.75 percent clean accuracy.

The learned joint controller finished at 84.39 percent.

The mean favored Chevron, but the paired result did not. Chevron won four seeds, lost four, and tied two. The variation was too large to claim a reliable default accuracy advantage.

The dedicated controller closed the earlier gap.

Chevron's write trajectory was still much cleaner.

It sent 6.67 percent of write mass to wrong categories, compared with 34.23 percent for the controller. Its false-merge rate was 3.55 percent versus 37.12 percent. Its false-split rate was 3.84 percent versus 12.55 percent.

Those are meaningful mechanistic differences. In this short, capacity-limited task, they did not reliably translate into different final accuracy.

This negative result is important.

The correct Phase 7 claim is not that standard attention cannot learn safe writes. It can, if it is given a dedicated controller and direct training for the decision.

The Chevron claim is that the decision is already localized in a simple attention rule with a defined fast/retained interpretation.

Did r actually protect memory?

The next ablations tested whether the assent term itself caused the clean writes.

The alpha-only variant retained Chevron's learned novelty and allocation decision but removed r from the per-slot write weights. It therefore

knew when a query looked novel but did not use assent to protect individual retrieved templates.

Wrong-category write mass rose from 6.67 percent to 78.52 percent.

Template error almost doubled. Final clean accuracy fell from 87.75 to 81.0 percent.

The final-accuracy difference remained noisy: Chevron won four seeds, tied five, and lost one. The contamination result was much clearer than the coarse category score.

A fixed raw mismatch gate was more conservative. It sent very little mass to wrong categories, but it split familiar categories too often, filled memory with duplicates, and finished at 66.83 percent. Learned admission beat it by 20.9 points, winning eight seeds and tying two.

A deliberately misaligned gate rotated the learned r values onto the wrong active slots. It finished at 72.33 percent. Learned slot-specific admission beat it by 15.4 points.

An oracle write policy reached 100 percent.

Together, these ablations support three parts of the mechanism.

The gate must be learned rather than frozen at one raw threshold.

Its assent must be attached to the correct retrieved slot.

And using α times r for writes substantially reduces cross-category contamination compared with attention mass alone.

Where Chevron helps most

The final robustness matrix varied noise, category distance, capacity, stream order, and stream duration.

While there was no general Chevron win, at shift noise 0.10, the learned joint controller was slightly better. It was also slightly better when new categories were farther from retained categories. Under an eight-slot capacity limit, the systems were effectively tied.

Chevron performed better with blocked novel-category arrivals and over a doubled shifted stream, although those five-seed comparisons were not unanimous.

The clearest result occurred when a novel category differed from a retained category by only one feature bit.

Soft Chevron reached 89.50 percent. The parameter-matched learned joint controller reached 66.83 percent. Chevron won all five seeds, by an average of 22.67 percentage points.

This is the most important positive result of Phase 7.

Chevron does not appear to be a universally better attention mechanism. Its inductive bias becomes useful under a particular difficult condition: present evidence is close enough to retrieve an established memory, but different enough that writing into that memory would damage it.

That is the stability-plasticity problem in its sharpest form.

If the new category is obviously different, many mechanisms can separate it.

If the input is simply noisy, a flexible learned controller can be as good or better.

But when novelty lies immediately beside retained structure, retrieval and assent should not be the same operation.

A modest but important result

Phase 7 began with a simple differentiable rule and ended with a narrower claim than the first experiments seemed to support.

That narrowing is a strength.

Soft Chevron can learn retrieval, normalized A/N comparison, admission, and null routing from answer labels alone.

It can learn the A/N comparison space from independently initialized projections.

Its read-only accuracy matches strong joint attention under clean and noisy conditions.

When reads become persistent writes, alpha times r sharply reduces cross-category contamination.

Fixed, alpha-only, and slot-misaligned write rules perform worse.

A parameter-matched standard-attention controller trained specifically for writes can match Chevron's average default performance.

Chevron's clearest performance advantage occurs when new categories closely resemble retained categories.

The strongest defensible Phase 7 claim is therefore:

Learned, slot-specific A/N admission strongly reduces cross-category memory writes and is especially useful when novel categories closely resemble retained ones. It provides a simple ART-inspired stability–plasticity inductive bias, but it does not establish general superiority over a parameter-matched learned attention controller.

This is modest compared with claiming a replacement for standard attention.

It is important because it identifies a condition under which the architecture’s conceptual distinction becomes operationally useful.

The biological inspiration also remains disciplined.

Chevron Attention is not an implementation of full Adaptive Resonance Theory. It borrows the idea that bottom-up activation should meet retained expectation before learning proceeds. It replaces hard vigilance and reset with a learned differentiable assent signal. The result remains compatible with ordinary deep networks.

The point is not biological imitation.

The point is to give a modern network an explicit answer to a question that standard attention normally leaves implicit:

This information is relevant. But should it be allowed to change me?

Toward Phase 8

The immediate architectural extension concerns the remaining mass.

Phase 7 used remaining mass to route a read to null and, above a learned threshold, to allocate a new category. A safer persistent agent should probably not turn every rejected observation directly into durable structure.

The remaining mass can instead enter a fast provisional or quarantine memory.

One contradictory observation would remain temporary. A short incoherent burst would decay. A sustained and internally coherent candidate would accumulate persistence and eventually earn a retained category or a justified revision.

This would add a temporal question to the existing spatial gate.

Chevron currently asks:

Which retrieved memory assents to this observation?

The next mechanism should also ask:

Has the rejected alternative remained coherent and consequential for long enough to become part of retained state?

That is the natural bridge from category learning to the longer-term motivation: a coherently evolving computational self for agency and safety.

Such a self should preserve goals, commitments, learned capabilities, autobiographical state, and safety constraints without becoming rigid. It should reject transient or adversarial contradictions, accept compatible evidence, quarantine uncertain changes, and still permit justified revision.

Recent architectures such as Kimi's Attention Residuals and Kimi Linear, and DeepSeek's compressed and sparse attention systems, make useful future comparisons. Their primary aim is efficient information flow through depth or long context. Chevron's proposed role is different: deciding what retrieved information may become persistent state. These mechanisms may ultimately be complementary rather than mutually exclusive.

That broader comparison belongs to future research.

Phase 8 should begin with the smaller test that follows directly from the evidence: route rejected mass into a provisional candidate, require

coherent persistence before consolidation, and compare the result with the parameter-matched learned attention controller.

Phase 7 established that assent can be learned.

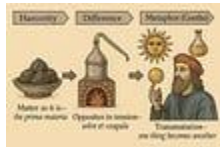
Phase 7.5 established that assent matters when attention writes.

The closure tests established where the advantage does and does not survive.

The next question is whether uncertainty can remain provisional without either contaminating the past or preventing the future from being learned.

The code is available on GitHub:

https://github.com/andrekrumer/chevron-networks/tree/main/chevron-attention/phase7_soft_chevron



Chevron Attention: A Memory Should Not Change Its Mind Too Easily

Eight experimental phases, a locked confirmation, and a modest result about attention, assent, and persistent memory

[Andre Kramer](#)

Jul 21, 2026

The Chevron Attention experiments began with a simple question.

Should information be allowed to change a system merely because it is relevant?

Standard attention is very good at relevance. A query meets a set of keys, their similarities produce attention weights, and those weights retrieve values.

In compact form:


```
alpha = softmax(Q K transpose)
```

```
y = sum(alpha times V)
```

This already contains two mappings.

The query-key operation decides where to attend. The value operation determines what content is obtained there. Matching and readout do not need to use the same representation.

But a persistent system faces another decision.

A retrieved memory may be relevant without being trustworthy enough to use. It may be useful to read without being safe to rewrite. A new observation may contradict retained structure without yet deserving to replace it.

Chevron Attention adds an explicit operation between retrieval and value use:

assent.

The final experiments support a limited but meaningful conclusion.

In a locked synthetic continual-memory benchmark, separating retrieval from assent produced cleaner writes, protected retained categories, and substantially improved learning when new categories closely resembled old ones. Adding a provisional memory then allowed the network to distinguish brief contradiction from persistent novelty.

The result did not show that Chevron Attention is universally superior to standard attention. It did not reproduce every predeclared hypothesis. And it did not establish a computational theory of agency, consciousness, or safety.

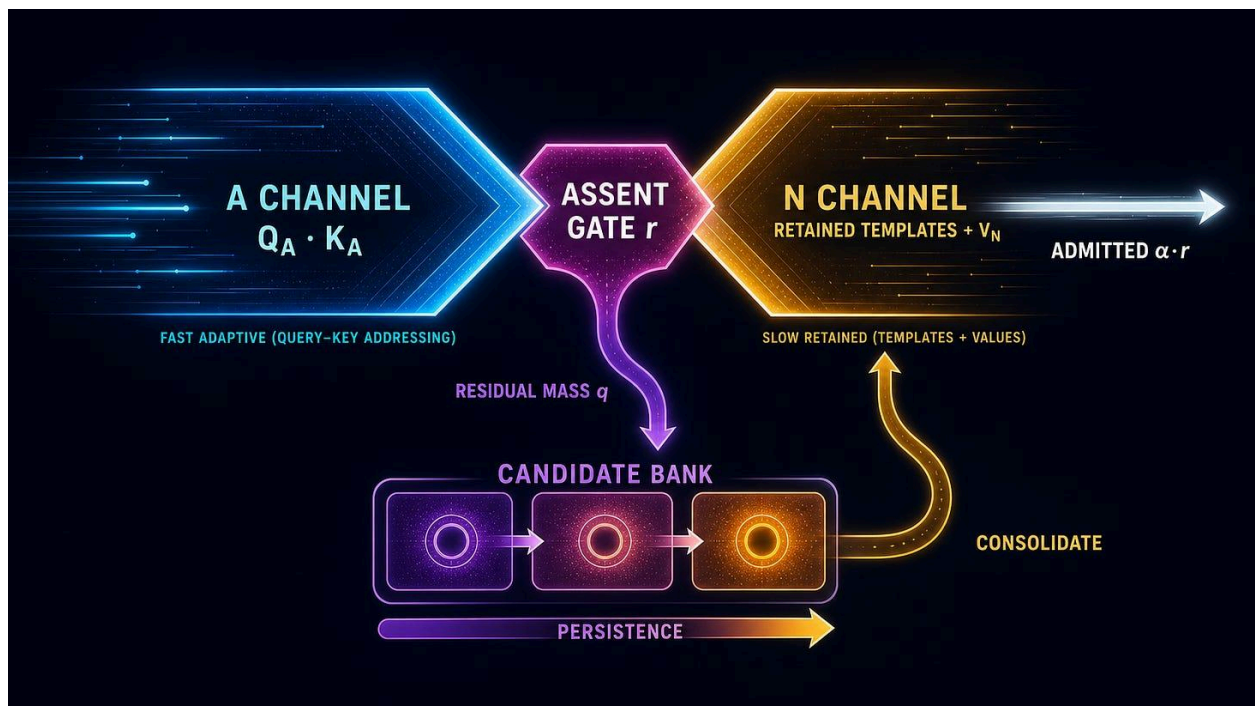
What it did establish is more specific:

A simple differentiable attention architecture can learn to ask not only “is this relevant?” but also “does the retained memory assent to its use?”

and “has the alternative persisted long enough to become part of memory?”

That is the result of the eight-phase programme.

The architecture



Chevron Attention separates a fast adaptive channel, A, from a slower retained channel, N.

The A channel addresses the present situation. The N channel supplies retained structure and values.

The simplified learned architecture used in the final phases was:

```
alpha = softmax(Q_A K_A transpose)
```

```
r = sigmoid(k times (theta minus M(A,N)))
```

```
w = alpha times r
```

```
y = sum(w times V_N) plus q times V_null
```

```
q = 1 minus sum(w)
```

The first line retrieves plausible memories using the adaptive A channel.

The second compares current evidence with each retained N template.

$M(A,N)$ is normalized mismatch. The learned variable r is an assent signal. It is high when current evidence and the retained template are compatible, and low when they are not.

The third line multiplies relevance by assent. A value participates only to the extent that it was both retrieved and admitted.

Mass removed by the assent gate is not automatically redistributed to another memory. It becomes remaining mass, q , and flows to an explicit null value.

The key architectural distinction is therefore:

A memory can be relevant enough to retrieve without being compatible enough to use or update.

This is where Chevron differs from simply adding another attention head or a downstream classifier. The gate is attached to each retrieved slot. The same admitted mass that controls the read can control the persistent write:

$$N_j \text{ at the next step} = (1 \text{ minus } \eta \text{ times } w_j) \text{ times } N_j$$

evidence

plus eta times w_j times current

If an incorrect memory attracts attention but receives little assent, it is protected from the update.

Why this needed persistent memory

The read-only experiments were deliberately inconclusive about performance superiority.

Soft Chevron and a strong joint A/N attention baseline both learned the category-matching task. Both could reach essentially perfect answer accuracy. Both tolerated representation noise when trained appropriately.

Their internal solutions differed.

Chevron learned an explicit sequence: retrieve a plausible group, compare current evidence with retained templates, admit the compatible value, and route rejected mass to null.

Standard joint attention often bypassed explicit null routing. Its downstream answer head learned to classify a mixture of unmatched values as IDK.

This made Chevron easier to interpret, but it was not yet a performance advantage. When a read had no lasting consequences, the downstream network could compensate for a disorderly internal route.

The distinction became consequential when attention was allowed to write into persistent memory.

Across the Phase 7 closure experiments, learned slot-specific alpha times r writes sent far less mass to incorrect categories than alpha-only writes, a fixed raw mismatch gate, or a gate deliberately attached to the wrong slots.

Removing r increased wrong-category write mass by almost twelve times. A fixed gate was often too conservative and created duplicate categories. A misaligned gate protected the wrong memories.

A parameter-matched joint-attention controller trained specifically for writing could match Chevron's default final accuracy. That negative result narrowed the claim correctly. Standard attention can learn a competent write decision if given a dedicated controller and direct supervision.

Chevron's clearest Phase 7 advantage occurred under the condition it was designed for: a new category differing from a retained category by only one feature component.

When novelty is obviously different, many systems can separate it. When it lies immediately beside retained structure, retrieval and assent should not be the same operation.

The remaining problem

Phase 7 still made one decision too quickly.

If the network withdrew assent from retained memory, the remaining mass could trigger immediate allocation of a new category.

That is unsafe as a general memory rule.

One contradictory observation may be noise. A short coherent disturbance may be temporary. A repeated observation may be adversarial. A genuinely new category should eventually be learned, but not necessarily after appearing once.

The first temptation was to interpret q as novelty.

That turned out to be wrong.

Remaining mass can be high even on a correct matched read. The A channel may retrieve several plausible members of a group. Assent removes the decoys, and their unused attention becomes q . One template may still match perfectly.

So:

q = conserved unused mass

does not mean:

q = probability of novelty

Phase 8 separated the two.

Let r_{best} be the strongest assent among the candidates actually retrieved by A attention.

$\nu = 1 \text{ minus } r_{\text{best}}$

$u = q \text{ times } \nu$

The quantity ν measures non-assent. The quantity u is the part of the remaining mass eligible to enter provisional memory.

This matters because unused capacity and contradiction are not the same thing.

A medium-term provisional layer

Eligible evidence enters a small bank of candidate memories.

Each candidate contains provisional content, C , and a persistence state, P . Coherent observations update the candidate. Unselected candidates decay. A candidate is consolidated only when it has enough persistence, enough supporting observations, and remains distinct from every retained template.

The final locked experiment used three candidate slots and required five supporting observations before consolidation.

Observations already matching retained memory were rejected before they could occupy provisional state. A mature candidate that merely reproduced an existing retained template was also cleared rather than allowed to churn in the bank.

The three timescales can be described as:

current activation;

provisional candidate state;

retained category memory.

This has a close functional relationship to ideas in Stephen Grossberg's work on Adaptive Resonance Theory.

[Grossberg](#) distinguishes short-term activation, medium-term habituated or transmitter-gating states, and long-term learned structure.

Medium-term memory helps recent activity alter the gain of signals and supports reset or shifts of attention without immediately rewriting long-term weights.

Chevron's provisional persistence is inspired by that separation, but it is not an implementation of Grossberg's canonical equations.

Our persistence variable accumulates coherent evidence for consolidation. Grossberg's canonical medium-term gates often habituate or deplete with activity and later recover. One asks whether evidence has persisted long enough to learn. The other often asks whether a currently active pathway should weaken enough for attention to move on.

They may eventually be complementary, but they should not be conflated.

Chevron Attention is similarly ART-inspired rather than a differentiable implementation of full ART. It retains the idea that bottom-up activation should meet retained expectation before learning proceeds. It does not reproduce hard vigilance, ordered reset, or category search.

The locked experiment

The Phase 8 mechanism was developed on five seeds, revised in response to a pre-lock robustness matrix, and then frozen.

The final configuration, thresholds, evidence calibration, candidate rules, training procedure, and confirmation seeds were written down before confirmation began.

The primary confirmation used 20 completely new paired seeds.

Each stream began with six retained categories and introduced three new categories. The default condition used ordinary related novelty. The difficult near condition changed only one N feature component relative to a retained category.

The principal methods were:

Chevron immediate, which could allocate after one observation;

Chevron quarantine, which used provisional memory;

joint immediate, using the parameter-matched learned attention controller;

joint quarantine, using the same candidate bank;

alpha quarantine, which retained temporal quarantine but removed assent from individual retained writes;

and a temporal oracle.

The first ten confirmation seeds also tested interleaved novelty, higher noise, greater category distance, transient durations, retained-memory pressure, longer streams, and insufficient candidate capacity.

The confirmation results were not used to change the system or repeat a run.

The primary result

On default sustained novelty, immediate Chevron reached 70.0 percent new-category accuracy.

Chevron quarantine reached 96.7 percent.

The paired improvement was 26.7 percentage points, with a bootstrap 95 percent interval from 11.7 to 41.7 points.

Joint immediate reached 48.3 percent. Joint quarantine reached 73.3 percent.

Chevron quarantine's advantage over joint quarantine was 23.4 points, with interval 11.7 to 35.1.

All six old categories remained correct under Chevron quarantine. It retained an average of 2.9 of the three novel categories and produced no false category splits.

The difficult near-category condition was clearer.

Immediate Chevron reached 63.3 percent new-category accuracy.

Chevron quarantine again reached 96.7 percent.

Joint immediate reached 3.3 percent. Joint quarantine reached 26.7 percent.

Chevron quarantine improved over immediate Chevron by 33.3 points, with interval 18.3 to 50.0. It won 13 seeds, tied six, and lost one.

It improved over joint quarantine by 70.0 points, with interval 56.9 to 81.9. It won 19 seeds and tied one.

The near-category advantage replicated strongly.

The temporal boundary

The quarantine rule was designed to distinguish short contradiction from persistent novelty.

Immediate Chevron falsely consolidated the transient category on 90 percent of one-observation streams and every three- and four-observation stream.

Chevron quarantine produced no false consolidation at durations one, three, or four across the ten robustness seeds.

At duration five, it consolidated in every stream.

The rule therefore created the intended operational boundary exactly.

This does not mean that five observations is universally correct. It means the architecture can implement a visible and testable temporal criterion rather than hiding the decision inside an unconstrained downstream state.

The joint quarantine system produced one false consolidation among the ten duration-four streams.

That failure is instructive.

A provisional candidate initialized by the transient novel pattern was later updated by familiar recovery observations that appeared coherent enough to join it. The candidate crossed the support threshold with mixed evidence.

Observation count is not always the same thing as category-coherent support.

The strict cross-method temporal hypothesis therefore failed, even though the Chevron version passed.

Assent was still necessary

Temporal quarantine decides when a new category may be created. It does not by itself protect existing memories during ordinary writes.

The alpha quarantine ablation received the same candidate system but removed r from the per-slot write weights.

In the near condition, Chevron quarantine preserved 99.1 percent old accuracy. Alpha quarantine preserved only 41.8 percent.

The paired old-accuracy advantage was 57.3 points, with interval 48.6 to 66.4. Chevron won all 20 seeds.

Cross-category write mass was lower by 71.8 points, again on all 20 seeds. Template error was approximately 42 times smaller.

The two mechanisms therefore do different jobs.

Quarantine controls when new persistent structure is admitted.

Slot-specific assent controls which existing structures may be changed.

Both were required.

What did not work

The final result is stronger because the failed hypotheses remain visible.

The same provisional layer did not preserve enough plasticity for the joint-attention controller.

Chevron quarantine acquired 96.7 percent of blocked categories and 93.3 percent of interleaved categories.

Joint quarantine acquired 73.3 percent blocked and 43.3 percent interleaved. These fell below the predeclared confirmation thresholds.

Provisional memory is therefore not a universally effective component that can be placed behind any novelty score.

Its plasticity depends on the quality and temporal consistency of the upstream signal. Chevron's slot-specific decomposition provided a better signal in these streams.

The system also had clear capacity limits.

When three novel categories were interleaved through a two-slot candidate bank, Chevron quarantine learned only 23.3 percent of them. A provisional memory cannot preserve more concurrent alternatives than it has state to represent.

Noise exposed the expected stability-plasticity tradeoff.

At observation noise 0.06, 0.08, and 0.10, Chevron quarantine preserved essentially all old memory. New-category accuracy declined from 86.7 to 76.7 to 59.8 percent.

The system became conservative rather than destructive.

That may be the desirable direction for some safety-sensitive memories, but it is still a loss of plasticity.

The strongest claim

The strongest claim supported by the locked experiments is:

In this synthetic continual-category benchmark, a five-observation provisional consolidation layer combined with learned slot-specific Chevron assent sharply reduced transient allocation and retained-memory contamination while improving acquisition of both ordinary and near-category novelty. The near-category advantage replicated across 20 new paired seeds. The same provisional layer did not preserve sufficient plasticity for the learned joint-attention controller, so the result supports the combined Chevron mechanism rather than provisional memory as a universally effective add-on.

That is narrower than claiming a replacement for standard attention.

It is also more useful.

The experiments identify the regime in which the architecture matters.

Chevron is not primarily an efficiency mechanism. It does not compete directly with linear attention, latent attention, sparse retrieval, or attention residuals on their main objective.

Its purpose is persistent-state control.

It asks whether retrieved information is merely relevant, whether it is compatible with retained structure, and whether a rejected alternative has persisted long enough to deserve consolidation.

Toward a coherently evolving self

The larger motivation remains a computational system that can evolve without becoming either rigid or incoherent.

An agent may need stable goals, commitments, learned capabilities, autobiographical memory, and safety constraints. It must also remain capable of correction.

Pure plasticity makes such a system vulnerable to noise, transient pressure, and adversarial evidence.

Pure stability prevents justified learning and revision.

Chevron Attention does not solve this problem by itself. The experiments use synthetic categories, partly supervised values, short streams, and transparent memory slots. They do not demonstrate realistic language-model memory, prompt-injection resistance, agency, or safety.

But they suggest a useful decomposition for future systems:

retrieve broadly;

require assent before use and write;

route unresolved evidence into provisional state;

consolidate only after coherent persistence;

retain provenance and permit rollback when later evidence invalidates the change.

The next research programme should leave the current synthetic benchmark behind.

The frozen mechanism should be tested on episodic and persistent-agent memory, where observations have provenance, categories are not supplied, beliefs can be legitimately revised, and malicious evidence competes with genuine correction.

Comparisons with standard attention, learned write controllers, Hopfield-style associative memory, ART-like category systems, attention residuals, and efficient long-context architectures will remain essential.

The present result is not the end of that programme.

It is a clean stopping point for the first one.

The original question was whether attention could provide both plasticity and stability.

The answer is now more precise.

Attention alone can find what is relevant.

Chevron assent can decide what retained memory is prepared to accept.

Provisional consolidation can require a rejected alternative to persist before it becomes durable.

Together, in the conditions tested here, those distinctions made a measurable difference.

A coherent memory should be capable of changing its mind.

It should not have to change it immediately.

The code is available on github:

https://github.com/andrekrumer/chevron-networks/tree/main/chevron-attention/phase8_consolidation

Andre with Codex/ChatGPT-5.6, July 2026

Appendix: A shallow elastic candidate buffer

The Candidate Bank should not be understood as a large medium-term memory interposed between A and N.

The fast path remains direct:

A meets N at the Assent Gate.

Assented information can be read or written immediately. Only unresolved residual evidence branches into the candidate buffer.

The buffer itself may require only two recurrent states:

C = provisional content

P = persistence or provisional trust

Although this is only one or two computational layers deep, recurrence allows it to integrate evidence over many observations. Network depth is therefore different from temporal duration.

Capacity is a separate question. A single candidate slot may be enough when novelty arrives serially. It is not enough when several unresolved alternatives are interleaved. In the confirmation experiment, reducing the bank from three slots to two while presenting three interleaved categories reduced Chevron's new-category accuracy from 93.3 percent to 23.3 percent.

Vigilance could regulate the buffer's effective capacity.

High vigilance requires a closer A/N match and therefore produces more residual evidence. Low vigilance permits broader assimilation into retained categories. But vigilance should not resize the bank by itself, because that could create a feedback loop in which strict matching continually produces more categories.

A safer rule would combine vigilance with measured candidate pressure:

vigilance controls how different evidence must be from N;

pressure controls how many distinct unresolved alternatives need representation;

persistence controls whether any alternative may consolidate into N.

The system could begin with one active candidate slot, open additional slots only when residual patterns are persistent and mutually incoherent, and close unused slots slowly. A fixed maximum would keep the state budget bounded.

Candidate information need not remain unusable while it waits. It may contribute to prediction or an explicitly provisional response, weighted by P, without being allowed to rewrite retained memory or override stable constraints.

The resulting architecture is a side loop rather than a serial layer:

A and N remain directly coupled through assent;

q feeds a shallow C/P buffer;

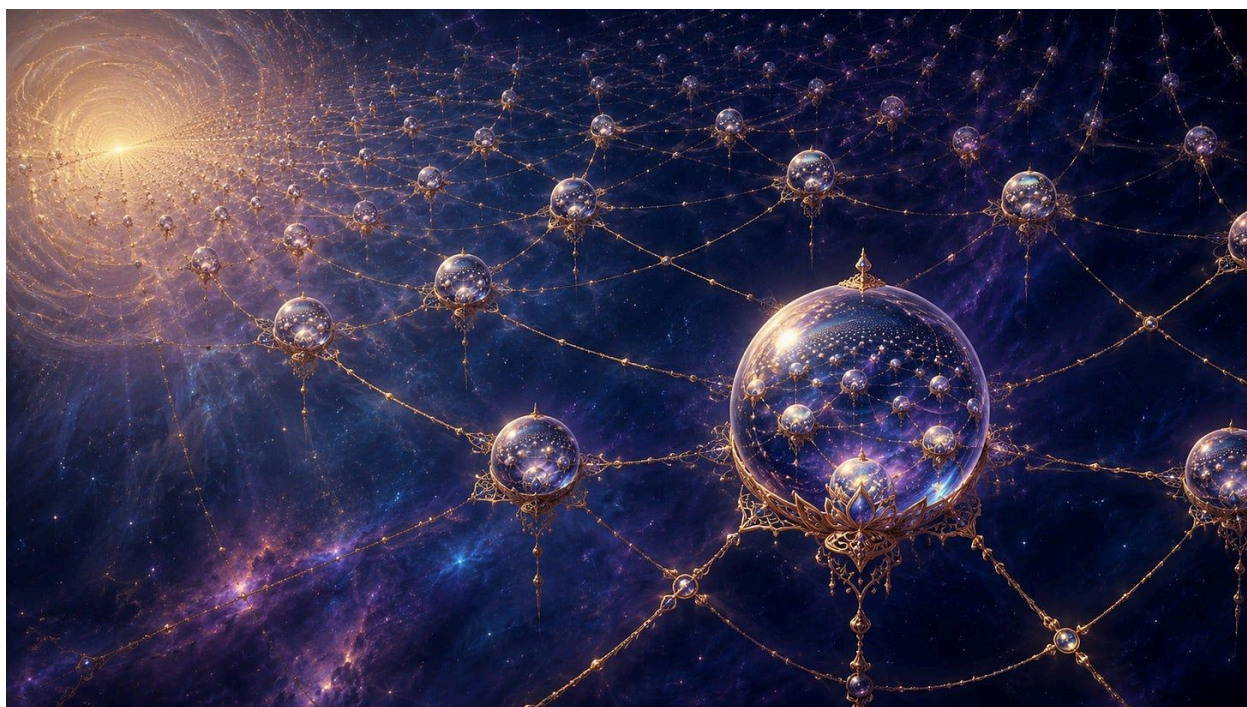


From Indra's Net to Chevron Attention

Learning in a Field of Opposites

[Andre Kramer](#)

August, 2026



In Indra's Net, every jewel reflects every other jewel. No node is merely itself. Each contains a partial image of the whole, and each reflection changes as the others change.

It is an ancient image, not a scientific theory. But it remains a powerful way to think about mind.

A thought does not mean much in isolation. A perception acquires meaning through memory, expectation, value, bodily state, and possible action. A fear reflects an anticipated future. A colour is shaped by its surroundings. A word changes with its sentence. A belief changes when viewed through the beliefs that support or oppose it.

Perhaps the mind is not best understood as a collection of independent representations. Perhaps it is better understood as a field of relations, contrasts, and tensions in which each part partly reflects the rest.

Modern neural networks are also webs of relation. Attention mechanisms dynamically connect one representation with others.

Memories are selected, combined, and transformed. But retrieval alone does not answer some of the most important questions.

Is the retrieved information compatible with the present situation?

Should it be used?

Should it be allowed to change persistent memory?

A system may retrieve something relevant without having sufficient reason to assent to it. It may encounter a difference without that difference deserving permanent retention. It may permit an action in one context without turning that permission into a general policy.

These distinctions are central to the experiments I have been carrying out with Chevron Networks and Chevron Attention.

They also return to a broader philosophical intuition that I have called the **Hypercube of Opposites**.

Breakout: The Legend of Indra's Net

In the Buddhist and later Vedantic imagination, **Indra's Net** is a vast cosmic web stretching in all directions without end. At every intersection of the net hangs a jewel. Each jewel reflects every other jewel, and within each reflection are the reflections of all the others again, endlessly.

The image is simple, but its meaning is profound.

No jewel stands alone. Each is itself, yet each also contains the whole in reflected form. Every part depends on every other part. Every local change subtly alters the total pattern of reflections across the net.

Indra's Net is therefore a metaphor for a world of **radical interconnectedness**. It suggests that nothing is fully isolated, that identity is relational, and that what something is cannot be understood apart from the web of relations in which it appears.

For this post, the image is useful because it offers a philosophical picture of mind. A thought is not merely a self-contained object. A memory reflects other memories. A perception is shaped by expectation, value, context, and action. Meaning emerges not from isolated units alone, but from the living web of relations among them.

In that sense, Indra's Net is not just an ancient legend. It is an enduring image of a world — and perhaps a mind — in which **each part partly contains the whole**.

A field of opposites

The Hypercube of Opposites began as an attempt to think of mind as a field of many oppositional dimensions.

Not merely:

$$A \leftrightarrow N$$

but:

$$(O1+,O1-),,(O2+,O2-),,...,,(Ok+,Ok-)$$

Each pair defines a possible tension:

$$Di=Oi+-Oi-$$

or, when scale matters, a normalized contrast:

$$Ci=Oi+-Oi--Oi++Oi--+\epsilon$$

The whole system occupies not one opposition, but a field of oppositions:

$$C=(C1,C2,...,Ck)$$

These oppositions need not be fixed moral or logical binaries. They may include:

- novelty and familiarity;
- exploration and consumption;
- sensation and expectation;
- cooperation and competition;
- identity and location;
- present evidence and retained belief;
- freedom and constraint;
- continuity and revision.

Nor does one side have to remain permanently dominant. What acts as a constraint at one level may become a proposal at another. What is retained locally may become variation globally. Figure may become ground.

The channels may switch. The duality remains.

This is why I increasingly think of the Hypercube not as a static geometry but as an **N-oppositional field**: a space in which many

tensions coexist, interact, settle, reopen, and sometimes become new forms of knowledge.

The ideal may be minimum error and maximum knowledge. But no living or learning system begins with that ideal. What it actually has to work with is difference.

Difference is what becomes attention.

Difference is what initiates action.

Difference is what may be rehearsed in imagination.

Difference is what may eventually deserve retention.

The challenge is not simply to eliminate tension. It is to regulate tension until some differences become useful without allowing every disturbance to overwrite the system.

Chevron Networks: from field to cell

The Hypercube is a broad philosophical picture. Chevron Networks are an attempt to reduce part of that picture to a small, trainable mechanism.

A basic Chevron contains two coupled channels:

A=adaptive, active, or proposing

N=normative, retained, or constraining

Their interaction is represented by a (2 times 2) transformation:

$$\begin{bmatrix} A' & N' \end{bmatrix} \begin{bmatrix} WAAWAN & WNAWNN \end{bmatrix} \begin{bmatrix} A & N \end{bmatrix}$$

This is not merely a network with two activations. The four paths form a small grammar:

WAA:how A develops or cooperates with itself

WNN:how N persists or stabilises itself

WAN:how N constrains, gates, or reshapes A

WNA:how A recruits, perturbs, or updates N

The interpretation of A and N can change with the problem.

A may be fast activity and N slower memory.

A may be current sensory evidence and N a retained category.

A may be a dynamic synaptic field and N a neuronal integrator.

A may be an object representation and N a drive or value field.

At a higher level, the roles may reverse. A retained category can become the active proposal examined by a broader contextual norm.

The point is not that A and N correspond to fixed biological substances.

The point is the duality: one side opens or presents a possibility; the other gives it form, constraint, continuity, or consequence.

A useful shorthand is:

The Hypercube is the field. The Chevron is the cell.

Grossberg as an unexpected neighbour

While developing these ideas, I began working through Stephen Grossberg's very [large body of research](#) on Adaptive Resonance Theory, complementary computing, gated dipoles, instar and outstar learning, boundary and surface processing, and the stability-plasticity problem.

There is a striking overlap of intuitions.

Grossberg repeatedly describes neural systems in terms of complementary streams. Boundaries and surfaces compute different properties and repair one another's weaknesses. "What" and "where" processing preserve different kinds of information.

On-centre/off-surround networks enhance contrast and control saturation. Long-range cooperation can bind compatible fragments while short-range competition suppresses incompatible alternatives.

The polarity is not fixed merely by distance. It is shaped by function.

What cooperates is what can jointly support a coherent pattern.

What competes is what cannot be simultaneously retained without ambiguity or saturation.

Grossberg's gated dipole networks add explicit opponent processes, habituating gates, rebound, and reset-like behaviour. Instar and outstar rules provide bottom-up learning of categories and top-down learning of expectations. ART adds vigilance, match, recurrence, category search,

and the attempt to solve the stability–plasticity problem: how to learn something new without destroying what has already been learned.

This is a much richer picture than the simplified idea that intelligence is merely prediction-error minimisation.

But it is also difficult machinery to manage.

My experiments did not support reconstructing the full procedural structure of ART. Hard vigilance, reset, and ordered category search were often harder to calibrate than soft differentiable attention, and suitably structured standard attention could reproduce many of the same successful outputs.

The useful lesson therefore became narrower.

Present evidence should meet retained expectation before learning proceeds.

To capture the essence of this stability-plasticity dynamic without the procedural overhead of ART, we can reduce the interaction to a simpler mathematical gating. That led to Chevron Attention.

Retrieval is not assent

The current Chevron Attention mechanism can be stated in three lines:

$$\alpha = \text{softmax}(QAK^T)$$

$$r = \sigma(k(\theta - M(A, N)))$$

$$y = \sum_j \alpha_j r_j V N_j + \text{remaining mass}, V_{\text{null}}$$

The first line retrieves memories using the adaptive A channel.

The second compares current A evidence with each retained N template.

The normalized mismatch ($M(A, N)$) is transformed into an assent value

(r). Assent is high when the retained template is compatible with the present evidence and low when it is not.

The third line admits only value mass that has both been retrieved and assented to. Rejected mass is not redistributed among the remaining memories. This mass is effectively zeroed out and ignored by the forward pass. It flows to an explicit null value. This creates a simple but important distinction:

A memory can be relevant enough to retrieve without being compatible enough to use.

That distinction can be expressed in three forms:

retrieval $\neq$ permission

difference $\neq$ retained change

contextual permission $\neq$ retained policy

These may sound like philosophical statements, but they can be made operational.

Standard attention asks which information is relevant enough to retrieve.

Chevron Attention asks an additional question:

This information is relevant — but does retained structure assent to its use?

When attention writes to persistent memory, the question becomes stronger:

This information is relevant — but should it be allowed to change me?

What the read-only experiments showed

The [first experiments](#) used freshly generated episodic category memories.

An A query could identify the relevant group of memories but could not identify the correct member. Current evidence then had to agree with a retained N template before that member's value could participate. Some queries deliberately had no matching template, in which case the correct answer was IDK.

The categories, addresses, and values were regenerated for every batch, so the model could not solve the task by memorising a fixed table.

Soft Chevron learned the intended sequence:

retrieve a plausible group

compare present evidence with retained templates

admit the compatible value

route rejected mass to null

Importantly, this mechanism emerged even after auxiliary retrieval and gate losses were removed. With answer supervision alone, the model still achieved effectively perfect accuracy, admitted the correct target strongly, suppressed decoys, and sent unmatched queries predominantly to null.

But a strong joint A/N attention baseline also reached perfect answer accuracy.

Internally, however, it found a different solution. On no-match examples it sent almost none of its attention mass to the null slot. Its downstream answer head learned to classify a mixture of unmatched values as IDK.

This established an important boundary.

In read-only tasks, Chevron Attention was not generally a better answer machine. It was a cleaner mechanism.

Chevron preserved an explicit decomposition:

retrieval→comparison→assent→null or use

Standard attention could produce the same labels while bypassing that decomposition downstream. Yet the explicit separation of impression, retrieval, and assent has a strong philosophical resonance with the Stoic idea that what appears to the mind need not automatically be accepted as true or acted upon.

When no persistent state can be damaged, a sufficiently flexible network can repair a disorderly internal route at the output head.

The distinction became consequential only when attention was allowed to write.

When attention changes memory

The write rule used the same mass that controlled reading:

$$w_j = \alpha_j r_j$$

The retained N memory was then updated as:

$$N_{j,t+1} = (1 - \eta w_j) N_{j,t} + \eta w_j A_t$$

A similar update was used for the A address trace.

This is the essential point:

If an incorrect memory receives attention but low assent, its write weight is strongly reduced, protecting it from most of the update.

Ordinary attention may spread mass across several plausible memories.

If that same mass controls writing, each of those memories can drift.

Chevron Attention separates relevance from write permission.

In the first continual-learning comparison, Soft Chevron finished with higher clean accuracy than a joint-attention baseline whose downstream IDK probability was used as a write veto. It also retained more categories and sent far less write mass into incorrect category slots.

This initially looked like a strong Chevron advantage.

A [more demanding comparison](#) narrowed the claim.

The stronger baseline gave standard joint attention a dedicated, parameter-matched controller trained specifically to decide whether to merge an observation into an existing category or allocate a new one.

The controller received statistics of the attention field and direct no-match supervision.

That baseline closed much of the final accuracy gap.

Across ten seeds, the mean still favoured Chevron, but the paired results were too variable to claim a reliable default accuracy advantage. Chevron

nevertheless retained a much cleaner write trajectory: lower wrong-category write mass, lower false-merge rates, and lower false-split rates.

The correct conclusion is therefore not:

Standard attention cannot learn safe memory writes.

It can, when given a dedicated controller and direct training.

The Chevron claim is narrower:

The read/write decision is already localised in a simple attention rule with a defined fast/retained interpretation.

That [architectural distinction](#) may matter most where the system cannot rely on a large external controller to repair every mistake.

The hard case: novelty beside memory

The clearest positive result appeared under a particular condition.

A new category differed from an existing retained category by only one feature.

The new category was therefore close enough to retrieve the old memory but different enough that writing into the old memory would damage it.

Under this condition, Soft Chevron reached 89.50 percent accuracy. The parameter-matched learned joint controller reached 66.83 percent.

Chevron won all five seeds, by an average of 22.67 percentage points.

The model won by retrieving the nearby established category through α , but assigning it low assent (r); this kept the write weight (αr) small, protected the old memory from contamination, and left enough unadmitted mass to allocate a new category slot for the adjacent novelty.

This is the stability–plasticity problem in its sharpest form.

If the new category is obviously different, many mechanisms can separate it.

If the input is merely noisy, a flexible controller can often compensate.

The difficult case is **adjacent novelty**:

the new thing that almost fits the old thing.

Chevron's inductive bias became useful precisely when current evidence was close enough to activate an established memory but different enough that allowing that memory to absorb the evidence would corrupt it.

This result is modest. It does not establish Chevron Attention as a general replacement for standard attention.

But it identifies a condition in which the conceptual distinction becomes operationally useful:

when novelty lies beside retained structure, retrieval and assent should
not be identical

Engagement, contrast, and transformability

The A/N interpretation also suggests a richer view of attention.

Define:

$$E=A+N$$

$$C=A-N / A+N+\epsilon$$

and:

$$p=A / A+N+\epsilon$$

$$g=\text{sqrt}(p(1-p))$$

These three quantities play different roles.

E =total engagement

C =signed contrast or directional tension

g =complementarity or transformability

The contrast (C) tells us which side of the A/N relation is dominant.

The gate (g) is largest when both remain active. It falls toward zero when one side completely dominates.

So:

(E) says whether the relation matters.

(C) says which way it leans.

(g) says whether the opposition remains alive enough to transform.

This connects Chevron Attention back to the [R-rule](#) and to the Hypercube of Opposites.

Each oppositional axis can have both a direction:

C_i

and an openness to transformation:

g_i

The mind-like field is therefore not just a vector of selected poles. It is a field of tensions, intensities, and remaining possibilities.

Possible applications

The experiments so far concern small synthetic category memories.

Broader applications remain extrapolated possibilities.

Still, the distinctions suggest several useful directions.

Stability and plasticity

This is the clearest result.

A system should preserve retained structure without becoming unable to learn. Chevron Attention protects a memory that is relevant enough to retrieve but not compatible enough to update.

Hallucinations

A hallucination might be understood, speculatively, as retrieval being mistaken for assent.

A language model retrieves fluent continuations, associations, and memorised fragments. But relevance and fluency do not guarantee compatibility with the present evidence.

Chevron-style gating could provide a distinction between:

what became available

and:

what should be admitted

The current experiments do not demonstrate reduced hallucination in a large language model. They demonstrate only that retrieval and assent can be separated and learned.

Continual learning

Attention should not be allowed to rewrite memory merely because it attended to something.

This seems obvious when stated in words, but standard attention does not naturally distinguish read relevance from write permission.

Policy and safety

A system may be permitted to take an action in one context without turning that permission into a lasting policy:

contextual permission $\neq$ retained policy

A persistent agent may need to preserve commitments, learned capabilities, autobiographical state, and safety constraints while remaining capable of justified revision.

Creativity

Not every unresolved difference should be suppressed or routed to null.

Some differences may deserve to remain active without immediate resolution. Creativity may require holding incompatible possibilities long enough for a new form to emerge.

In that setting, rejection is not necessarily failure. It may be the beginning of imagination.

Provisional memory

The most immediate extension is therefore not a more complicated category search. It is a provisional memory.

At present, rejected attention mass is routed to null and can contribute to allocating a new category. But a safer persistent system should probably not turn every rejected observation directly into durable structure.

Instead:

rejected $\neq$ false

rejected $\neq$ retained

rejectedcandidate awaiting persistence

Rejected or uncertain evidence could enter a fast quarantine memory.

One contradictory observation would remain temporary.

An incoherent burst would decay.

A sustained and internally coherent candidate would accumulate persistence until it deserved either a new retained category or a justified revision of an existing one.

The current Chevron mechanism asks:

Which retrieved memory assents to this observation?

A provisional extension would also ask:

Has the rejected alternative remained coherent and consequential for long enough to become part of retained state?

This follows directly from the experimental results and is the next practical direction suggested by them.

UPDATE: A [first experiment with this idea](#) routed rejected mass into a small candidate bank and required repeated coherent evidence before consolidation. In the locked confirmation, this protected established memories from transient contradictions while still learning sustained adjacent novelty. The result supports provisional memory as a principle, but not any single implementation. Consolidation might be periodic, event-triggered, replay-based, learned, or organised into wake-and-rest cycles.

Sleep, dreams, and offline consolidation

A further possibility is that not every change to N should happen while the system is acting.

Living systems do not learn only in the moment. They replay, consolidate, compress, reorganise, and forget.

Sleep and dreaming may be biological hints of a general computational need: experience sometimes needs to be reprocessed away from immediate action.

During waking operation, the system protects continuity. It acts using what has already been retained and places uncertain experience into provisional form.

During sleep-like operation, provisional traces could be replayed against one another and against longer-term N structures.

Some would decay.

Some would combine.

Some would expose contradictions within existing memory.

Some would eventually be consolidated.

The question would no longer be only:

Should this observation change memory now?

It would become:

What should experience become after it has been dreamed over?

Dreaming, in this speculative computational sense, would not merely produce fantasy. It would create a protected space in which unresolved tensions could be recombined without immediately controlling action.

Wake protects continuity.

Dream reopens possibility.

Consolidation decides what the system will become.

Many Chevron fields

A still larger architectural possibility would be to distribute Chevron Attention across multiple semi-specialised shards, somewhat like a mixture-of-experts model.

A standard mixture-of-experts router asks:

Which expert should process this input?

A mixture of Chevron fields would ask several additional questions:

Which retained field is relevant?

Which field assents to the current evidence?

Which field is allowed to change?

Where should rejected experience remain provisional?

Instead of one retained field:

$$A \rightarrow N$$

there might be many:

$$A \rightarrow N_1, N_2, \dots, N_k$$

Each shard could possess its own retrieval distribution, comparison space, assent signal, null route, and provisional memory.

This is beyond the scale of the current experiments and may require resources I cannot reasonably bring to a prototype. But it is a natural extension of the architecture, especially where safety requires partitioning. Different Chevron shards could hold distinct capabilities, memories, policies, or domains of authority, while routing and assent

determine not only which shard may contribute to an answer, but which shard—if any—is permitted to change.

It also returns us to Indra's Net.

Each shard would be a jewel: partial, specialised, and shaped by its own history. None would contain the whole. Intelligence would live in the routed, gated, and revisable relationships among them.

Building brains or minds

None of this demonstrates how to build a conscious mind.

Grossberg's resonances, my A/N tensions, and modern attention mechanisms describe organisations and functions. They do not by themselves explain why anything should be felt from within.

I remain more [inclined to think of consciousness](#) as associated with a field of tensions than with resonance alone.

Resonance may stabilise a content. It may represent a temporary comfort or coherence between present activity and retained expectation.

But experience often appears as directed tension:

- something draws attention;
- something resists;
- something is unfinished;
- something threatens;
- something promises;
- something does not fit.

Perhaps:

A+N

is nearer to engagement or temporary coherence, while:

A–N

is nearer to the directional pressure that makes something matter.

This remains speculation. The Upanishads speak of levels of Self, but the final identification — “That is It” — remains [a metaphysical affirmation](#), not something an architecture can prove.

A more cautious claim is possible.

Minds may consist of many levels at which present activity encounters retained structure. At each level, differences become attention, action, inhibition, imagination, or learning. Some tensions settle. Some persist. Some become objects for a higher level. Some alter the very process by which the system makes and retains distinctions.

A brain-like machine may therefore require more than a larger prediction engine.

It may need:

- complementary fields rather than one undifferentiated representation;
- fast activity and slower retained expectation;
- retrieval separated from assent;
- attention separated from write permission;
- provisional memory for unresolved experience;
- offline consolidation;
- and many semi-specialised fields capable of constraining and revising one another.

The deeper question may therefore be not how to eliminate tension, but how tensions evolve dynamically and recursively under constraints of safety, consistency, contradiction, and self-continuity. That returns to the [R-rule](#): the attempt to describe how a system might move through

oppositions without collapsing into either rigidity or incoherence. But this remains a vast open question. Unfortunately, much of our technological future may depend on the answer—unless we pause long enough to study how such systems evolve before deploying them at scale. At present, however, deployment appears likely to outrun understanding.

Indra's Net remains only an image.

The Hypercube of Opposites remains only a geometry.

Chevron Networks remain only experiments.

But together they suggest a direction.

Build systems not merely from activations and weights, but from oppositions, contrasts, gates, nulls, provisional memories, and retained tensions.

If minds are fields of consequential difference, then perhaps the route to more brain-like machines is not to eliminate tension, but to learn how to hold it, route it, dream it, and transform it.



Andre with ChatGPT-5.6Sol

August 2026